Data_Exploration_Preparation_Spambase

February 19, 2026

1 Stampbase Dataset

- **The first steps on this work on 'stampbase' dataset were import the libraries and upload the csv file to analyse the dataset shape, duplicates and null values.**

[1]:
```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
import numpy as np
```

[2]:
```
data=pd.read_csv('spambase.csv')
```

[3]:
```
data.shape
```

[3]:
```
(4601, 59)
```

[4]:
```
data.head()
```

[4]:

	Unnamed: 0	word_freq_make	word_freq_address	word_freq_all	word_freq_3d	\
0	0	0.00	0.64	0.64	0.0	
1	1	0.21	0.28	0.50	0.0	
2	2	0.06	0.00	0.71	0.0	
3	3	0.00	0.00	0.00	0.0	
4	4	0.00	0.00	0.00	0.0	

	word_freq_our	word_freq_over	word_freq_remove	word_freq_internet	\
0	0.32	0.00	0.00	0.00	
1	0.14	0.28	0.21	0.07	
2	1.23	0.19	0.19	0.12	
3	0.63	0.00	0.31	0.63	
4	0.63	0.00	0.31	0.63	

	word_freq_order	…	char_freq_;	char_freq_(	char_freq_[	char_freq_!	\
0	0.00	…	0.00	0.000	0.0	0.778	
1	0.00	…	0.00	0.132	0.0	0.372	
2	0.64	…	0.01	0.143	0.0	0.276	
3	0.31	…	0.00	0.137	0.0	0.137	
4	0.31	…	0.00	0.135	0.0	0.135	

```
   char_freq_$  char_freq_#  capital_run_length_average  \
0        0.000        0.000                       3.756
1        0.180        0.048                       5.114
2        0.184        0.010                       9.821
3        0.000        0.000                       3.537
4        0.000        0.000                       3.537

   capital_run_length_longest  capital_run_length_total  is_spam
0                          61                       278     True
1                         101                      1028     True
2                         485                      2259     True
3                          40                       191     True
4                          40                       191     True

[5 rows x 59 columns]
```

[5]:
```
df=data.drop(columns='Unnamed: 0')
```

[6]:
```
df.duplicated().sum()
```

[6]:
```
383
```

[7]:
```
dff=df.drop_duplicates().reset_index(drop=True)
```

[8]:
```
dff.duplicated().sum()
```

[8]:
```
0
```

[9]:
```
dff.isnull().sum()
```

[9]:
```
word_freq_make                0
word_freq_address             0
word_freq_all                 2
word_freq_3d                  2
word_freq_our                 0
word_freq_over                1
word_freq_remove              0
word_freq_internet           14
word_freq_order               0
word_freq_mail                0
word_freq_receive             0
word_freq_will                0
word_freq_people              0
word_freq_report              0
word_freq_addresses           9
word_freq_free                0
word_freq_business            0
```

```
word_freq_email                 0
word_freq_you                   0
word_freq_credit                0
word_freq_your                  0
word_freq_font                  0
word_freq_000                   0
word_freq_money                 0
word_freq_hp                    0
word_freq_hpl                   0
word_freq_george                0
word_freq_650                   0
word_freq_lab                   0
word_freq_labs                231
word_freq_telnet                0
word_freq_857                   0
word_freq_data                  0
word_freq_415                   0
word_freq_85                    0
word_freq_technology            0
word_freq_1999                  0
word_freq_parts                 0
word_freq_pm                    0
word_freq_direct                3
word_freq_cs                   14
word_freq_meeting               1
word_freq_original              0
word_freq_project               0
word_freq_re                    0
word_freq_edu                   0
word_freq_table                40
word_freq_conference            0
char_freq_;                     0
char_freq_(                     0
char_freq_[                     0
char_freq_!                     0
char_freq_$                     0
char_freq_#                     0
capital_run_length_average      0
capital_run_length_longest      0
capital_run_length_total        0
is_spam                         0
dtype: int64
```

[10]:
```
dff.fillna(0, inplace=True)
```

[11]:
```
dff.isnull().sum()
```

[11]:
```
word_freq_make              0
word_freq_address           0
word_freq_all               0
word_freq_3d                0
word_freq_our               0
word_freq_over              0
word_freq_remove            0
word_freq_internet          0
word_freq_order             0
word_freq_mail              0
word_freq_receive           0
word_freq_will              0
word_freq_people            0
word_freq_report            0
word_freq_addresses         0
word_freq_free              0
word_freq_business          0
word_freq_email             0
word_freq_you               0
word_freq_credit            0
word_freq_your              0
word_freq_font              0
word_freq_000               0
word_freq_money             0
word_freq_hp                0
word_freq_hpl               0
word_freq_george            0
word_freq_650               0
word_freq_lab               0
word_freq_labs              0
word_freq_telnet            0
word_freq_857               0
word_freq_data              0
word_freq_415               0
word_freq_85                0
word_freq_technology        0
word_freq_1999              0
word_freq_parts             0
word_freq_pm                0
word_freq_direct            0
word_freq_cs                0
word_freq_meeting           0
word_freq_original          0
word_freq_project           0
word_freq_re                0
word_freq_edu               0
word_freq_table             0
```

```
word_freq_conference          0
char_freq_;                   0
char_freq_(                   0
char_freq_[                   0
char_freq_!                   0
char_freq_$                   0
char_freq_#                   0
capital_run_length_average    0
capital_run_length_longest    0
capital_run_length_total      0
is_spam                       0
dtype: int64
```

- **After drop the unnamed column on the dataset, first I have checked for the duplicates values on the dataset and dropped then.**
- **Then I've checked for null values on the dataset, as the features present in the dataset are numerics, and they count the frequence of each feature, I concluded the null values are equal to zero, so I've filled then with the number zero.**

[12]:
```
dff
```

[12]:
```
      word_freq_make  word_freq_address  word_freq_all  word_freq_3d  \
0               0.00               0.64           0.64           0.0
1               0.21               0.28           0.50           0.0
2               0.06               0.00           0.71           0.0
3               0.00               0.00           0.00           0.0
4               0.00               0.00           0.00           0.0
…                …                  …              …              …
4213            0.31               0.00           0.62           0.0
4214            0.00               0.00           0.00           0.0
4215            0.30               0.00           0.30           0.0
4216            0.96               0.00           0.00           0.0
4217            0.00               0.00           0.65           0.0

     word_freq_our  word_freq_over  word_freq_remove  word_freq_internet  \
0             0.32            0.00              0.00                0.00
1             0.14            0.28              0.21                0.07
2             1.23            0.19              0.19                0.12
3             0.63            0.00              0.31                0.63
4             0.63            0.00              0.31                0.63
…              …               …                 …                   …
4213             0            0.31              0.00                0.00
4214             0            0.00              0.00                0.00
4215             0            0.00              0.00                0.00
4216          0.32            0.00              0.00                0.00
4217             0            0.00              0.00                0.00

      word_freq_order  word_freq_mail  …  char_freq_;  char_freq_(  \
```

```
0                0.00            0.00  …        0.000        0.000
1                0.00            0.94  …        0.000        0.132
2                0.64            0.25  …        0.010        0.143
3                0.31            0.63  …        0.000        0.137
4                0.31            0.63  …        0.000        0.135
…                …               …     …        …            …
4213             0.00            0.00  …        0.000        0.232
4214             0.00            0.00  …        0.000        0.000
4215             0.00            0.00  …        0.102        0.718
4216             0.00            0.00  …        0.000        0.057
4217             0.00            0.00  …        0.000        0.000

      char_freq_[  char_freq_!  char_freq_$  char_freq_#  \
0             0.0        0.778        0.000        0.000
1             0.0        0.372        0.180        0.048
2             0.0        0.276        0.184        0.010
3             0.0        0.137        0.000        0.000
4             0.0        0.135        0.000        0.000
…             …          …            …            …
4213          0.0        0.000        0.000        0.000
4214          0.0        0.353        0.000        0.000
4215          0.0        0.000        0.000        0.000
4216          0.0        0.000        0.000        0.000
4217          0.0        0.125        0.000        0.000

      capital_run_length_average  capital_run_length_longest  \
0                          3.756                          61
1                          5.114                         101
2                          9.821                         485
3                          3.537                          40
4                          3.537                          40
…                          …                              …
4213                       1.142                           3
4214                       1.555                           4
4215                       1.404                           6
4216                       1.147                           5
4217                       1.250                           5

      capital_run_length_total  is_spam
0                          278     True
1                         1028     True
2                         2259     True
3                          191     True
4                          191     True
…                          …       …
4213                        88    False
4214                        14    False
```

```
4215                        118    False
4216                         78    False
4217                         40    False

[4218 rows x 58 columns]
```

[13]:
```
lines, columns=dff.shape
print ("Number of rows in the data is: ", lines)
print ("Number of features in the data is: ", columns)
```

```
Number of rows in the data is:  4218
Number of features in the data is:  58
```

[14]:
```
dff.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 4218 entries, 0 to 4217
Data columns (total 58 columns):
 #   Column                      Non-Null Count  Dtype
---  ------                      --------------  -----
 0   word_freq_make              4218 non-null   float64
 1   word_freq_address           4218 non-null   float64
 2   word_freq_all               4218 non-null   float64
 3   word_freq_3d                4218 non-null   float64
 4   word_freq_our               4218 non-null   object
 5   word_freq_over              4218 non-null   float64
 6   word_freq_remove            4218 non-null   float64
 7   word_freq_internet          4218 non-null   float64
 8   word_freq_order             4218 non-null   float64
 9   word_freq_mail              4218 non-null   float64
 10  word_freq_receive           4218 non-null   float64
 11  word_freq_will              4218 non-null   float64
 12  word_freq_people            4218 non-null   float64
 13  word_freq_report            4218 non-null   float64
 14  word_freq_addresses         4218 non-null   float64
 15  word_freq_free              4218 non-null   float64
 16  word_freq_business          4218 non-null   float64
 17  word_freq_email             4218 non-null   float64
 18  word_freq_you               4218 non-null   float64
 19  word_freq_credit            4218 non-null   float64
 20  word_freq_your              4218 non-null   float64
 21  word_freq_font              4218 non-null   float64
 22  word_freq_000               4218 non-null   object
 23  word_freq_money             4218 non-null   float64
 24  word_freq_hp                4218 non-null   float64
 25  word_freq_hpl               4218 non-null   object
 26  word_freq_george            4218 non-null   float64
 27  word_freq_650               4218 non-null   float64
```

```
 28  word_freq_lab               4218 non-null   float64
 29  word_freq_labs              4218 non-null   object
 30  word_freq_telnet            4218 non-null   float64
 31  word_freq_857               4218 non-null   float64
 32  word_freq_data              4218 non-null   float64
 33  word_freq_415               4218 non-null   float64
 34  word_freq_85                4218 non-null   float64
 35  word_freq_technology        4218 non-null   float64
 36  word_freq_1999              4218 non-null   float64
 37  word_freq_parts             4218 non-null   float64
 38  word_freq_pm                4218 non-null   float64
 39  word_freq_direct            4218 non-null   float64
 40  word_freq_cs                4218 non-null   float64
 41  word_freq_meeting           4218 non-null   float64
 42  word_freq_original          4218 non-null   float64
 43  word_freq_project           4218 non-null   float64
 44  word_freq_re                4218 non-null   float64
 45  word_freq_edu               4218 non-null   float64
 46  word_freq_table             4218 non-null   float64
 47  word_freq_conference        4218 non-null   float64
 48  char_freq_;                 4218 non-null   float64
 49  char_freq_(                 4218 non-null   float64
 50  char_freq_[                 4218 non-null   float64
 51  char_freq_!                 4218 non-null   float64
 52  char_freq_$                 4218 non-null   float64
 53  char_freq_#                 4218 non-null   float64
 54  capital_run_length_average  4218 non-null   float64
 55  capital_run_length_longest  4218 non-null   int64
 56  capital_run_length_total    4218 non-null   int64
 57  is_spam                     4218 non-null   bool
dtypes: bool(1), float64(51), int64(2), object(4)
memory usage: 1.8+ MB
```

[15]:
```
dff["word_freq_our"]=pd.to_numeric(dff["word_freq_our"], errors="coerce")
```

[16]:
```
dff["word_freq_000"]=pd.to_numeric(dff["word_freq_000"], errors="coerce")
```

[17]:
```
dff["word_freq_hpl"]=pd.to_numeric(dff["word_freq_hpl"], errors="coerce")
```

[18]:
```
dff["word_freq_labs"]=pd.to_numeric(dff["word_freq_labs"], errors="coerce")
```

[19]:
```
dff.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 4218 entries, 0 to 4217
Data columns (total 58 columns):
 #   Column                      Non-Null Count  Dtype
---  ------                      --------------  -----
```

```
0   word_freq_make              4218 non-null   float64
1   word_freq_address           4218 non-null   float64
2   word_freq_all               4218 non-null   float64
3   word_freq_3d                4218 non-null   float64
4   word_freq_our               4217 non-null   float64
5   word_freq_over              4218 non-null   float64
6   word_freq_remove            4218 non-null   float64
7   word_freq_internet          4218 non-null   float64
8   word_freq_order             4218 non-null   float64
9   word_freq_mail              4218 non-null   float64
10  word_freq_receive           4218 non-null   float64
11  word_freq_will              4218 non-null   float64
12  word_freq_people            4218 non-null   float64
13  word_freq_report            4218 non-null   float64
14  word_freq_addresses         4218 non-null   float64
15  word_freq_free              4218 non-null   float64
16  word_freq_business          4218 non-null   float64
17  word_freq_email             4218 non-null   float64
18  word_freq_you               4218 non-null   float64
19  word_freq_credit            4218 non-null   float64
20  word_freq_your              4218 non-null   float64
21  word_freq_font              4218 non-null   float64
22  word_freq_000               4217 non-null   float64
23  word_freq_money             4218 non-null   float64
24  word_freq_hp                4218 non-null   float64
25  word_freq_hpl               4217 non-null   float64
26  word_freq_george            4218 non-null   float64
27  word_freq_650               4218 non-null   float64
28  word_freq_lab               4218 non-null   float64
29  word_freq_labs              4217 non-null   float64
30  word_freq_telnet            4218 non-null   float64
31  word_freq_857               4218 non-null   float64
32  word_freq_data              4218 non-null   float64
33  word_freq_415               4218 non-null   float64
34  word_freq_85                4218 non-null   float64
35  word_freq_technology        4218 non-null   float64
36  word_freq_1999              4218 non-null   float64
37  word_freq_parts             4218 non-null   float64
38  word_freq_pm                4218 non-null   float64
39  word_freq_direct            4218 non-null   float64
40  word_freq_cs                4218 non-null   float64
41  word_freq_meeting           4218 non-null   float64
42  word_freq_original          4218 non-null   float64
43  word_freq_project           4218 non-null   float64
44  word_freq_re                4218 non-null   float64
45  word_freq_edu               4218 non-null   float64
46  word_freq_table             4218 non-null   float64
47  word_freq_conference        4218 non-null   float64
```

```
 48  char_freq_;                 4218 non-null   float64
 49  char_freq_(                 4218 non-null   float64
 50  char_freq_[                 4218 non-null   float64
 51  char_freq_!                 4218 non-null   float64
 52  char_freq_$                 4218 non-null   float64
 53  char_freq_#                 4218 non-null   float64
 54  capital_run_length_average  4218 non-null   float64
 55  capital_run_length_longest  4218 non-null   int64
 56  capital_run_length_total    4218 non-null   int64
 57  is_spam                     4218 non-null   bool
dtypes: bool(1), float64(55), int64(2)
memory usage: 1.8 MB
```

- **After checking the dataset info I've notice 4 'object' features, as they represent numerics features, except for the 'is_spam' feature, I've converted then into numerics to continue the analysis.**

[20]:
```
dff.describe()
```

[20]:

	word_freq_make	word_freq_address	word_freq_all	word_freq_3d	\
count	4218.000000	4218.000000	4218.000000	4218.000000	
mean	0.104168	0.112442	0.290955	0.062959	
std	0.299755	0.453855	0.515370	1.351206	
min	0.000000	0.000000	0.000000	0.000000	
25%	0.000000	0.000000	0.000000	0.000000	
50%	0.000000	0.000000	0.000000	0.000000	
75%	0.000000	0.000000	0.440000	0.000000	
max	4.540000	14.280000	5.100000	42.810000	

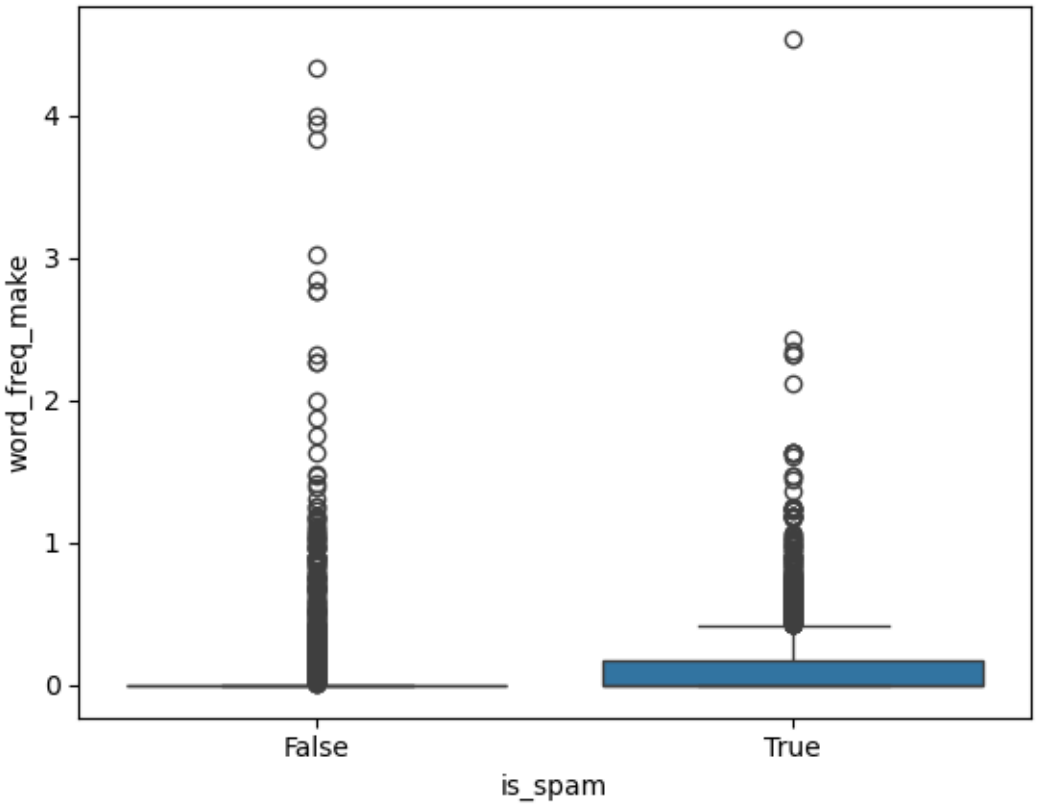
	word_freq_our	word_freq_over	word_freq_remove	word_freq_internet	\
count	4217.000000	4218.000000	4218.000000	4218.000000	
mean	0.324854	0.096472	0.117288	0.107795	
std	0.687343	0.275800	0.396936	0.409920	
min	0.000000	0.000000	0.000000	0.000000	
25%	0.000000	0.000000	0.000000	0.000000	
50%	0.000000	0.000000	0.000000	0.000000	
75%	0.410000	0.000000	0.000000	0.000000	
max	10.000000	5.880000	7.270000	11.110000	

	word_freq_order	word_freq_mail	…	word_freq_conference	\
count	4218.000000	4218.000000	…	4218.000000	
mean	0.091835	0.247985	…	0.034680	
std	0.282023	0.656095	…	0.298242	
min	0.000000	0.000000	…	0.000000	
25%	0.000000	0.000000	…	0.000000	
50%	0.000000	0.000000	…	0.000000	
75%	0.000000	0.190000	…	0.000000	
max	5.260000	18.180000	…	10.000000	

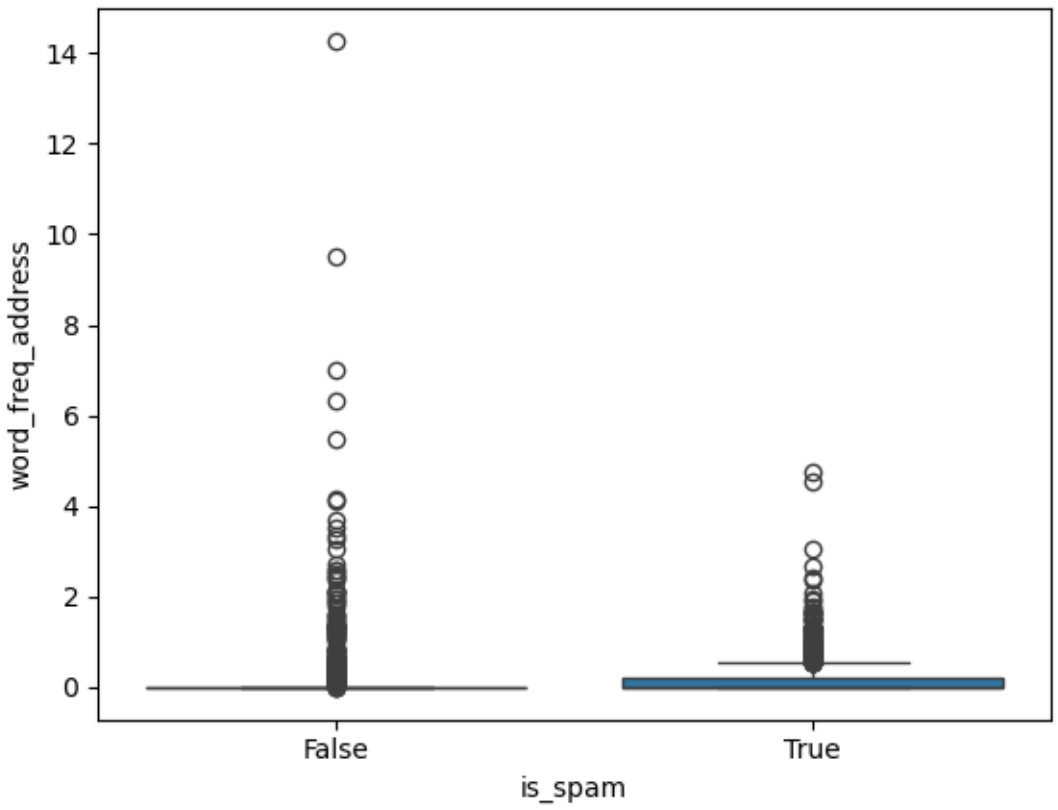
```
       char_freq_;  char_freq_(  char_freq_[  char_freq_!  char_freq_$  \
count  4218.000000  4218.000000  4218.000000  4218.000000  4218.000000
mean      0.040326     0.143796     0.017343     0.280807     0.076080
std       0.252299     0.274060     0.105633     0.842646     0.239698
min       0.000000     0.000000     0.000000     0.000000     0.000000
25%       0.000000     0.000000     0.000000     0.000000     0.000000
50%       0.000000     0.073000     0.000000     0.016000     0.000000
75%       0.000000     0.194000     0.000000     0.331000     0.053000
max       4.385000     9.752000     4.081000    32.478000     6.003000

       char_freq_#  capital_run_length_average  capital_run_length_longest  \
count  4218.000000                 4218.000000                 4218.000000
mean      0.045716                    5.376630                   52.054765
std       0.435515                   33.116383                  199.403634
min       0.000000                    1.000000                    1.000000
25%       0.000000                    1.625000                    7.000000
50%       0.000000                    2.295000                   15.000000
75%       0.000000                    3.705750                   44.000000
max      19.829000                 1102.500000                 9989.000000

       capital_run_length_total
count               4218.000000
mean                 290.904931
std                  618.312166
min                    1.000000
25%                   39.000000
50%                  101.000000
75%                  273.000000
max                15841.000000

[8 rows x 57 columns]
```

- **These observations and variables from the 4,218 emails are important to understand the structure of an undesired (spam) email. By analyzing the patterns formed by spam messages, it is possible to identify which emails are unwanted and therefore improve the accuracy of spam detection.**
- **In the next step, I will execute boxplots to identify the distribution of each attribute and mainly identify the presence of outliers, seeking to understand them.**

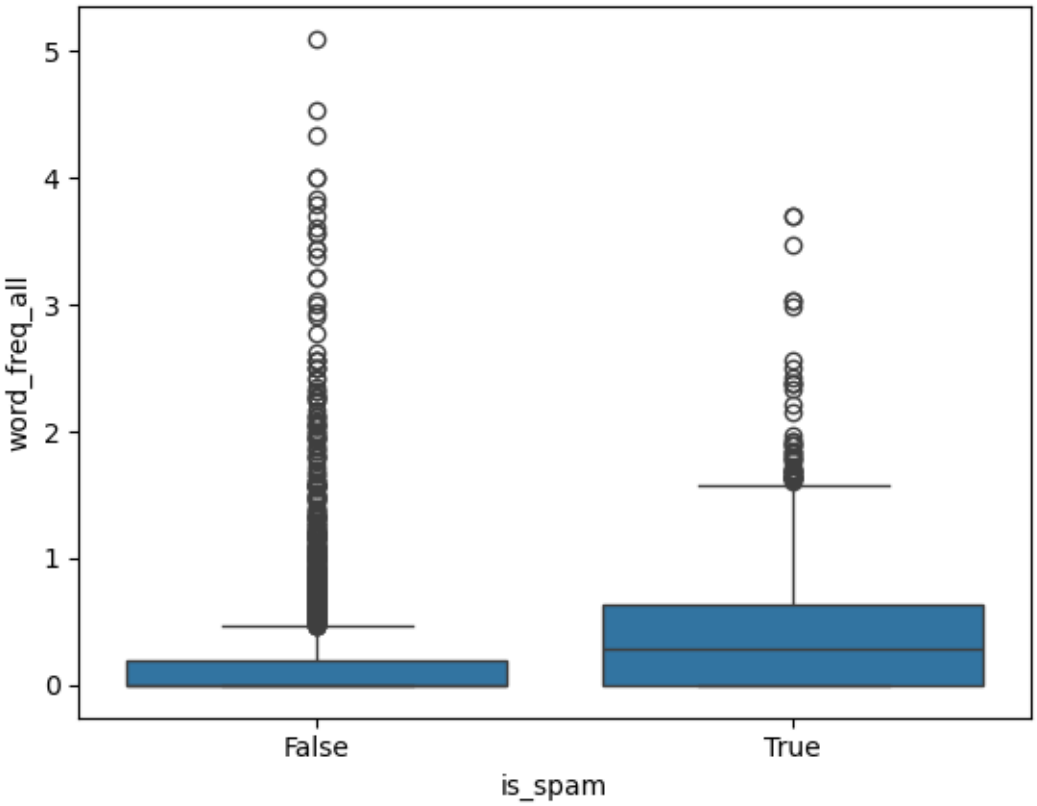
[21]:
```
#1
sns.boxplot(y="word_freq_make", x="is_spam", data=dff)
plt.show()
```



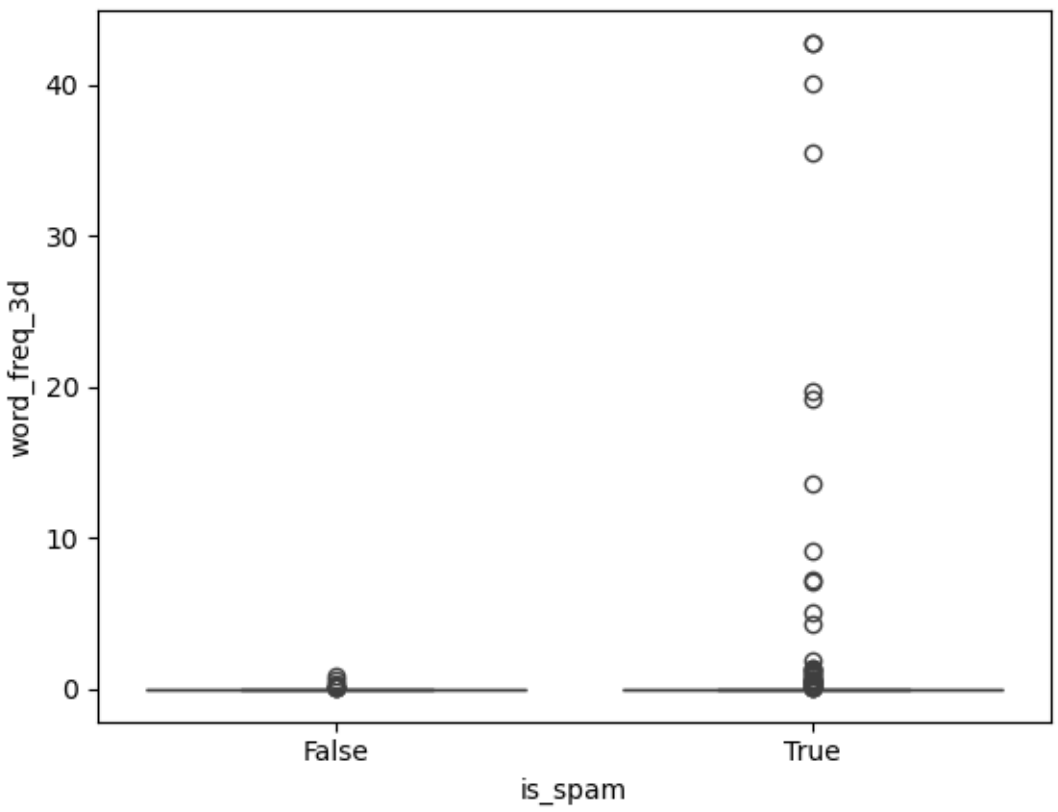
[22]:
```
#2
sns.boxplot(y="word_freq_address", x="is_spam", data=dff)
plt.show()
```



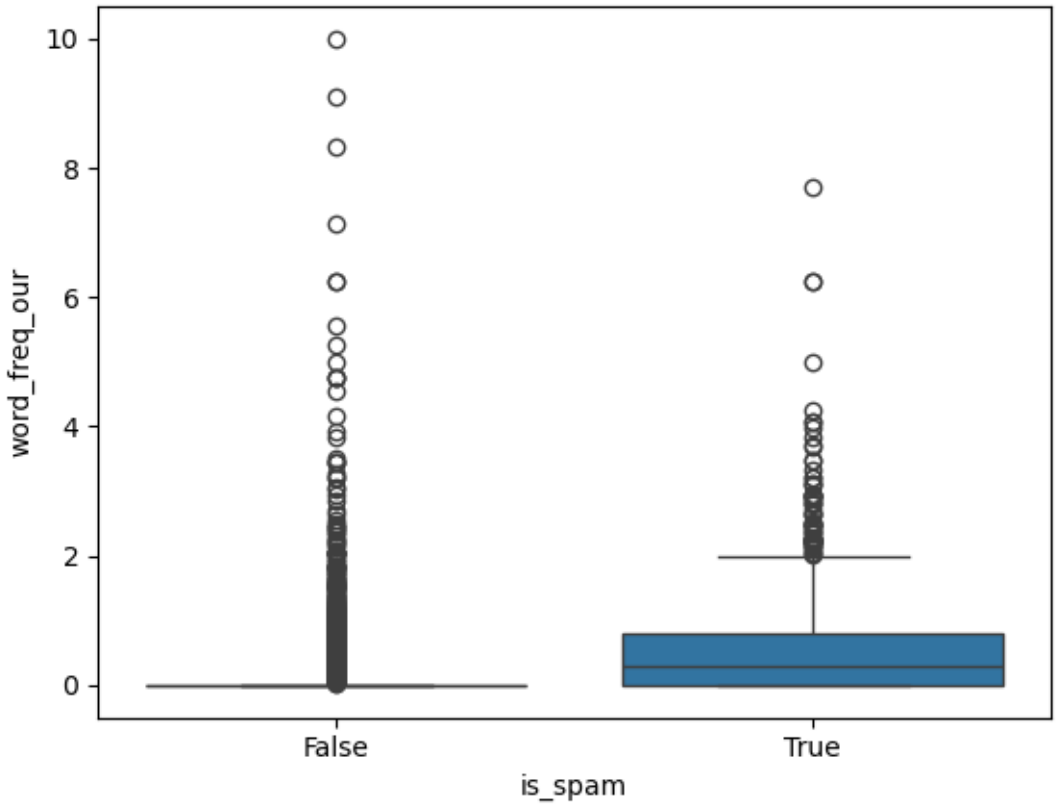
[23]:
```
#3
sns.boxplot(y="word_freq_all", x="is_spam", data=dff)
plt.show()
```



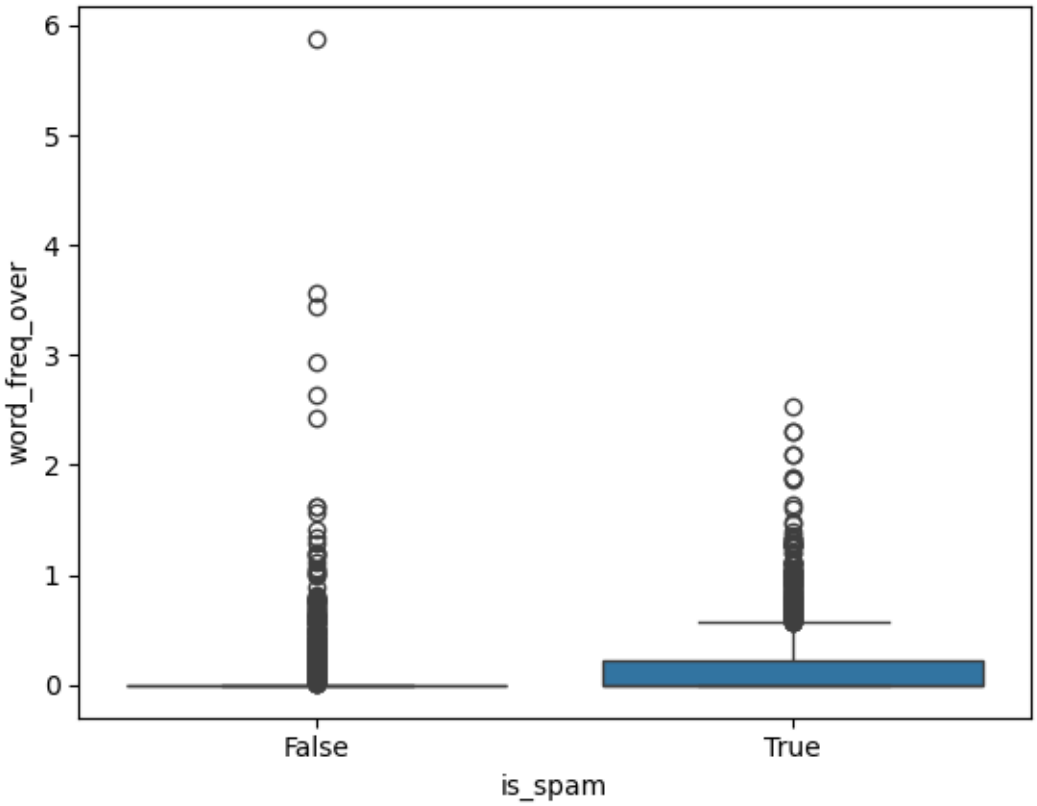
[24]:
```
#4
sns.boxplot(y="word_freq_3d", x="is_spam", data=dff)
plt.show()
```



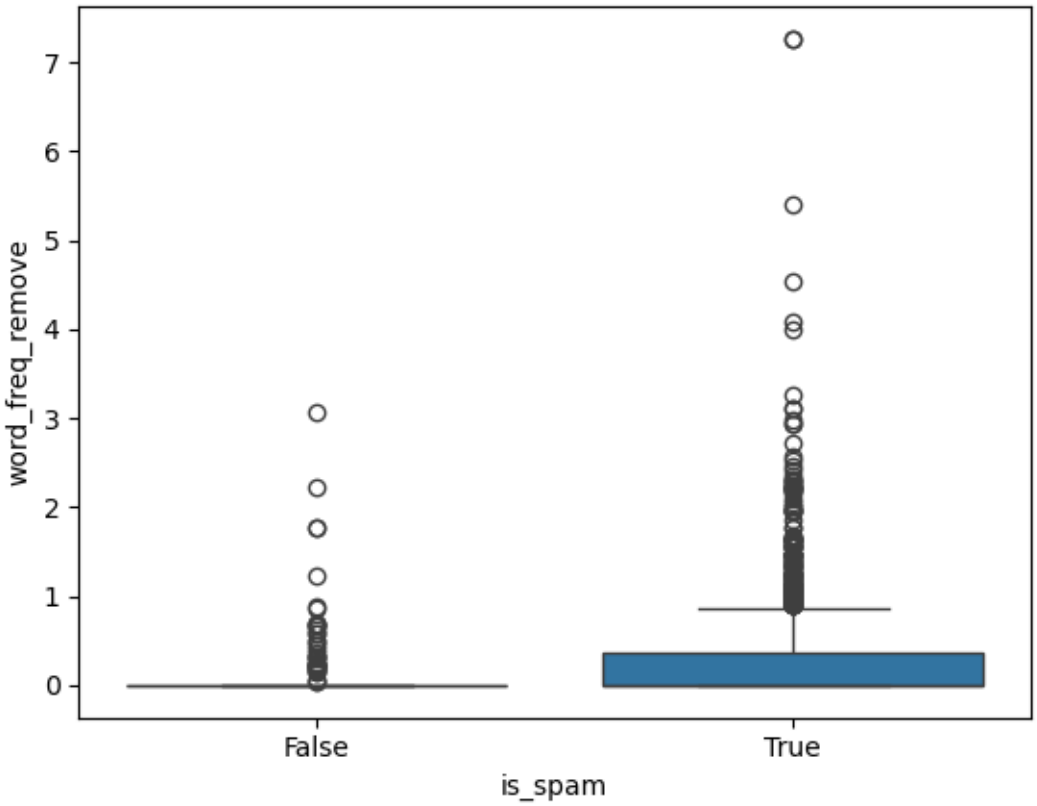
[25]:
```
#5
sns.boxplot(y="word_freq_our", x="is_spam", data=dff)
plt.show()
```



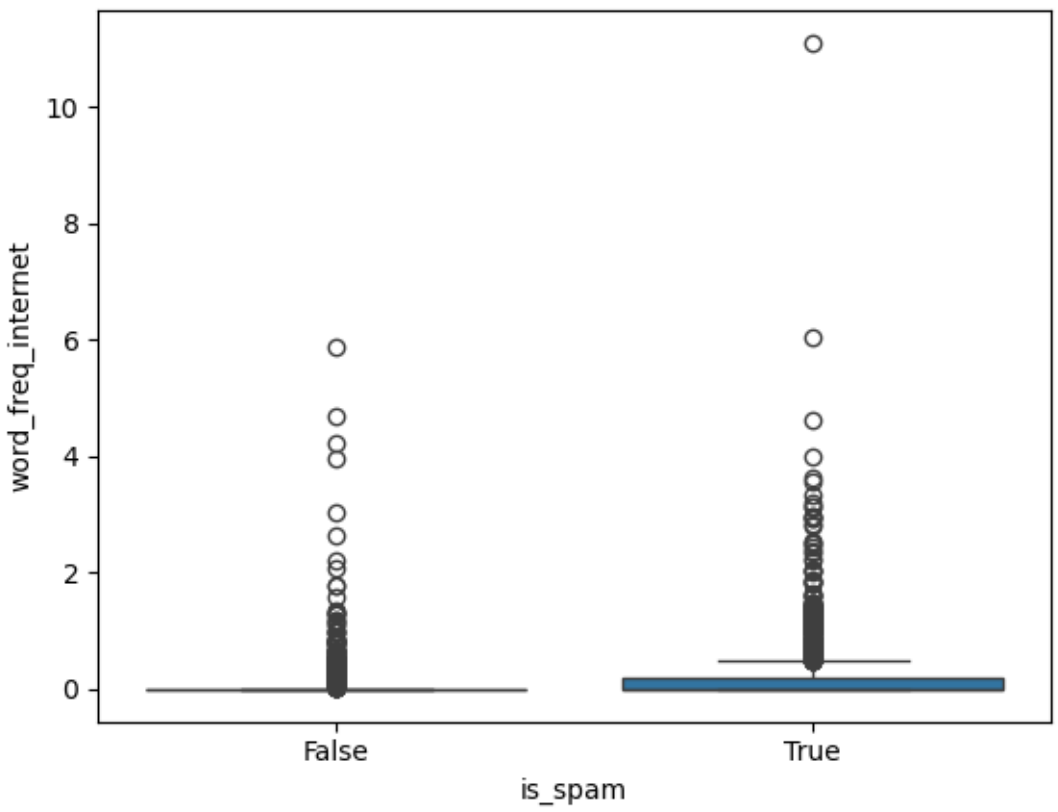
[26]:
```
#6
sns.boxplot(y="word_freq_over", x="is_spam", data=dff)
plt.show()
```



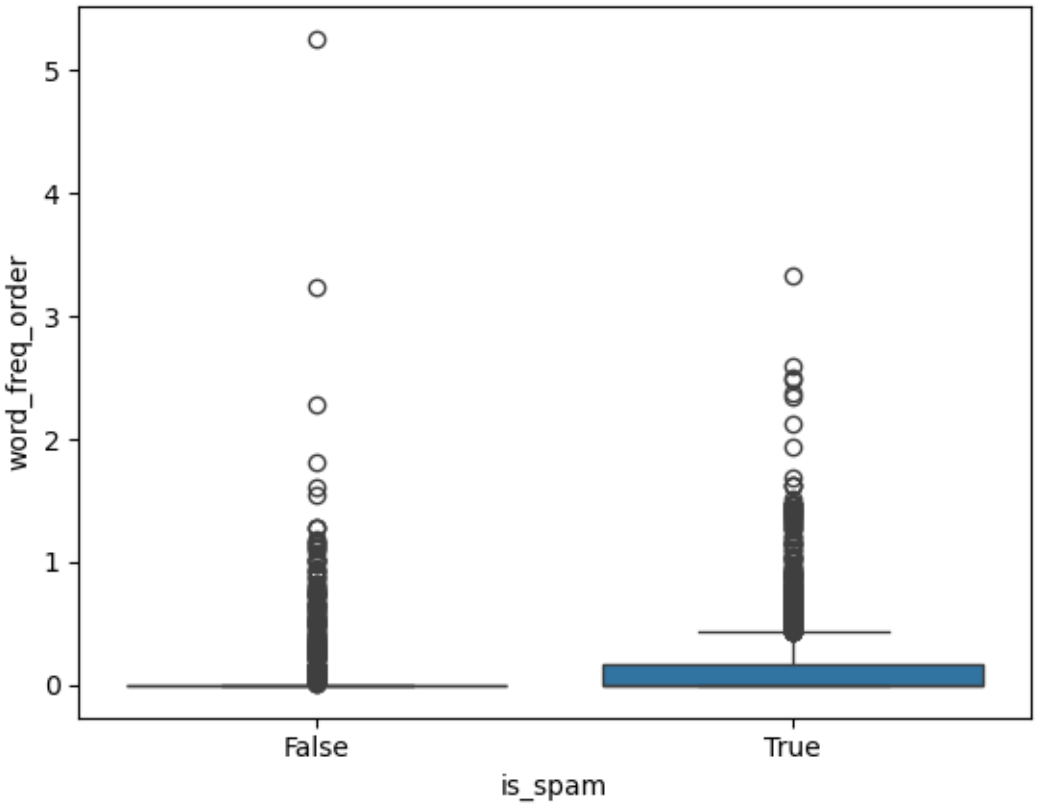
[27]:
```
#7
sns.boxplot(y="word_freq_remove", x="is_spam", data=dff)
plt.show()
```



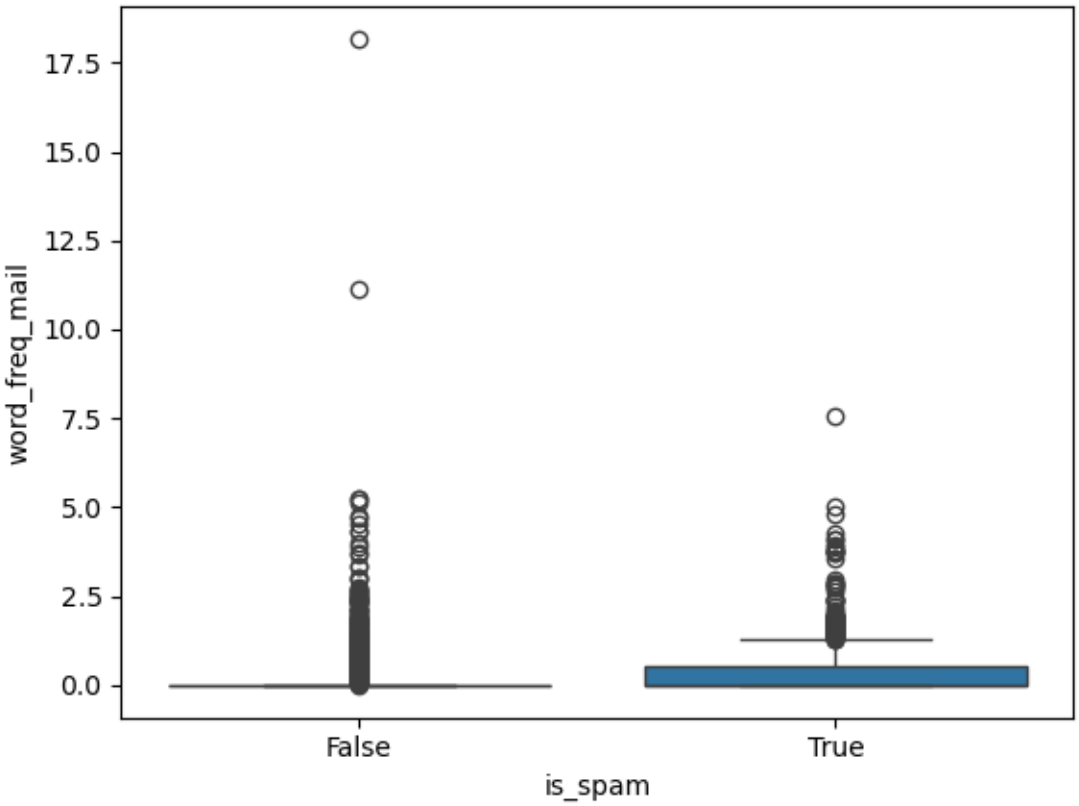
[28]:
```
#8
sns.boxplot(y="word_freq_internet", x="is_spam", data=dff)
plt.show()
```



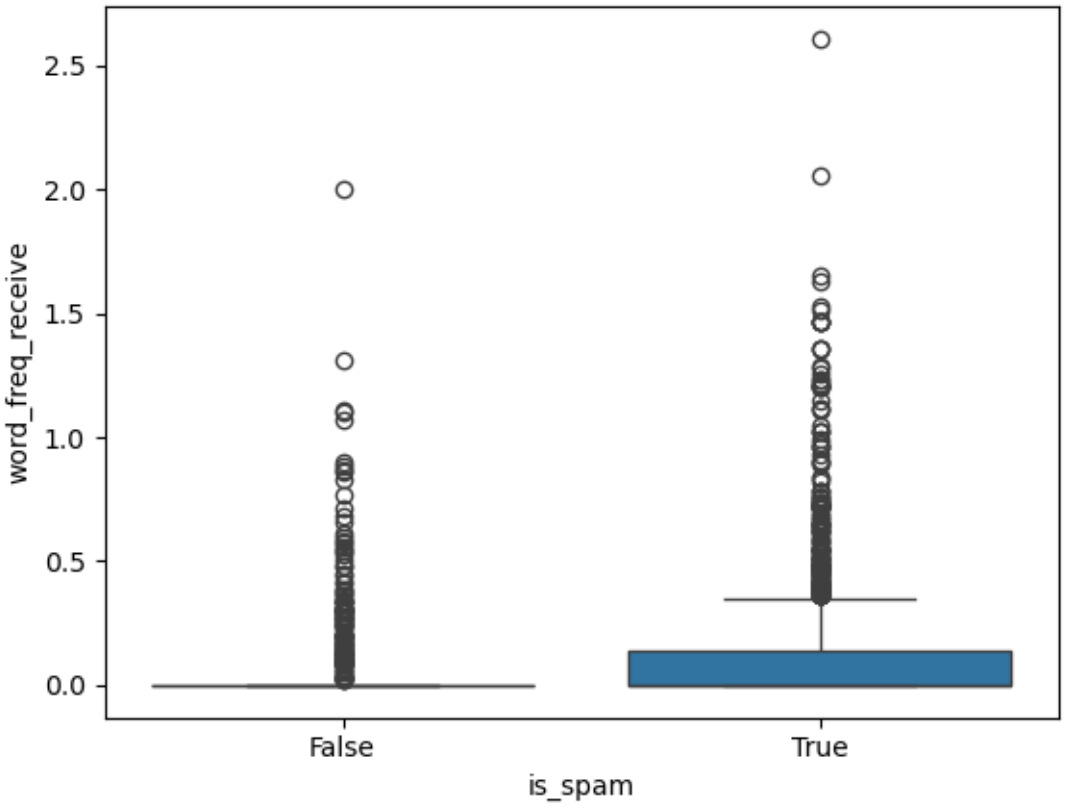
[29]:
```
#9
sns.boxplot(y="word_freq_order", x="is_spam", data=dff)
plt.show()
```



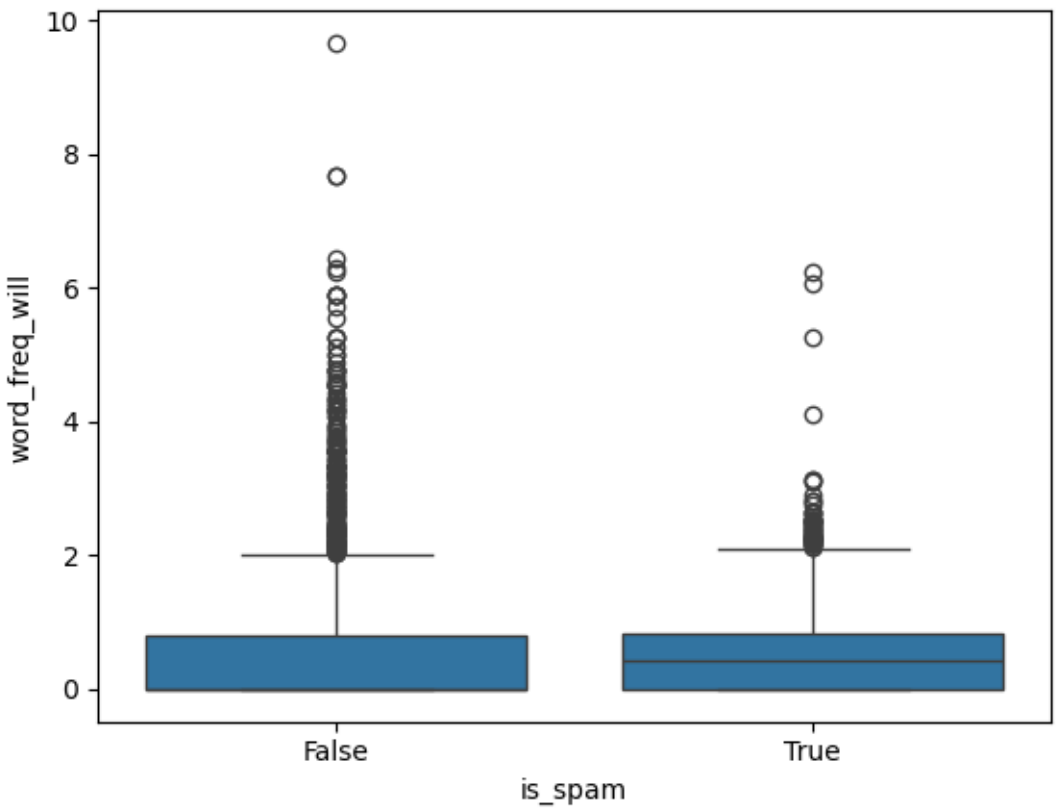
[30]:
```
#10
sns.boxplot(y="word_freq_mail", x="is_spam", data=dff)
plt.show()
```



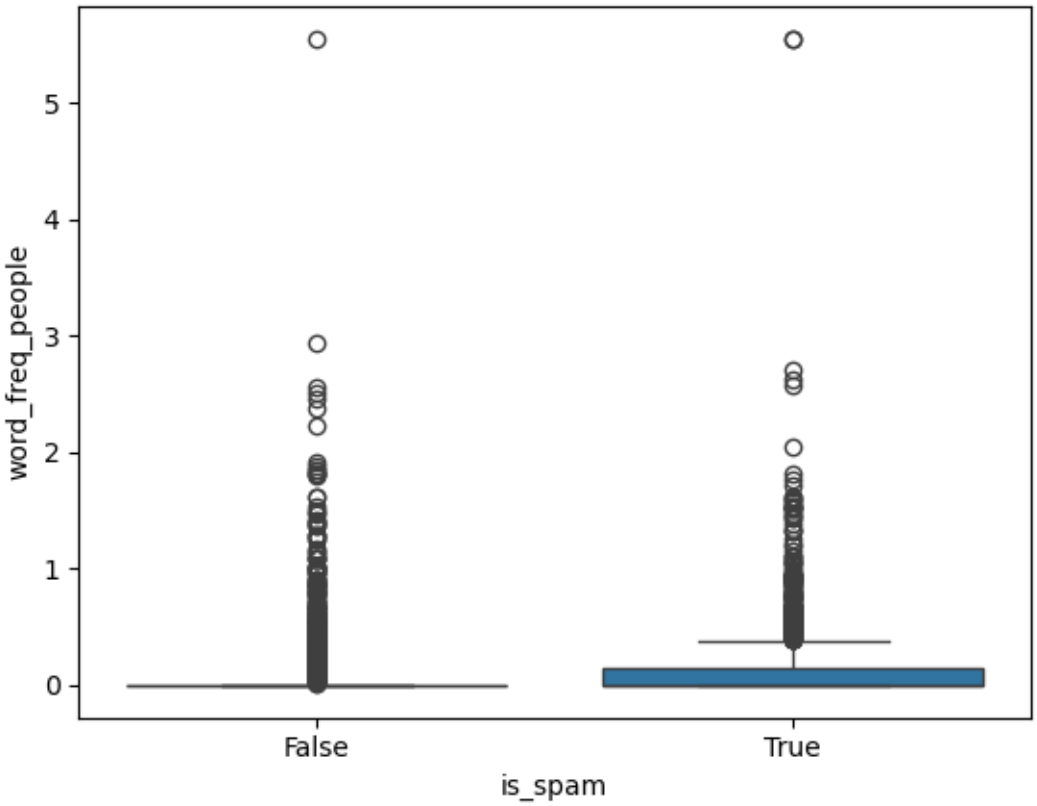
[31]:
```
#11
sns.boxplot(y="word_freq_receive", x="is_spam", data=dff)
plt.show()
```



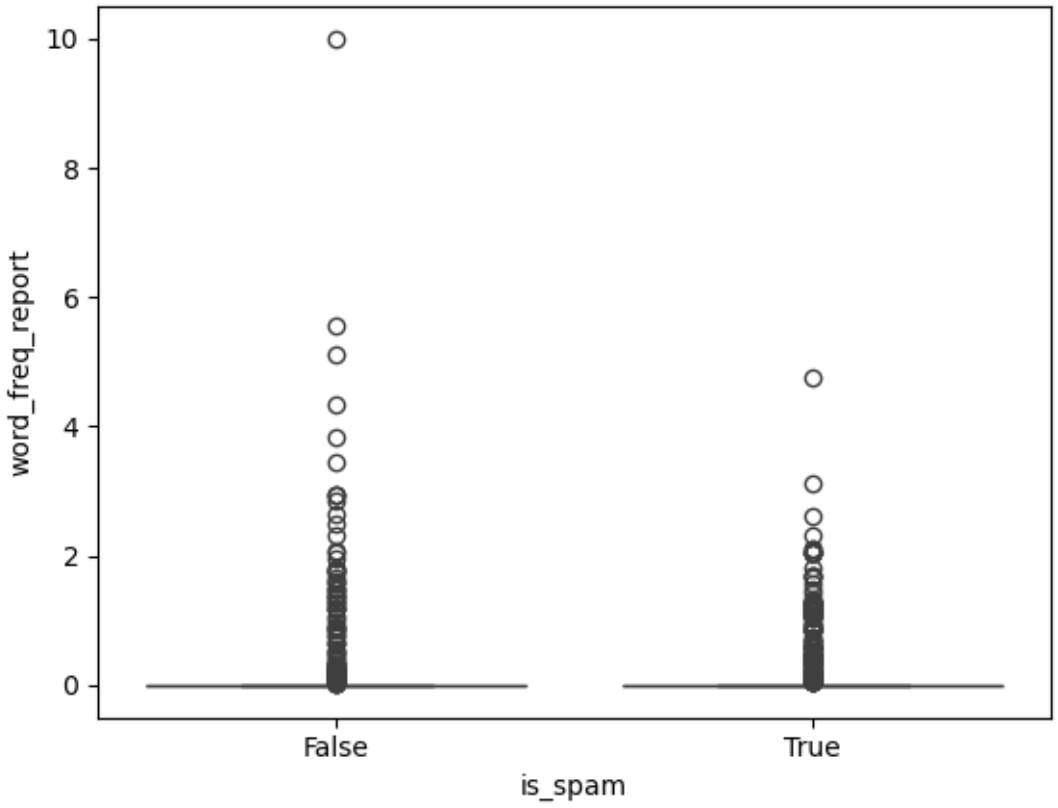
[32]:
```
#12
sns.boxplot(y="word_freq_will", x="is_spam", data=dff)
plt.show()
```



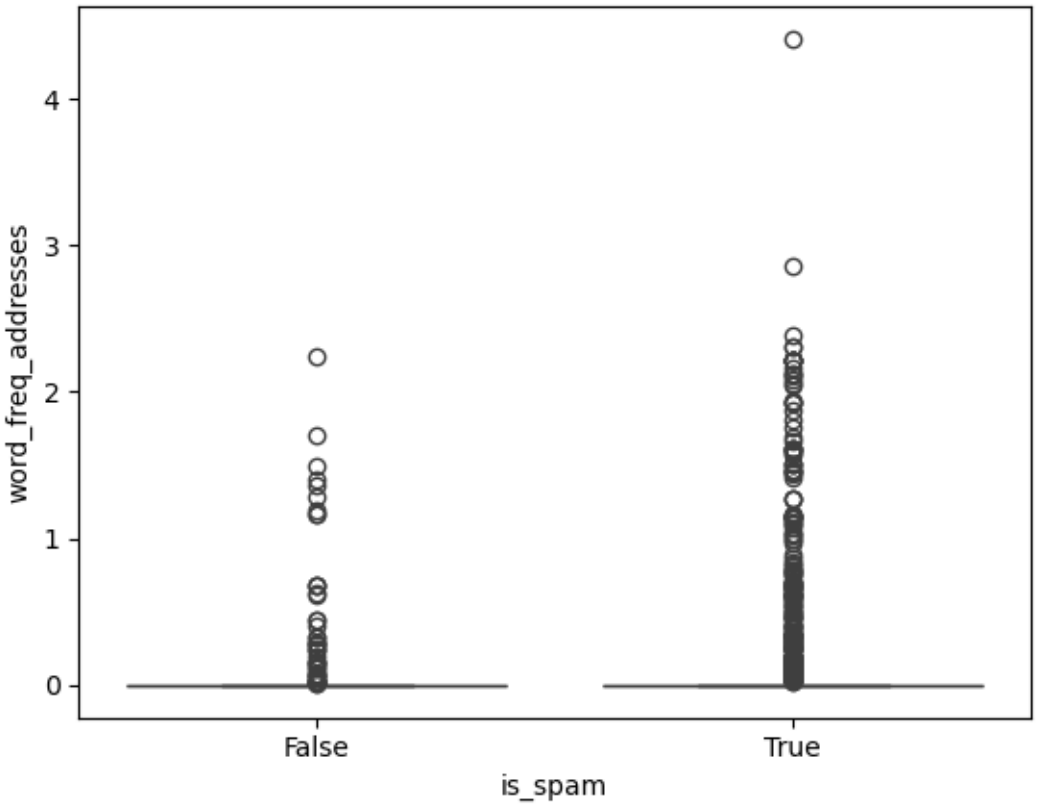
[33]:
```
#13
sns.boxplot(y="word_freq_people", x="is_spam", data=dff)
plt.show()
```



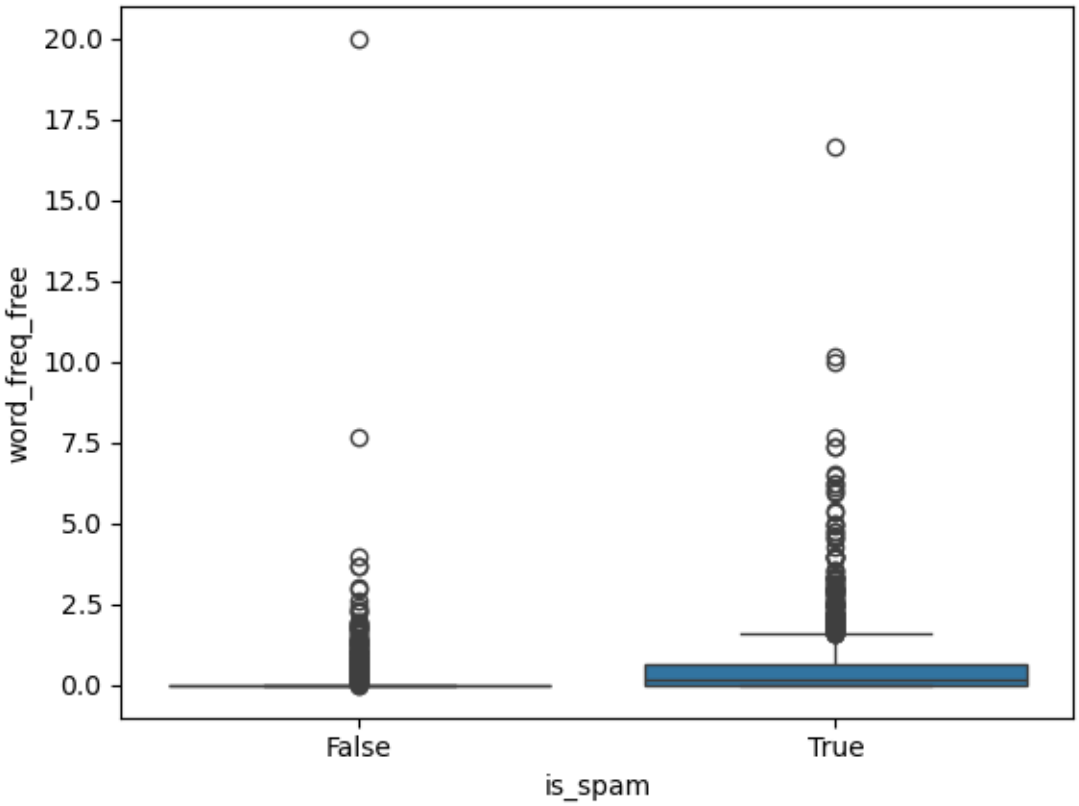
[34]:
```
#14
sns.boxplot(y="word_freq_report", x="is_spam", data=dff)
plt.show()
```



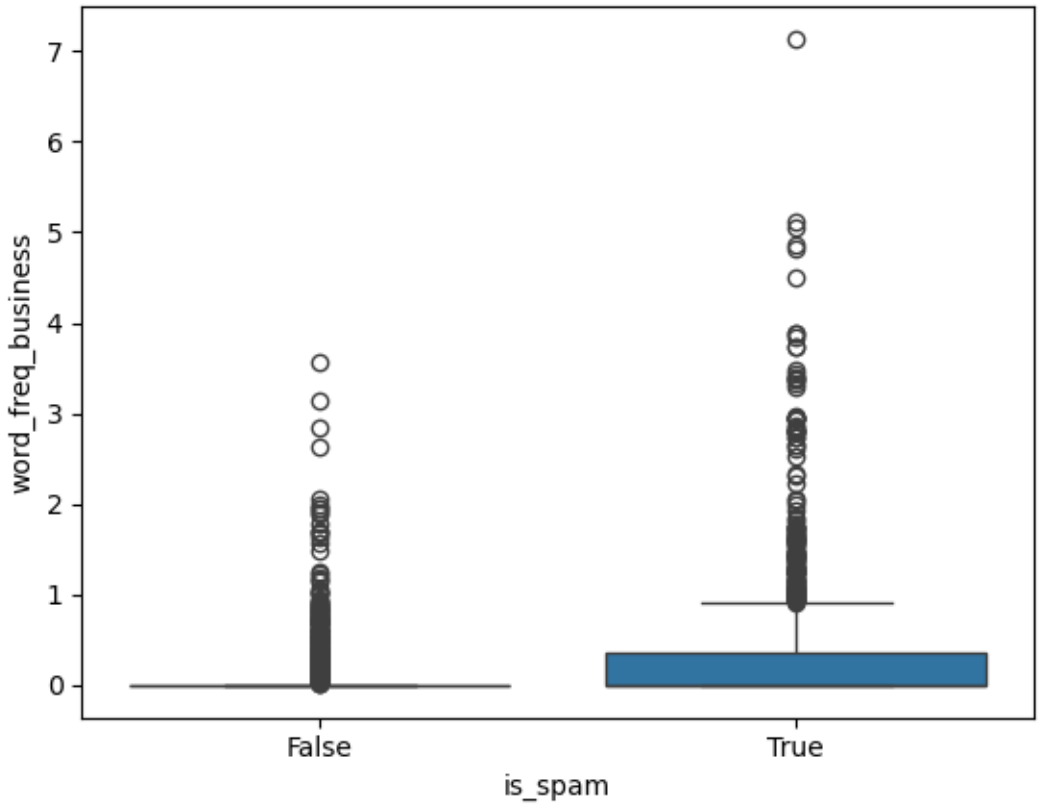
[35]:
```
#15
sns.boxplot(y="word_freq_addresses", x="is_spam", data=dff)
plt.show()
```



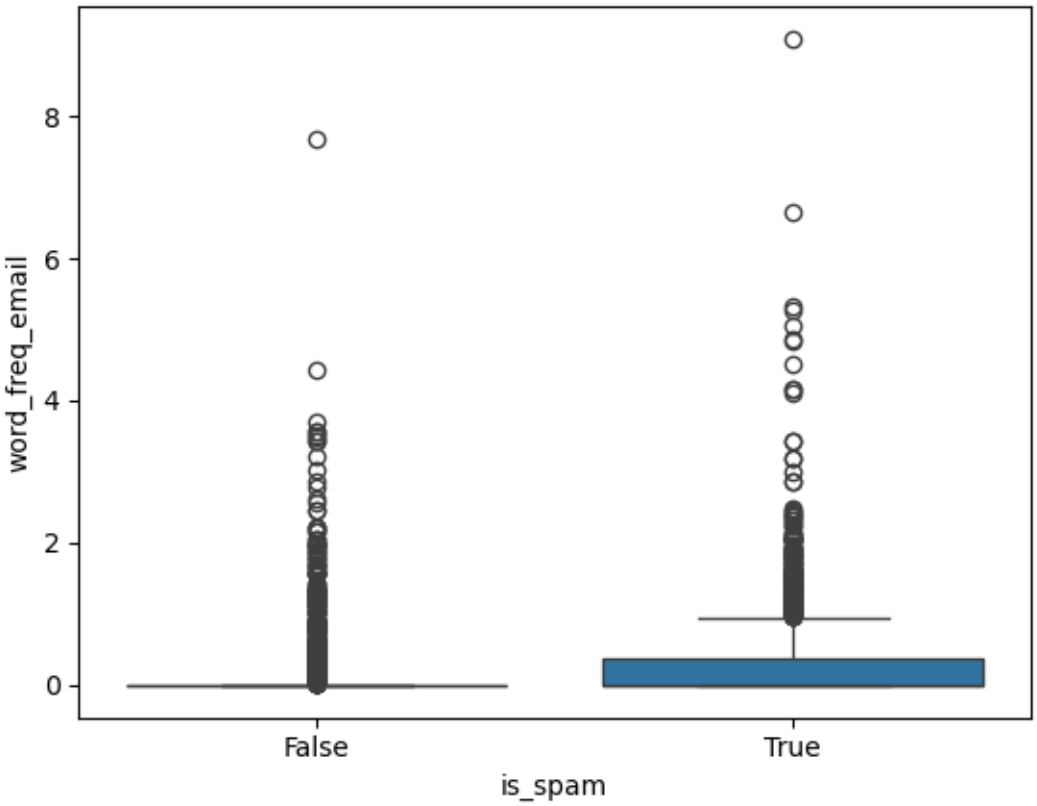
[36]:
```
#16
sns.boxplot(y="word_freq_free", x="is_spam", data=dff)
plt.show()
```



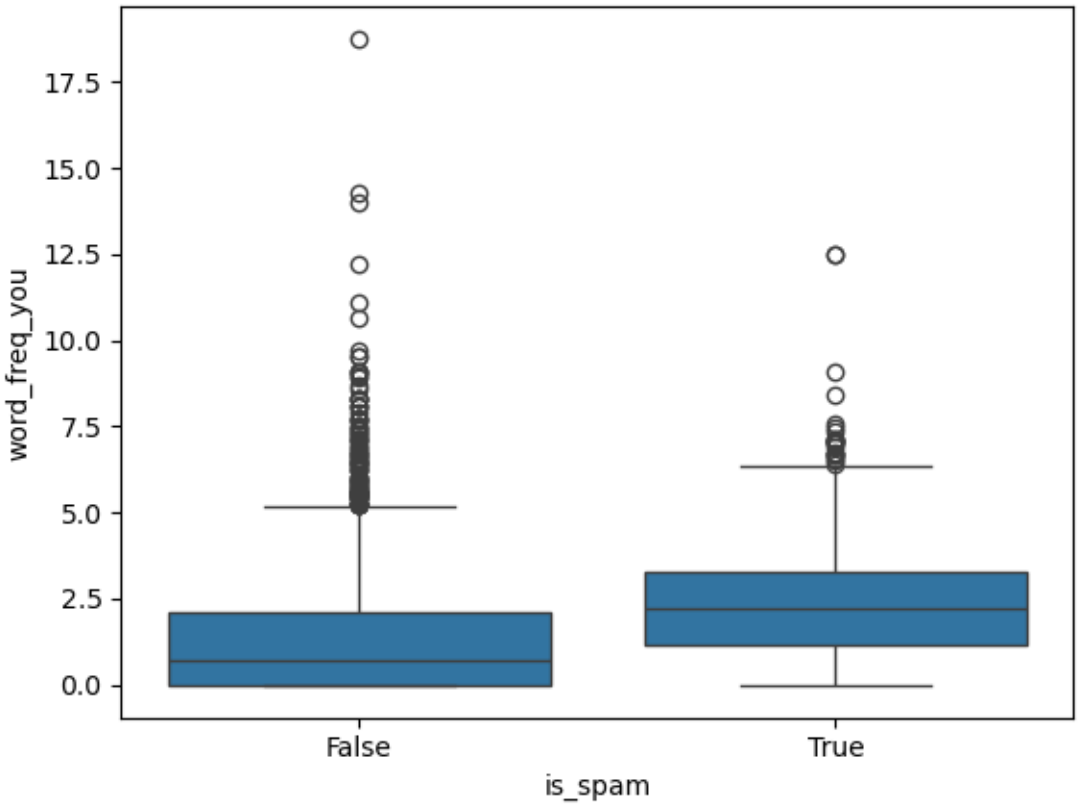
[37]:
```
#17
sns.boxplot(y="word_freq_business", x="is_spam", data=dff)
plt.show()
```



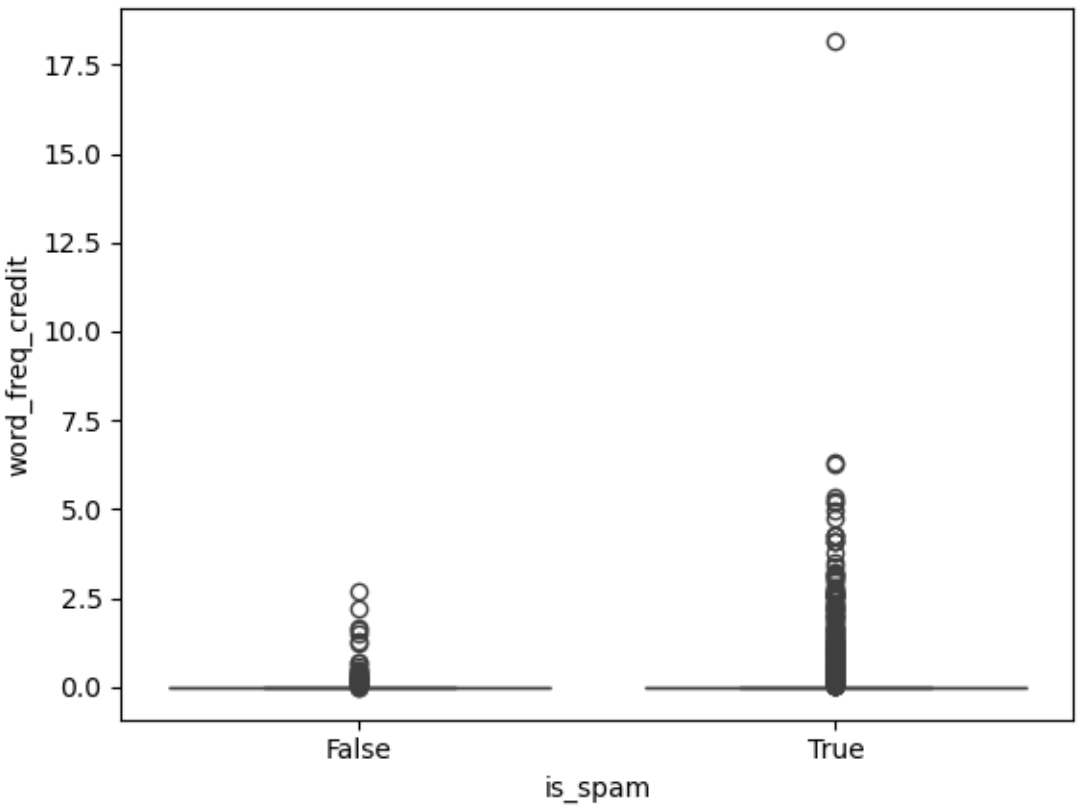
[38]:
```
#18
sns.boxplot(y="word_freq_email", x="is_spam", data=dff)
plt.show()
```



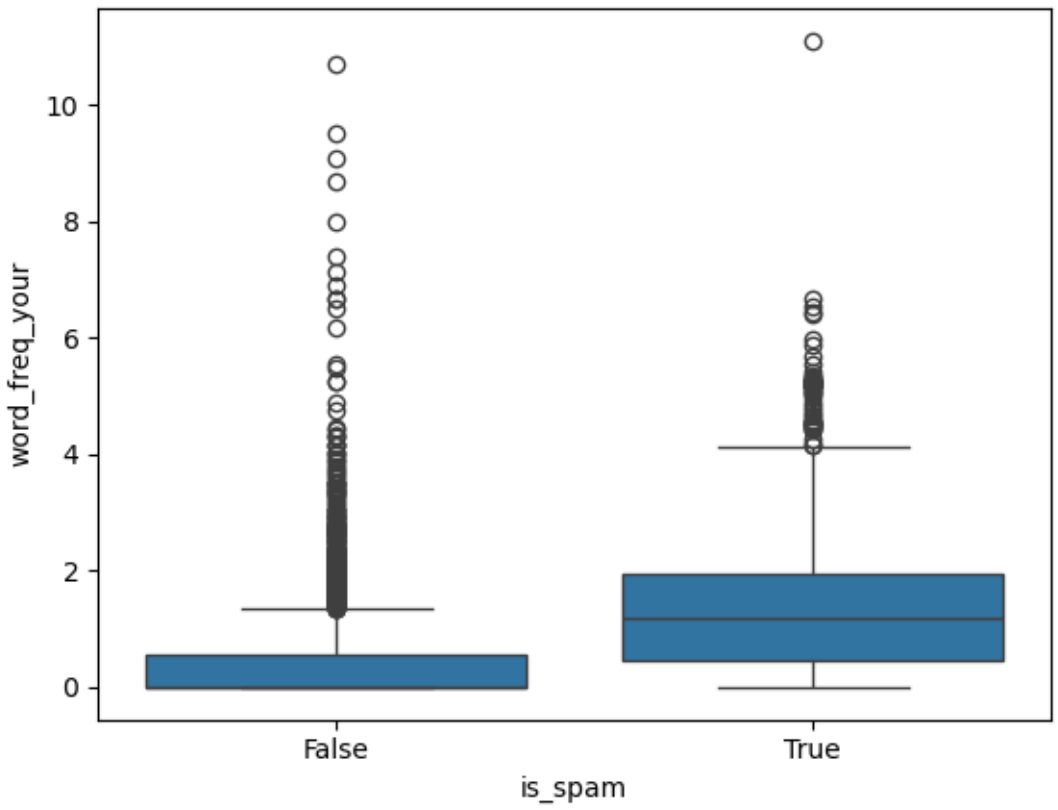
[39]:
```
#19
sns.boxplot(y="word_freq_you", x="is_spam", data=dff)
plt.show()
```



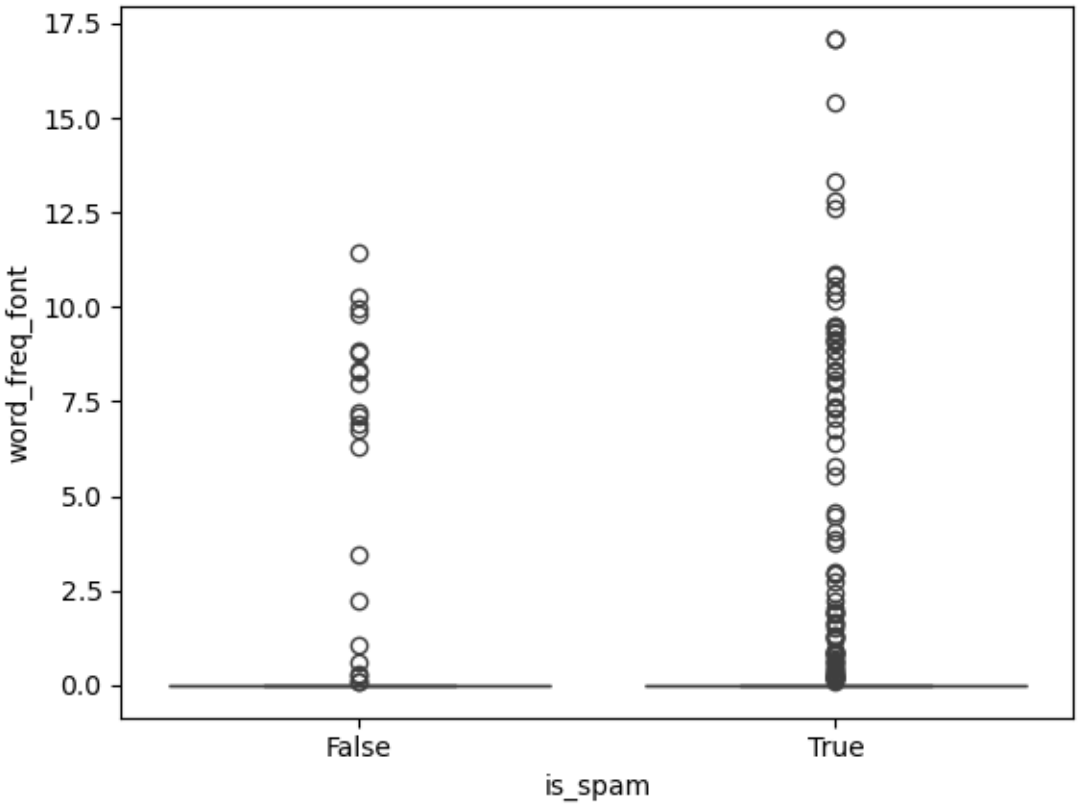
[40]:
```
#20
sns.boxplot(y="word_freq_credit", x="is_spam", data=dff)
plt.show()
```



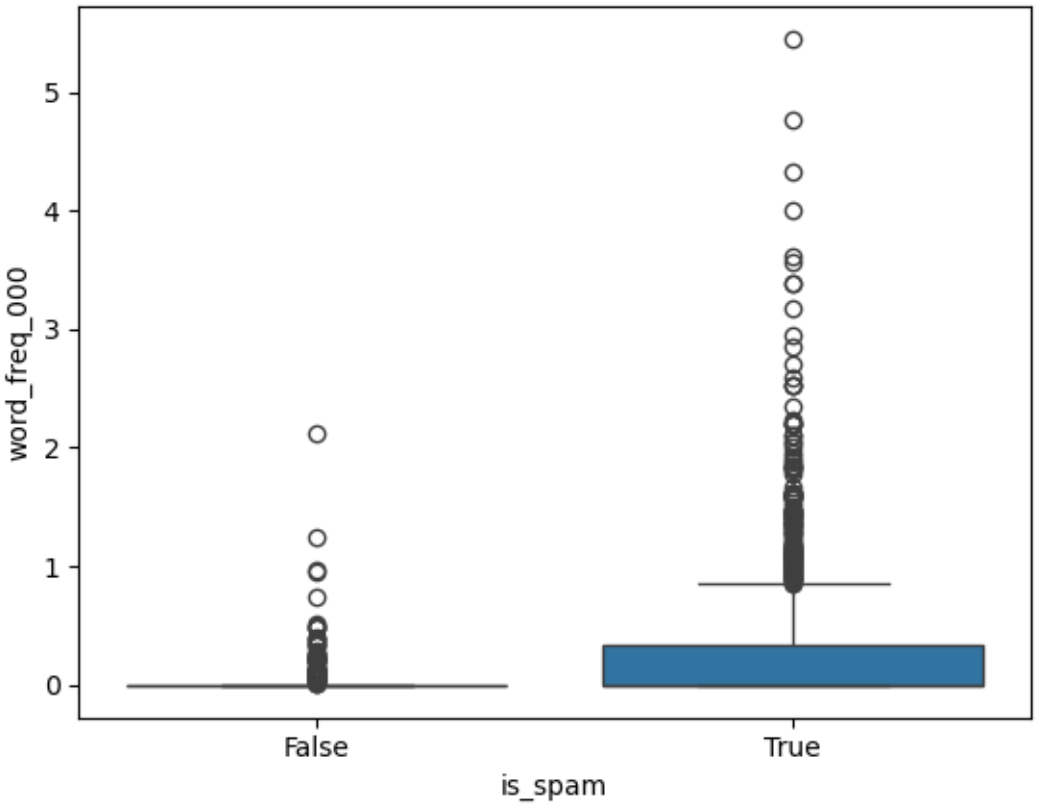
[41]:
```
#21
sns.boxplot(y="word_freq_your", x="is_spam", data=dff)
plt.show()
```



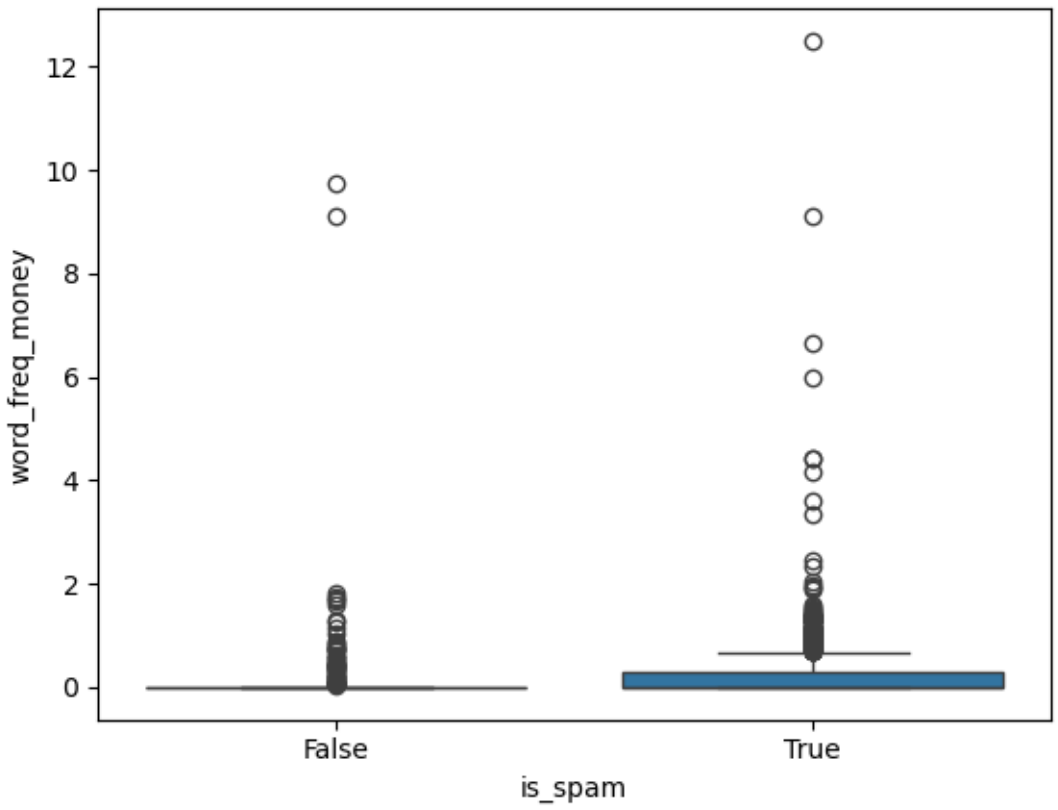
[42]:
```
#22
sns.boxplot(y="word_freq_font", x="is_spam", data=dff)
plt.show()
```



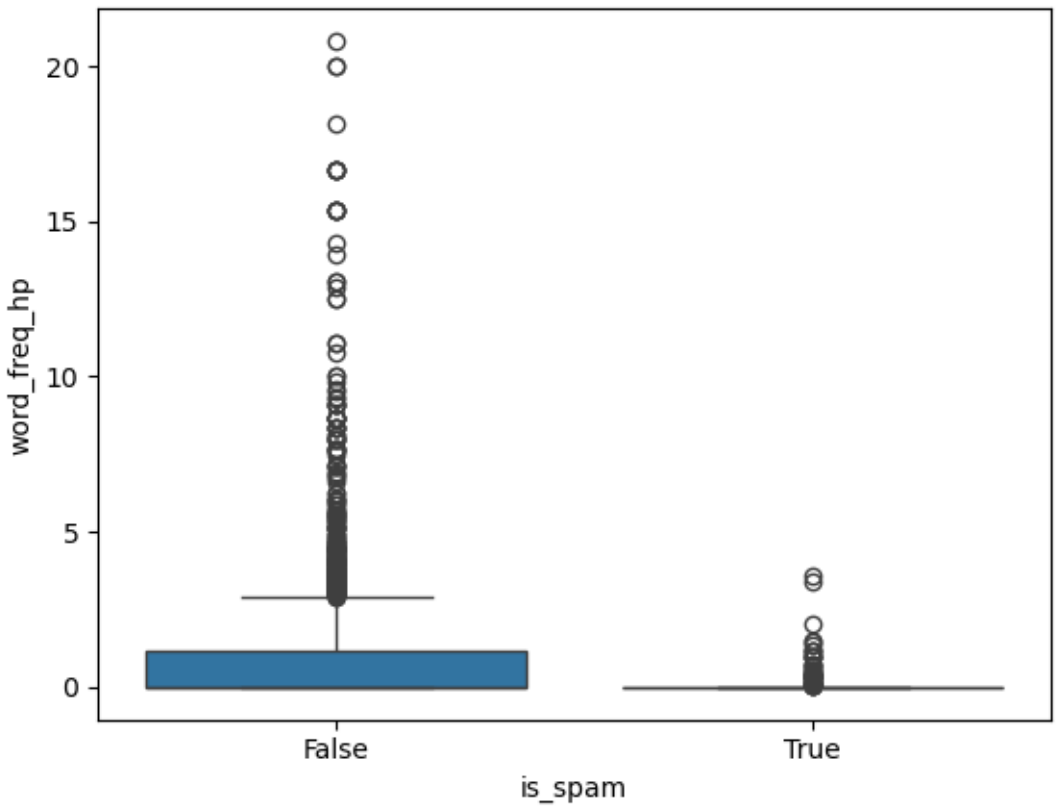
[43]:
```
#23
sns.boxplot(y="word_freq_000", x="is_spam", data=dff)
plt.show()
```



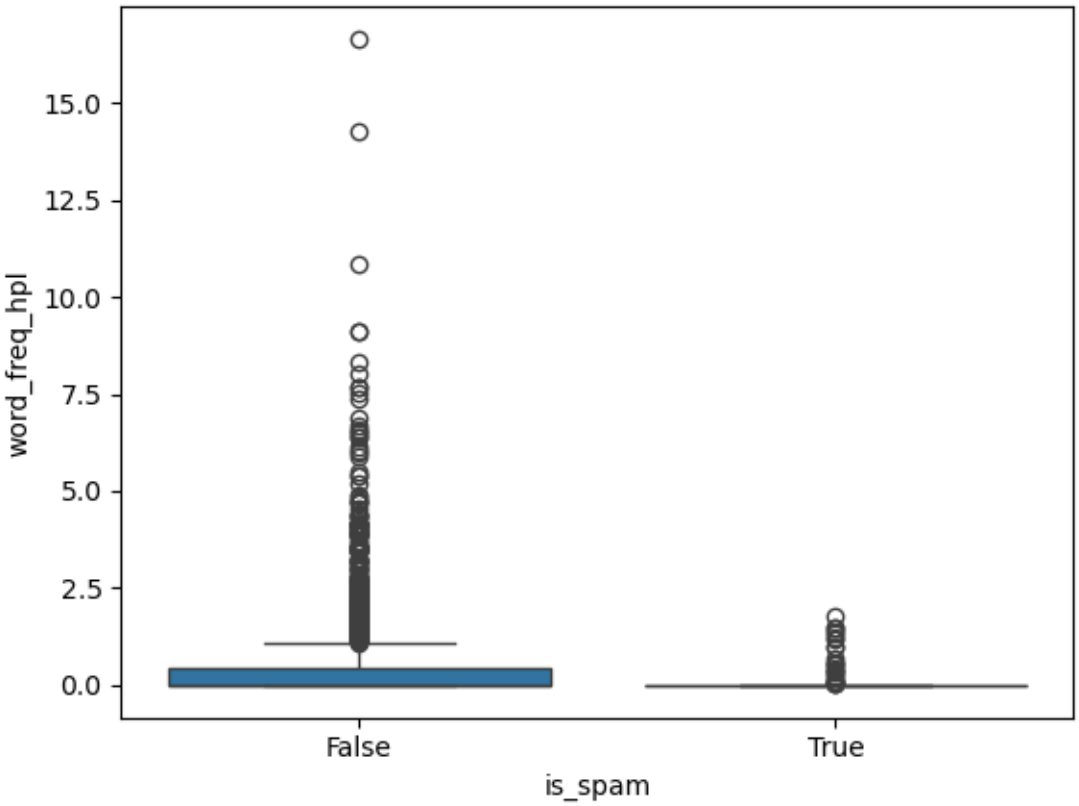
[44]:
```
#24
sns.boxplot(y="word_freq_money", x="is_spam", data=dff)
plt.show()
```



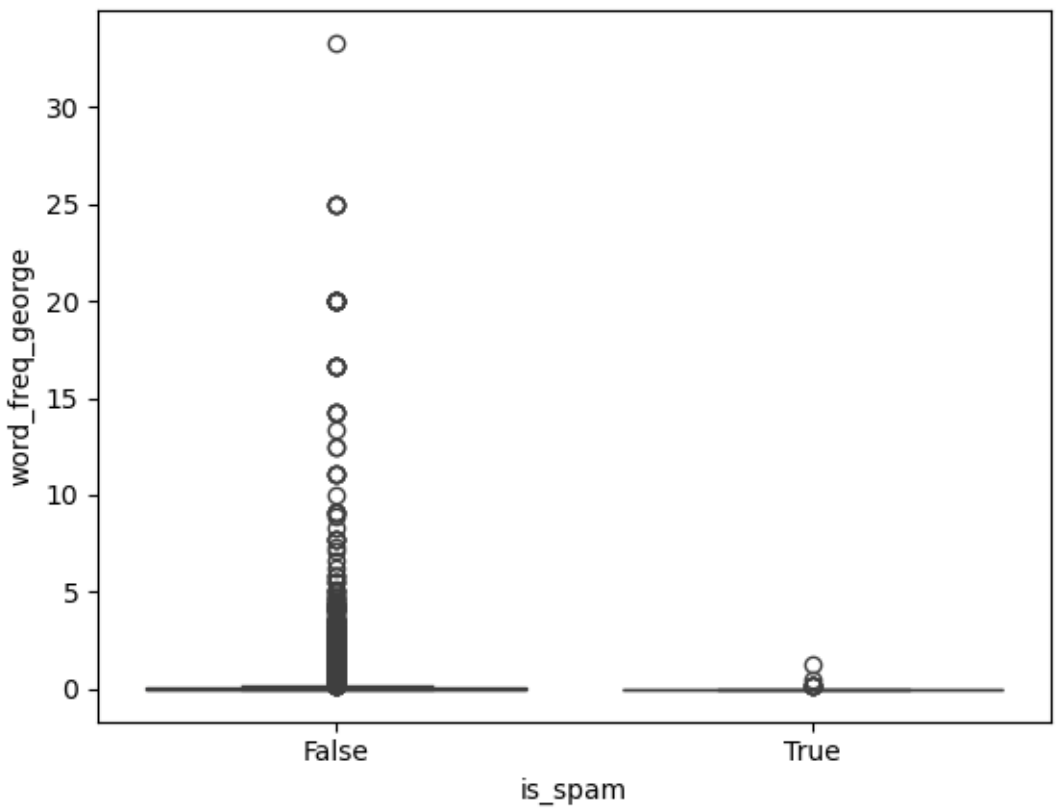
[45]:
```
#25
sns.boxplot(y="word_freq_hp", x="is_spam", data=dff)
plt.show()
```



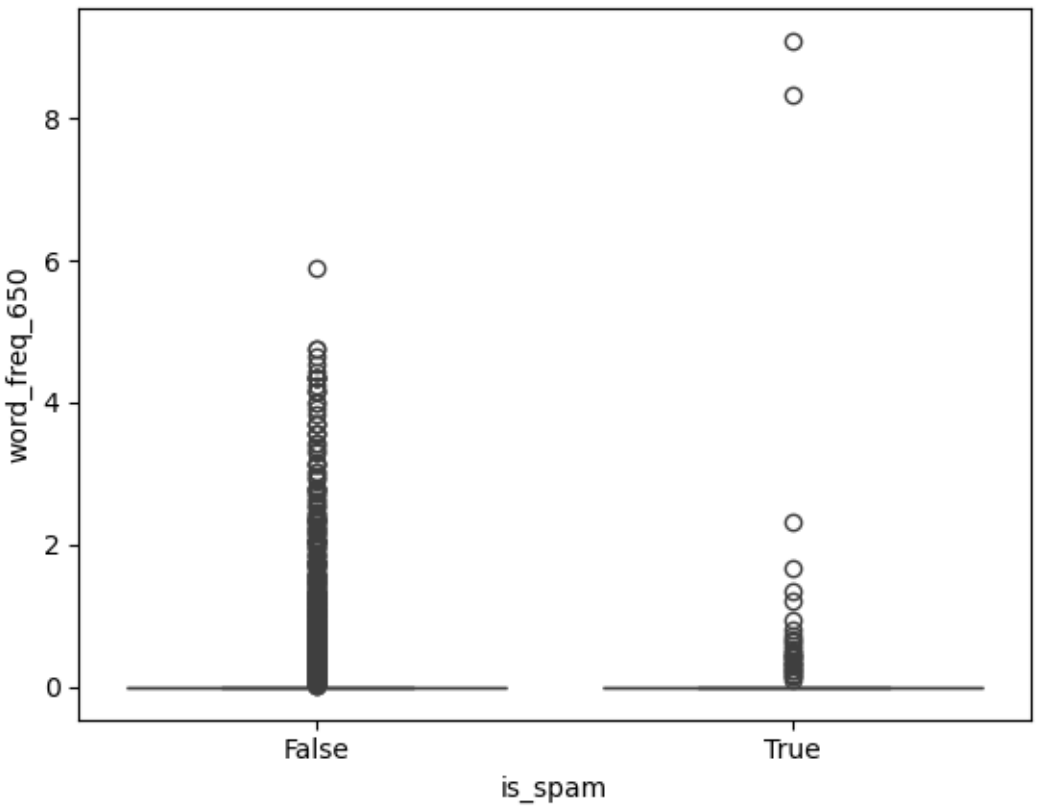
[46]:
```
#26
sns.boxplot(y="word_freq_hpl", x="is_spam", data=dff)
plt.show()
```



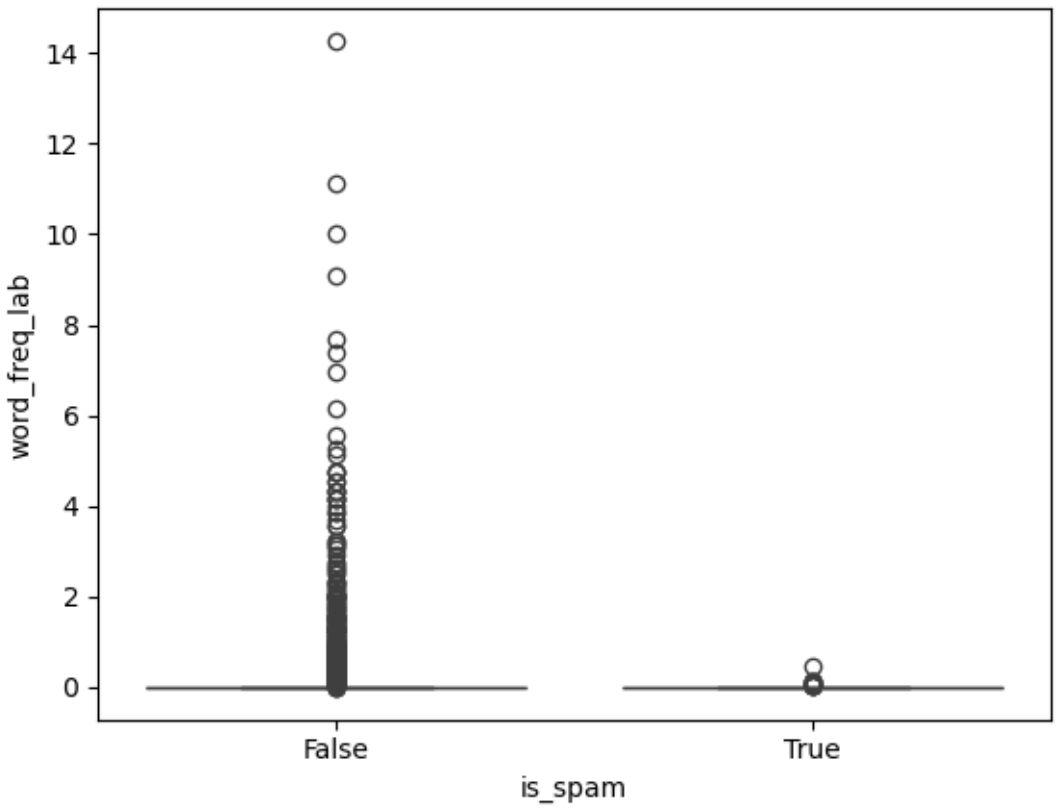
[47]:
```
#27
sns.boxplot(y="word_freq_george", x="is_spam", data=dff)
plt.show()
```



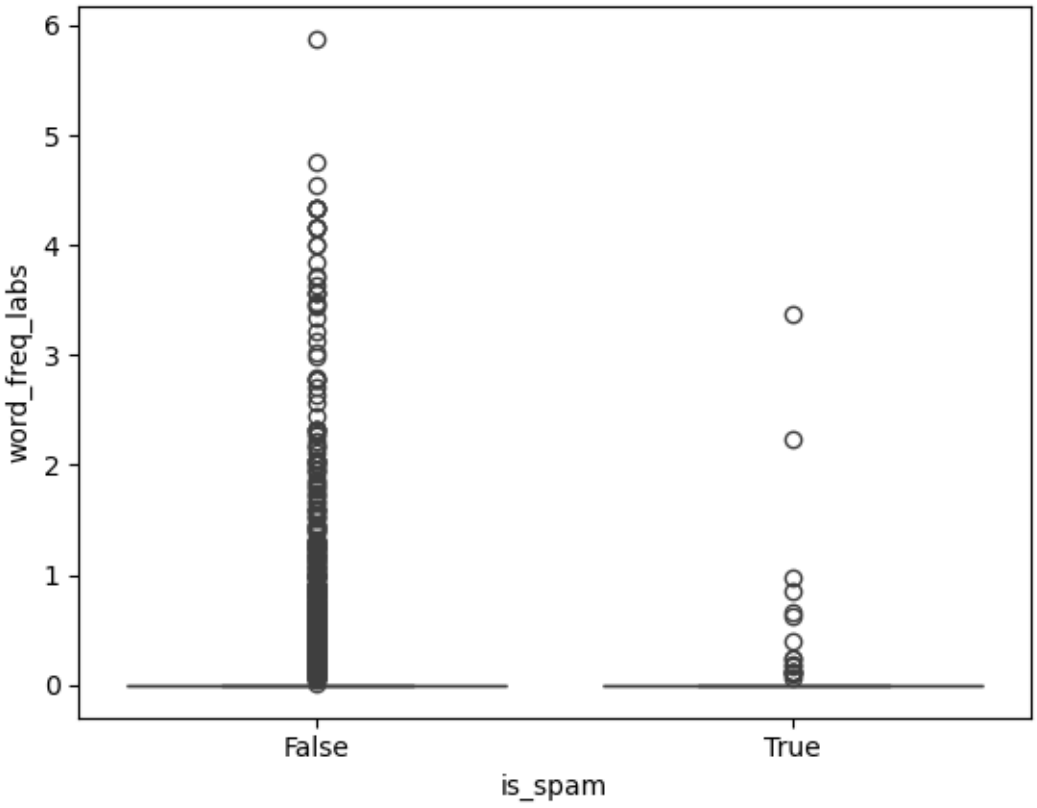
[48]:
```
#28
sns.boxplot(y="word_freq_650", x="is_spam", data=dff)
plt.show()
```



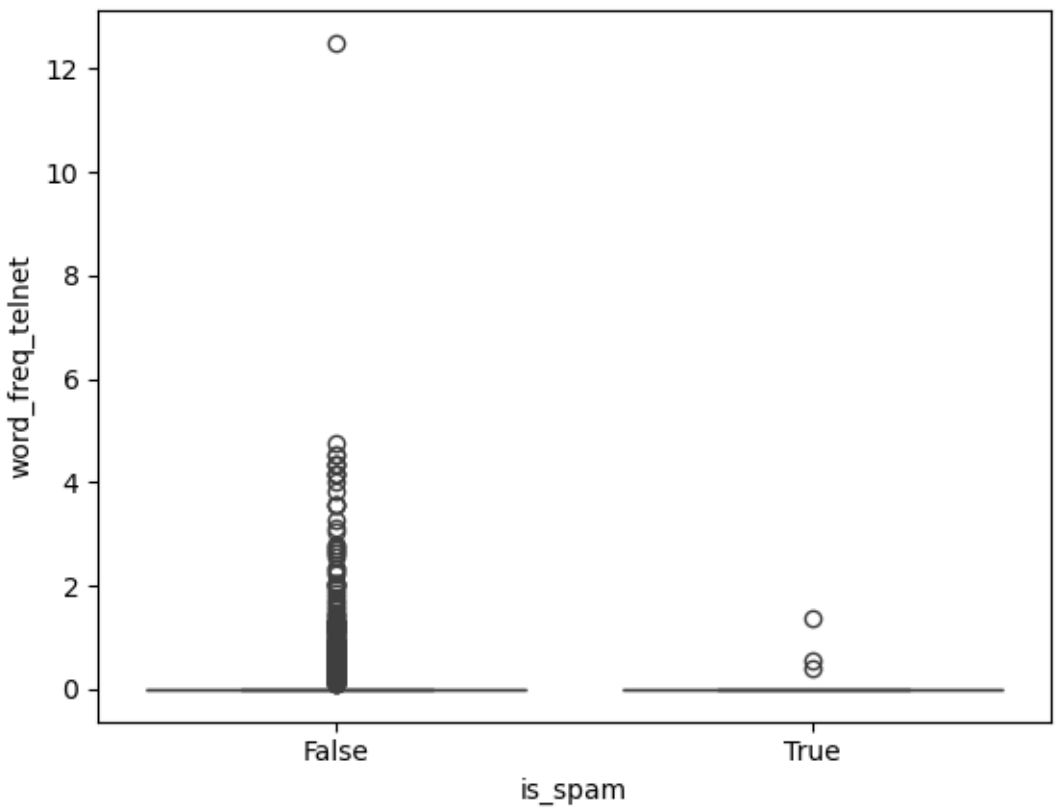
[49]:
```
#29
sns.boxplot(y="word_freq_lab", x="is_spam", data=dff)
plt.show()
```



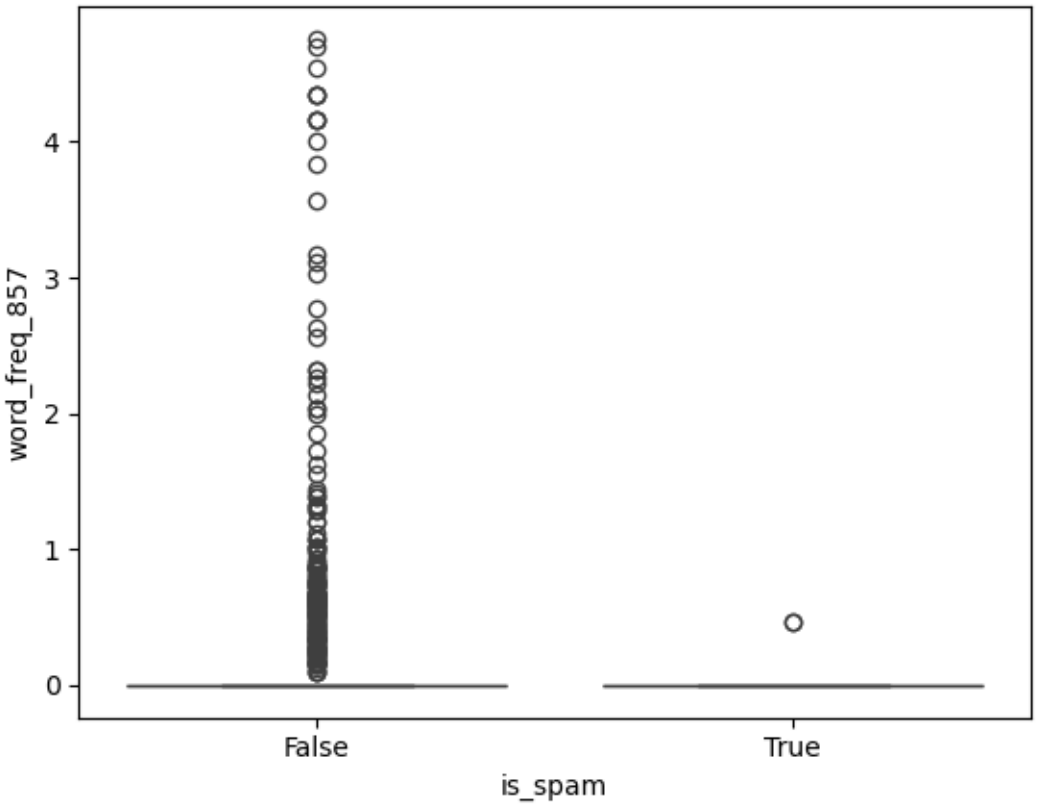
[50]:
```
#30
sns.boxplot(y="word_freq_labs", x="is_spam", data=dff)
plt.show()
```



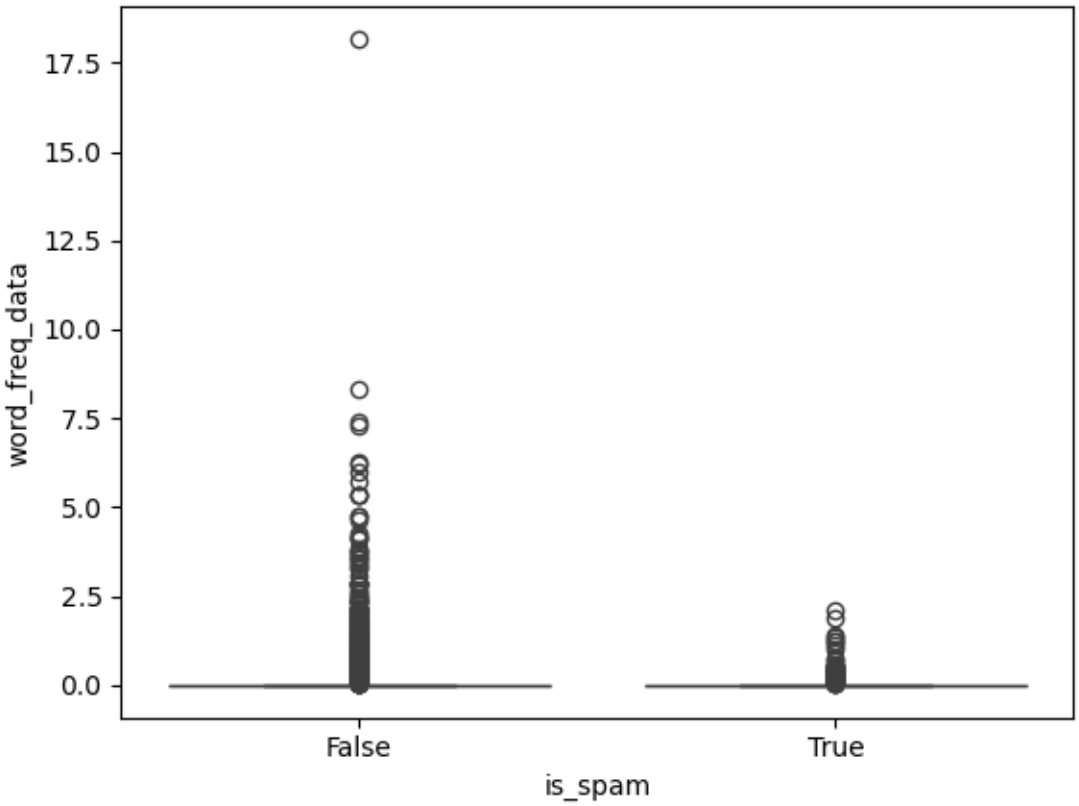
[51]:
```
#31
sns.boxplot(y="word_freq_telnet", x="is_spam", data=dff)
plt.show()
```



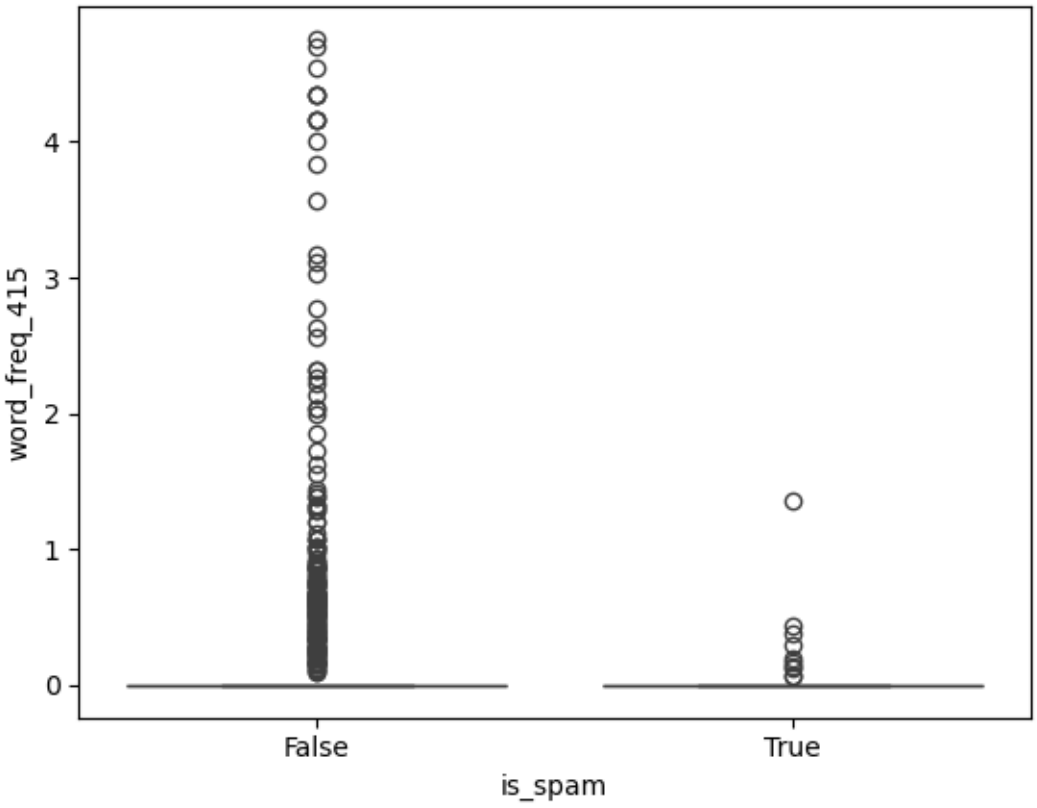
[52]:
```
#32
sns.boxplot(y="word_freq_857", x="is_spam", data=dff)
plt.show()
```



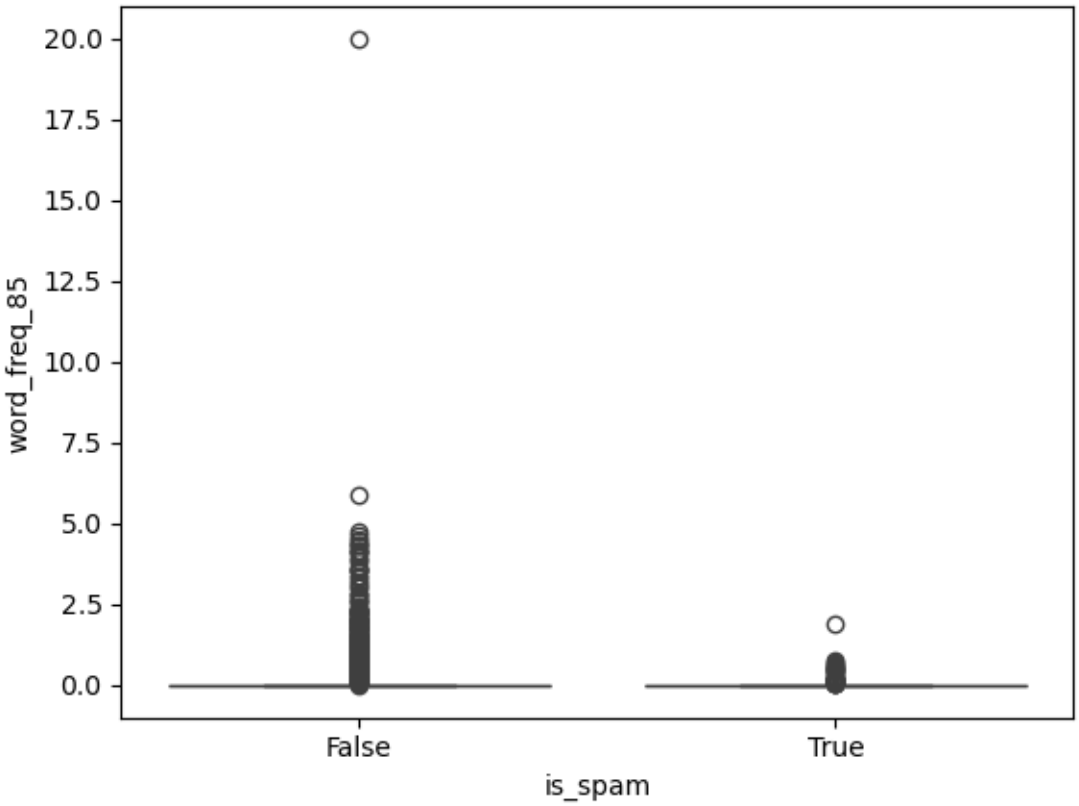
[53]:
```
#33
sns.boxplot(y="word_freq_data", x="is_spam", data=dff)
plt.show()
```



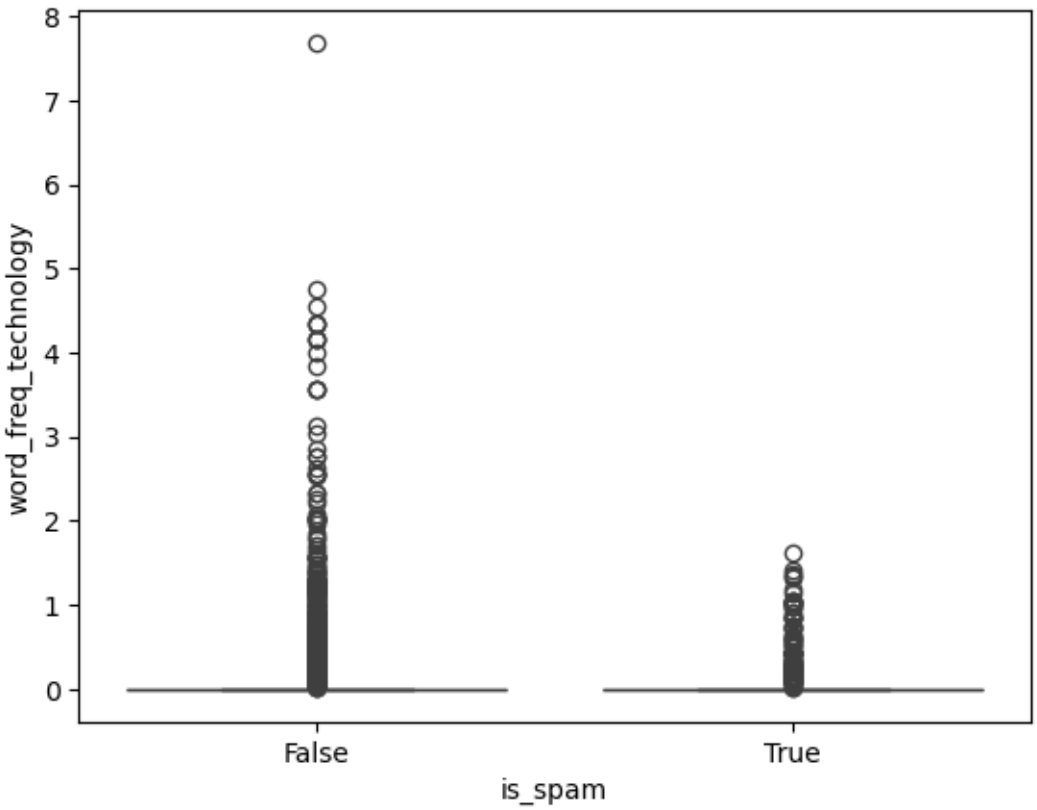
[54]:
```
#34
sns.boxplot(y="word_freq_415", x="is_spam", data=dff)
plt.show()
```



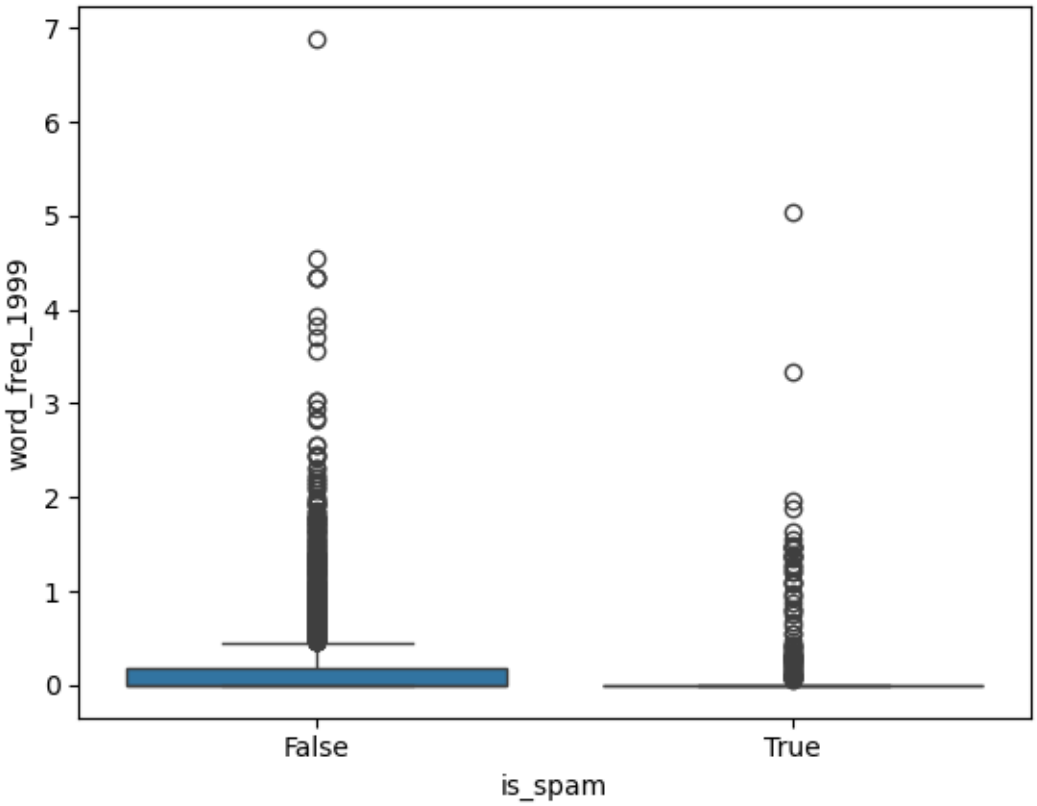
[55]:
```
#35
sns.boxplot(y="word_freq_85", x="is_spam", data=dff)
plt.show()
```



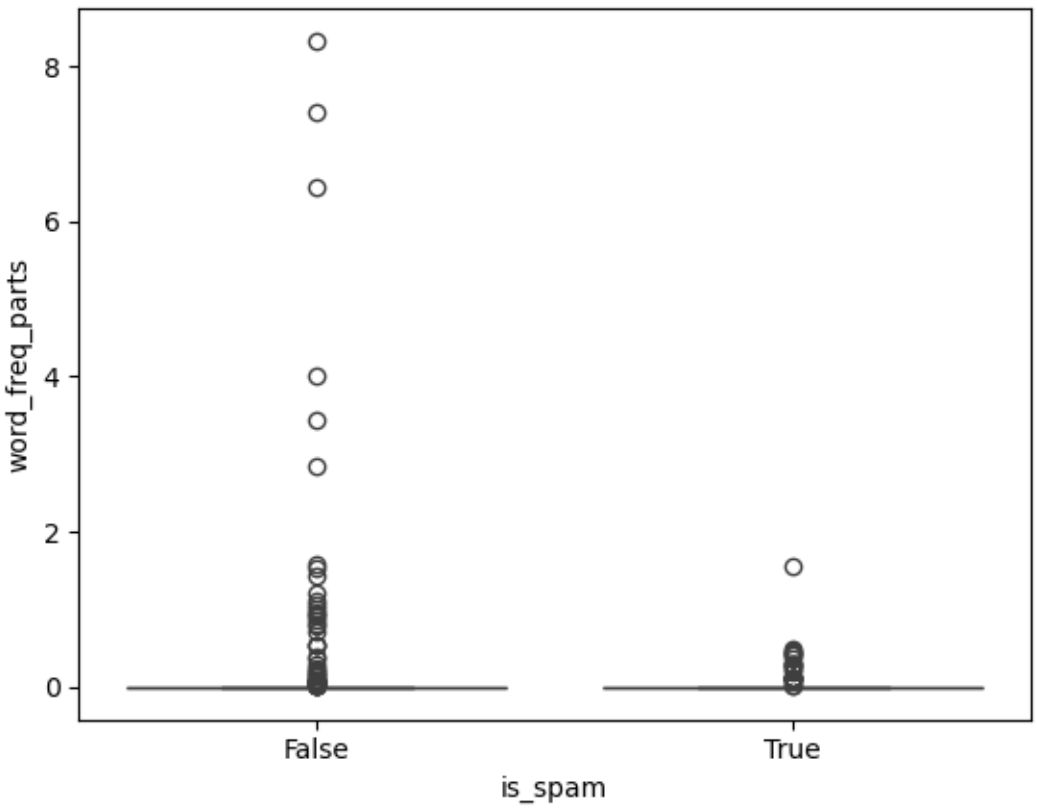
[56]:
```
#36
sns.boxplot(y="word_freq_technology", x="is_spam", data=dff)
plt.show()
```



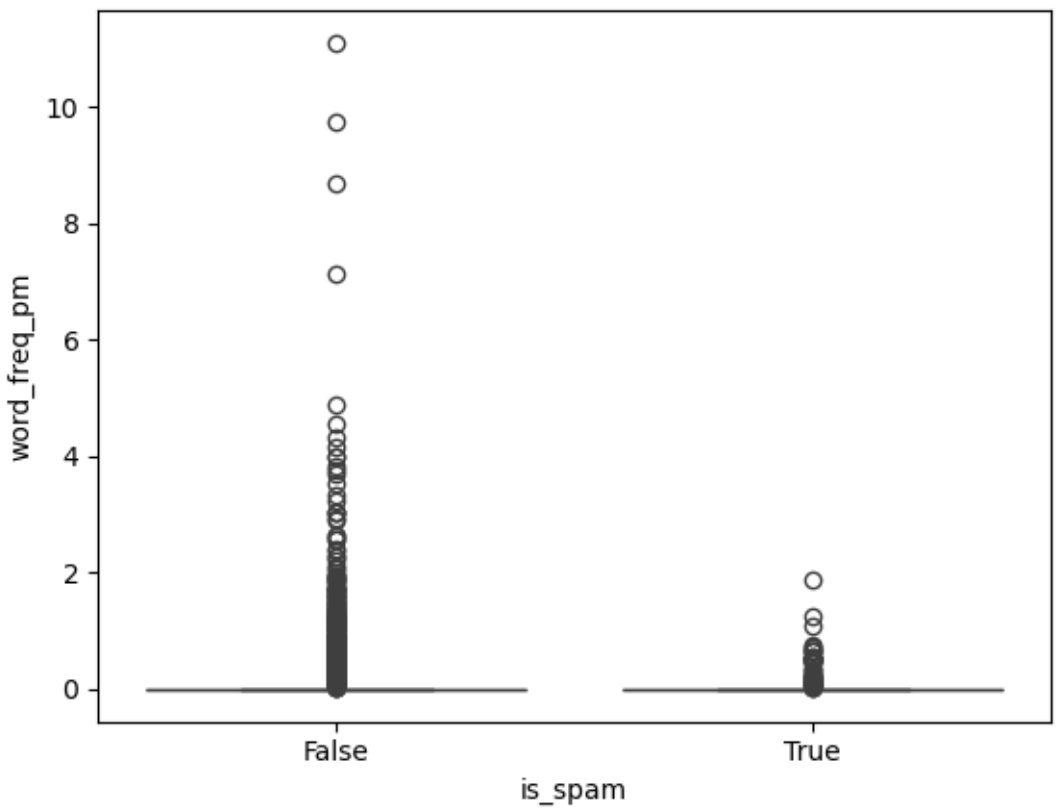
[57]:
```
#37
sns.boxplot(y="word_freq_1999", x="is_spam", data=dff)
plt.show()
```



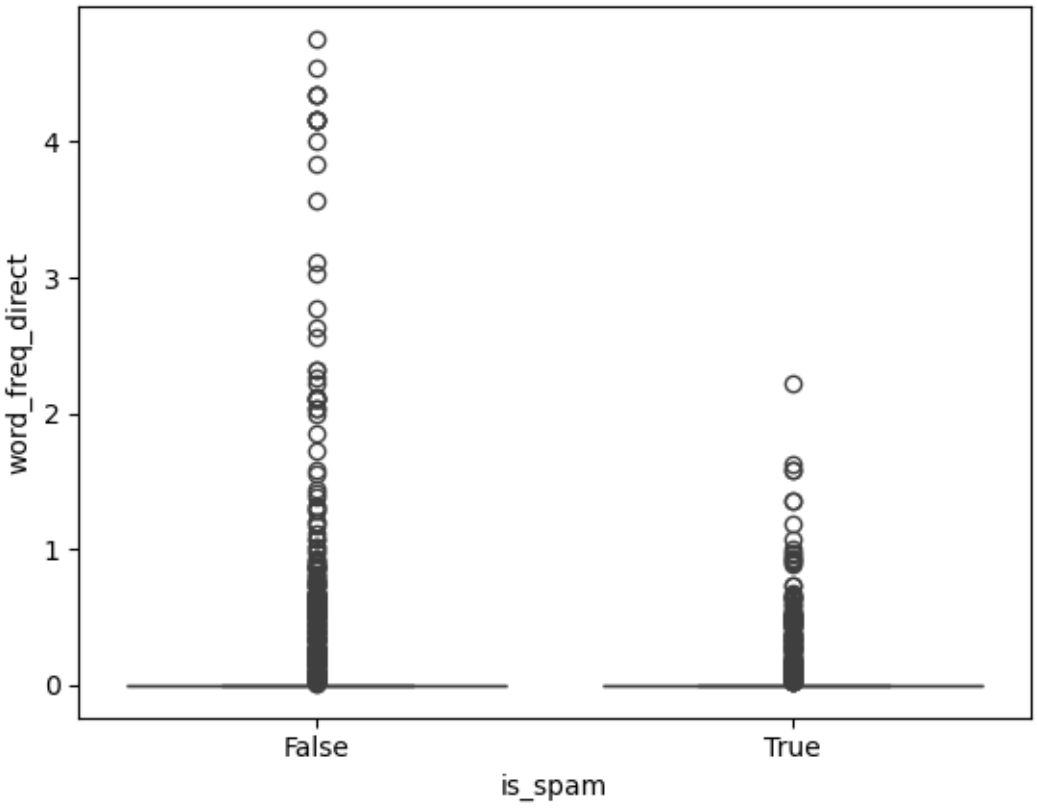
[58]:
```
#38
sns.boxplot(y="word_freq_parts", x="is_spam", data=dff)
plt.show()
```



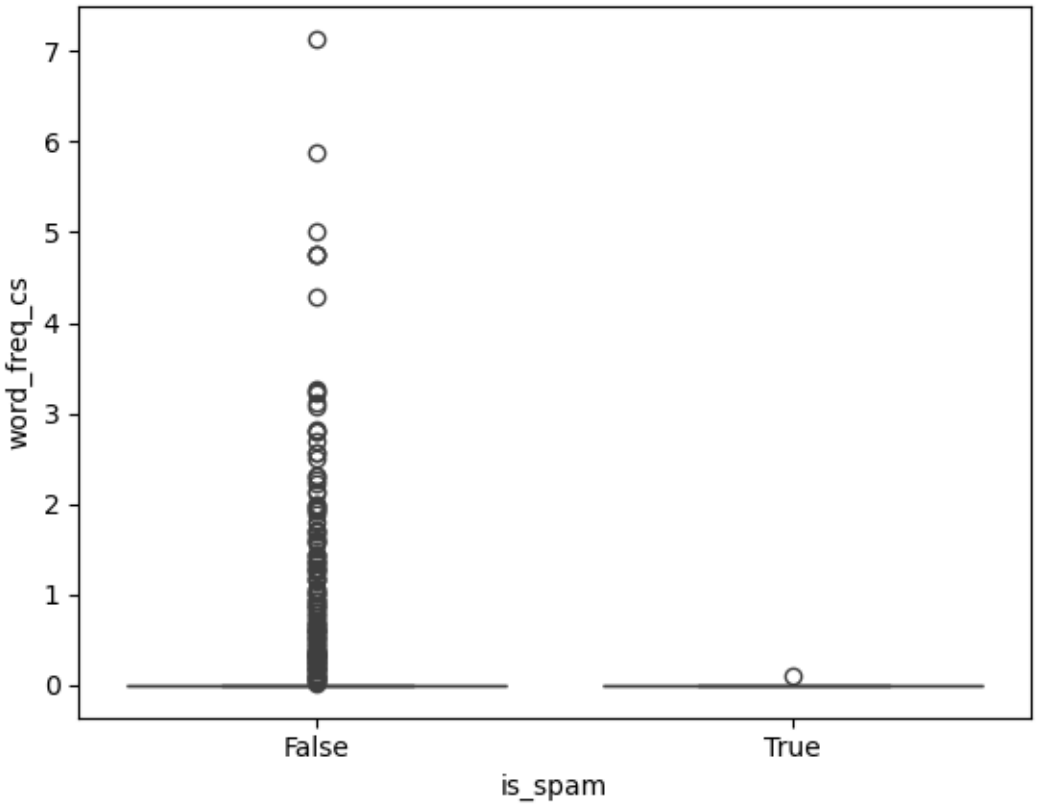
[59]:
```
#39
sns.boxplot(y="word_freq_pm", x="is_spam", data=dff)
plt.show()
```



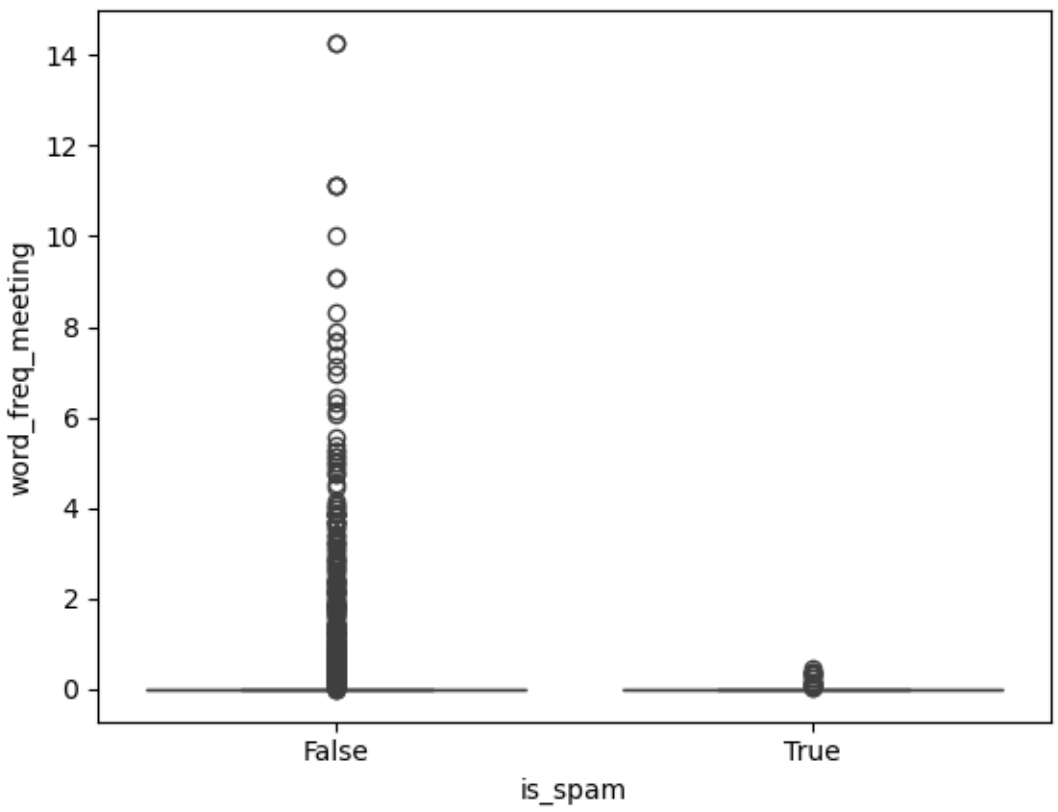
[60]:
```
#40
sns.boxplot(y="word_freq_direct", x="is_spam", data=dff)
plt.show()
```



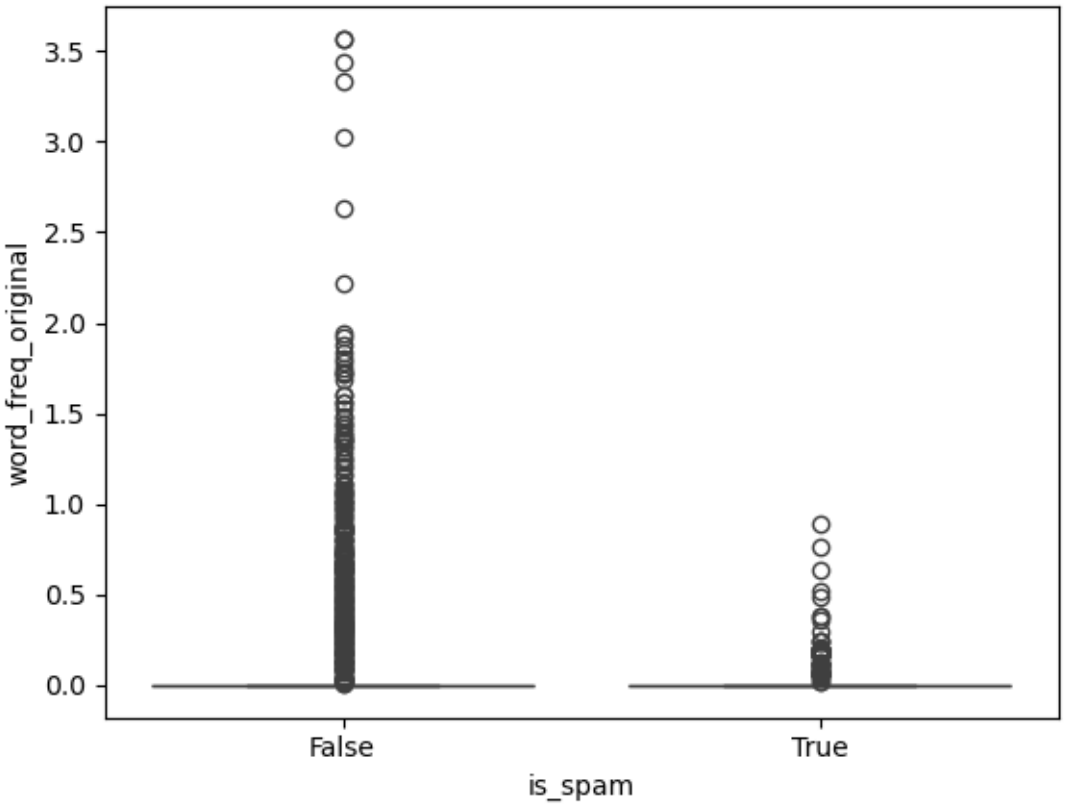
[61]:
```
#41
sns.boxplot(y="word_freq_cs", x="is_spam", data=dff)
plt.show()
```



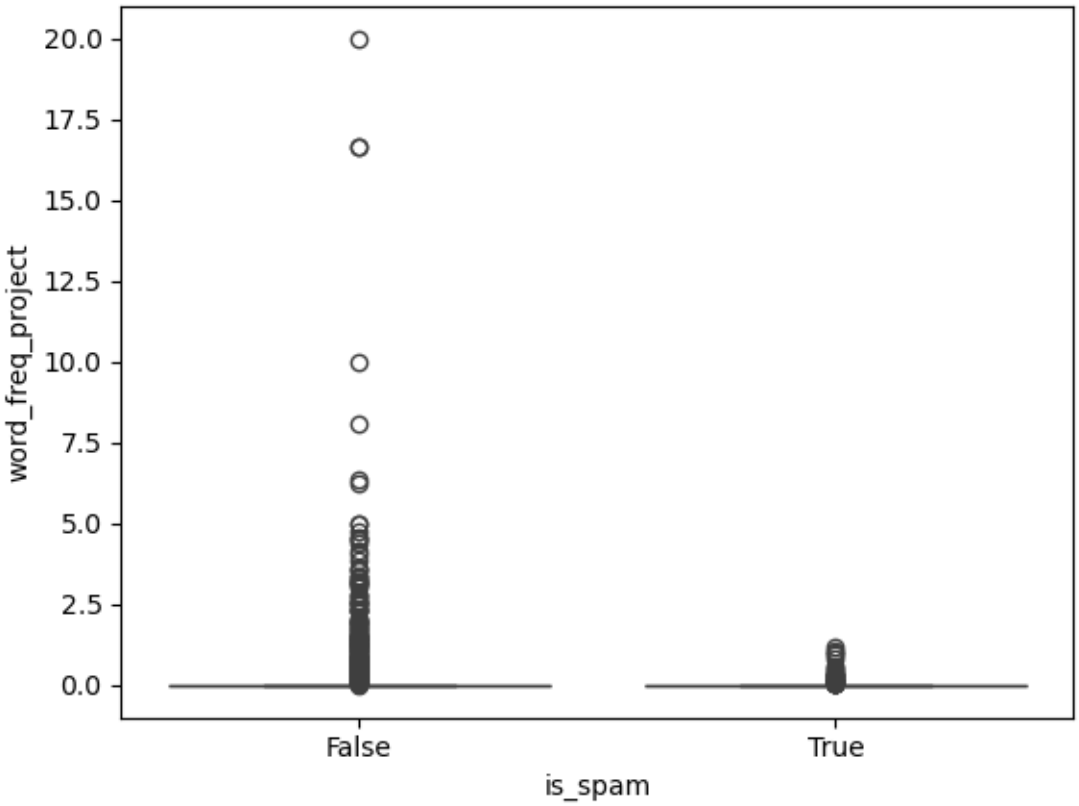
[62]:
```
#42
sns.boxplot(y="word_freq_meeting", x="is_spam", data=dff)
plt.show()
```



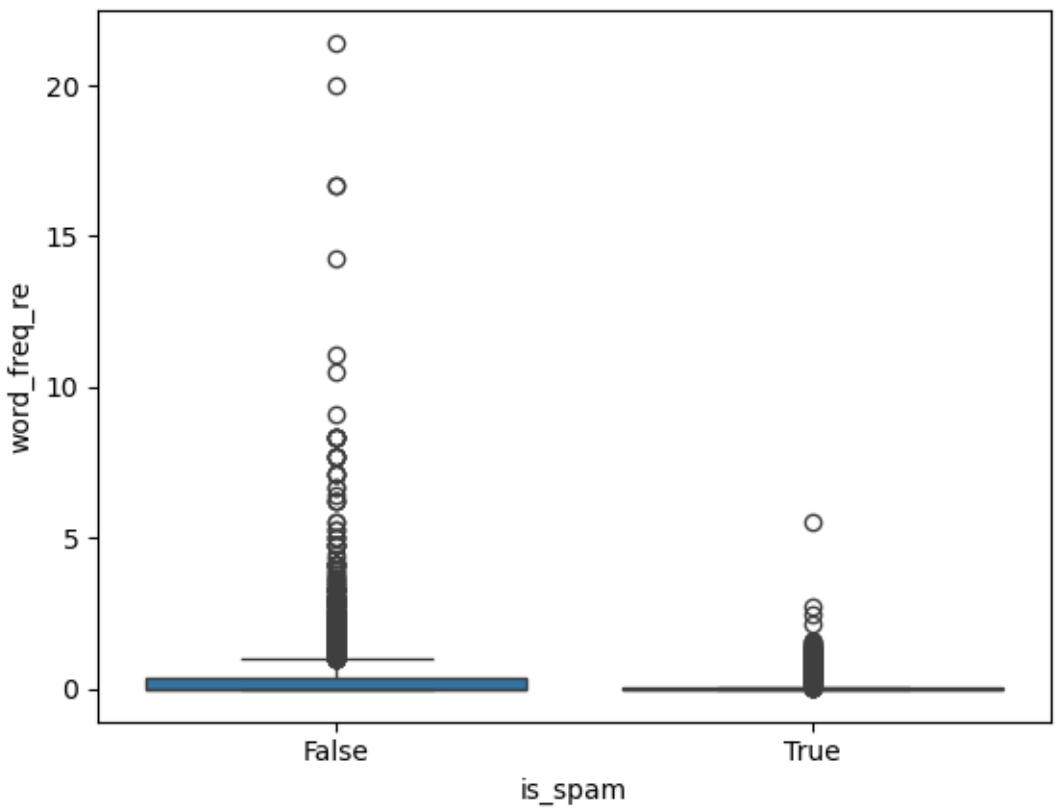
[63]:
```
#43
sns.boxplot(y="word_freq_original", x="is_spam", data=dff)
plt.show()
```



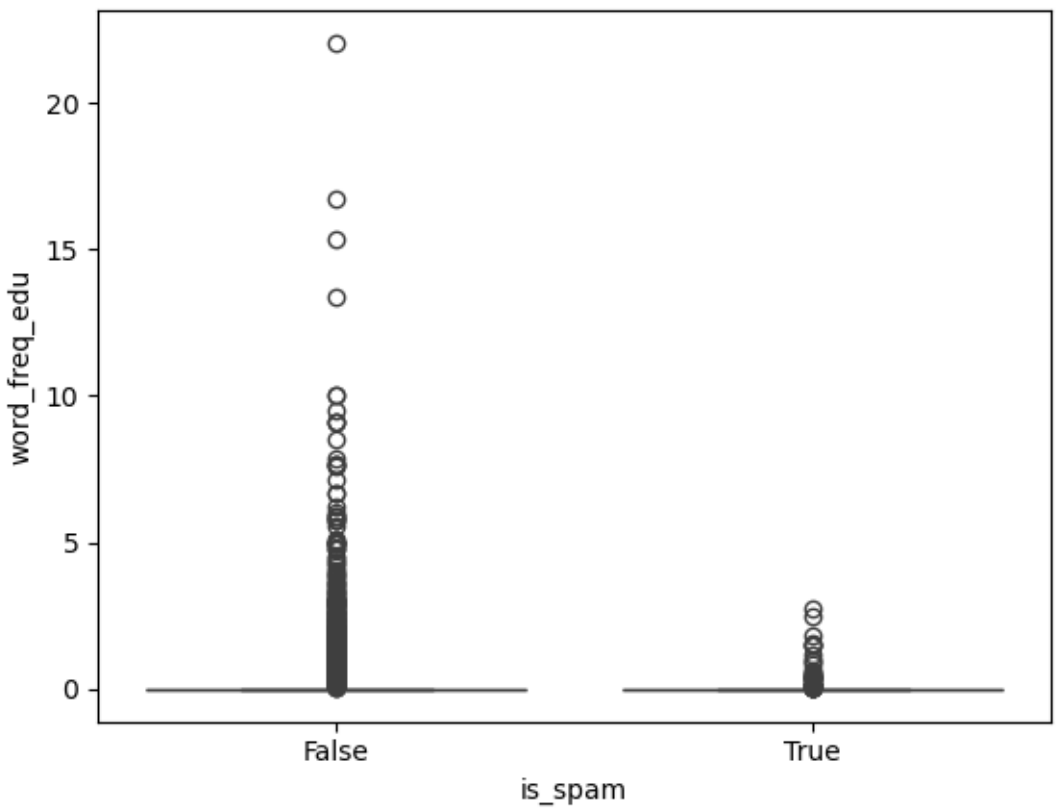
[64]:
```
#44
sns.boxplot(y="word_freq_project", x="is_spam", data=dff)
plt.show()
```



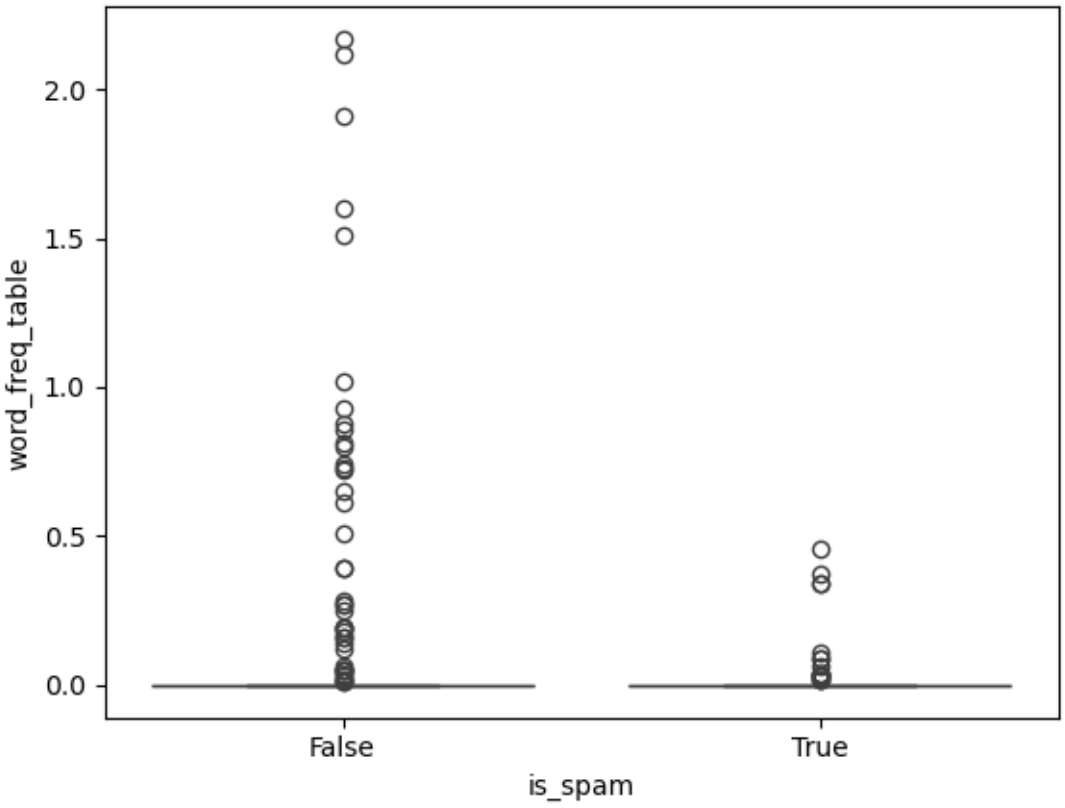
[65]:
```
#45
sns.boxplot(y="word_freq_re", x="is_spam", data=dff)
plt.show()
```



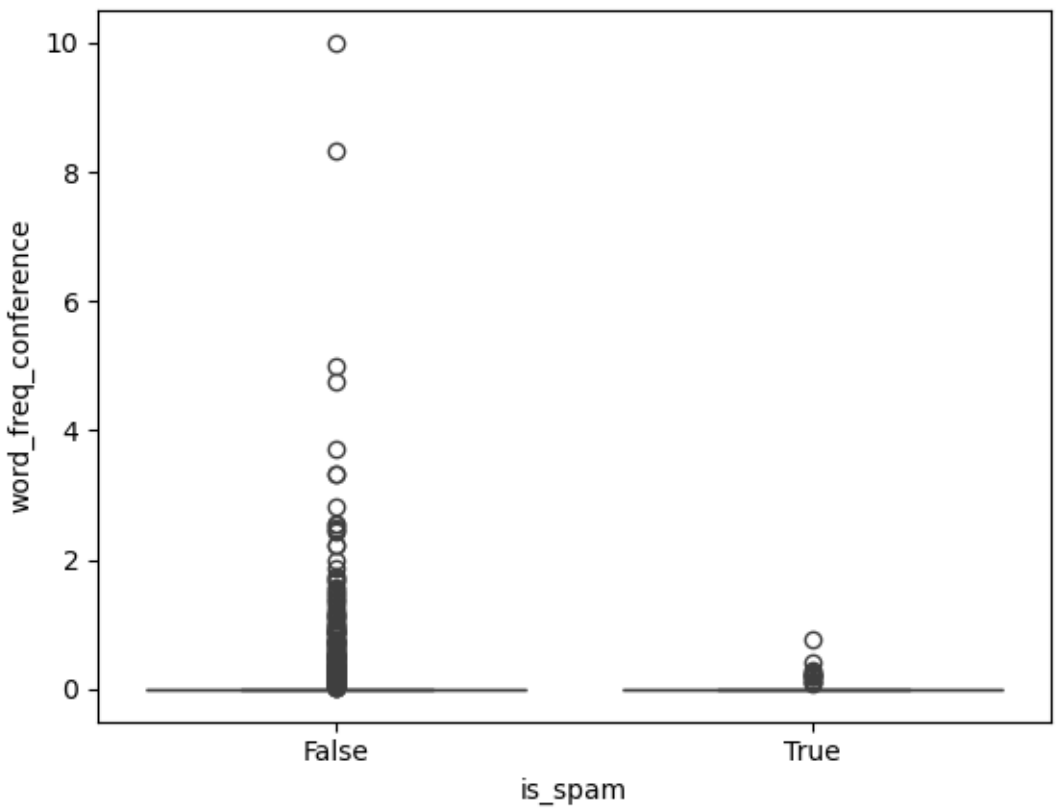
[66]:
```
#46
sns.boxplot(y="word_freq_edu", x="is_spam", data=dff)
plt.show()
```



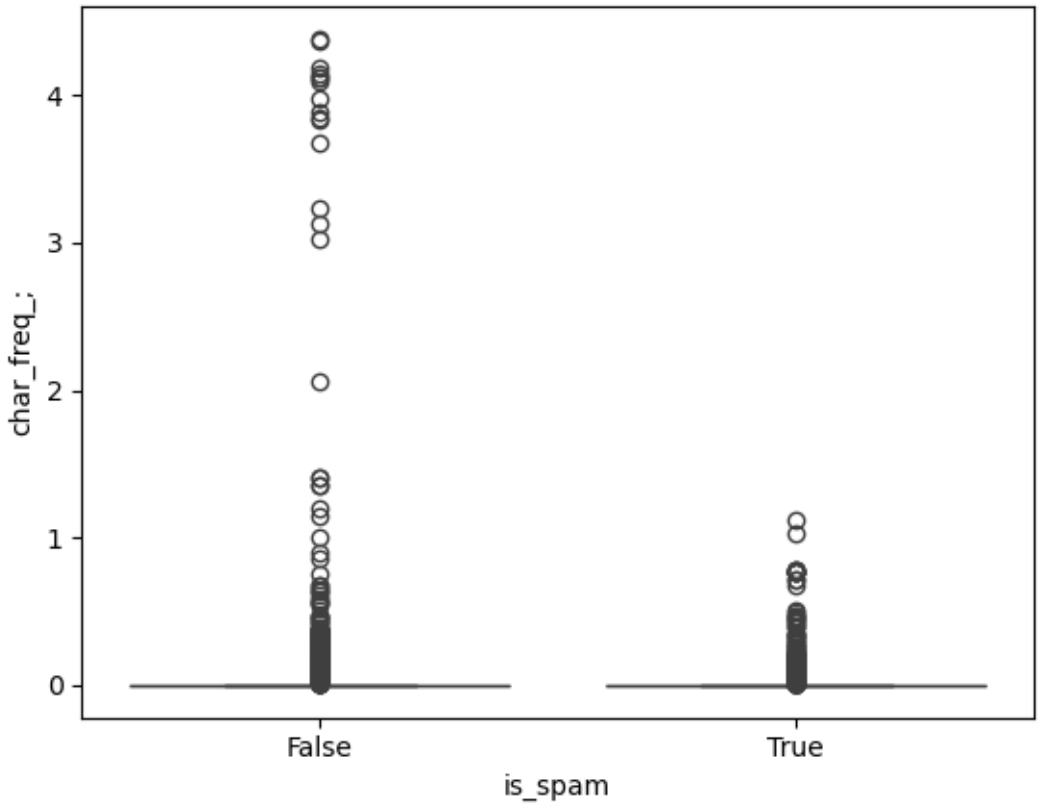
[67]:
```
#47
sns.boxplot(y="word_freq_table", x="is_spam", data=dff)
plt.show()
```



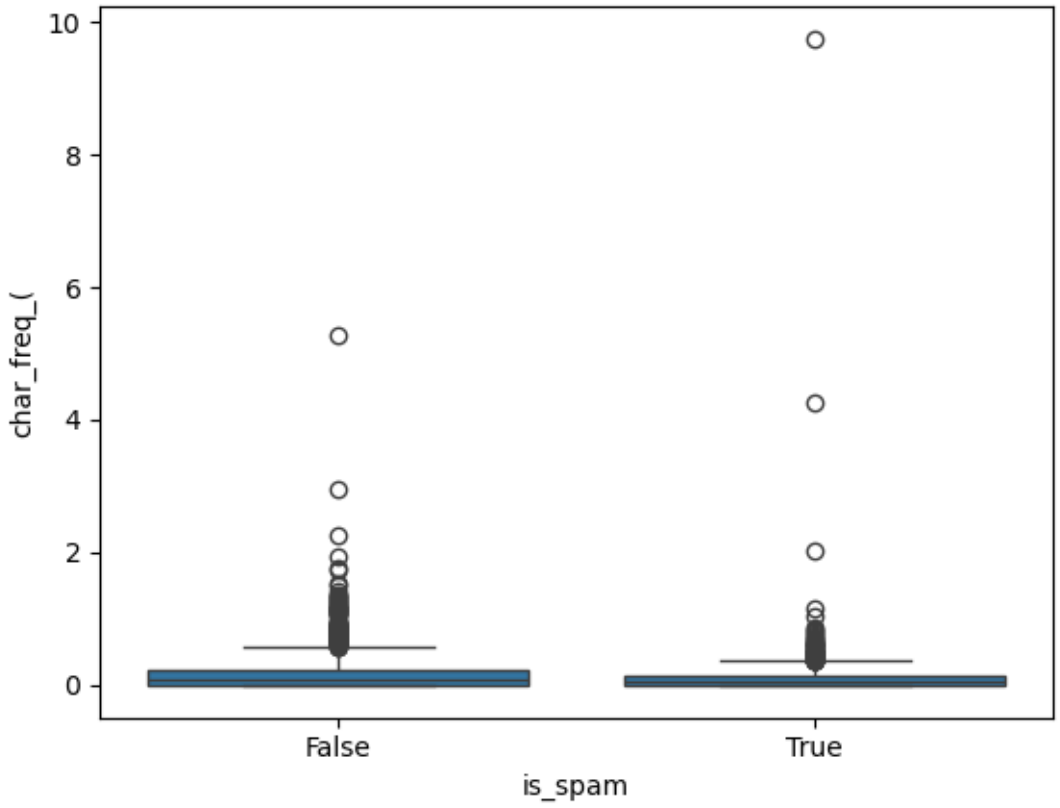
[68]:
```
#48
sns.boxplot(y="word_freq_conference", x="is_spam", data=dff)
plt.show()
```



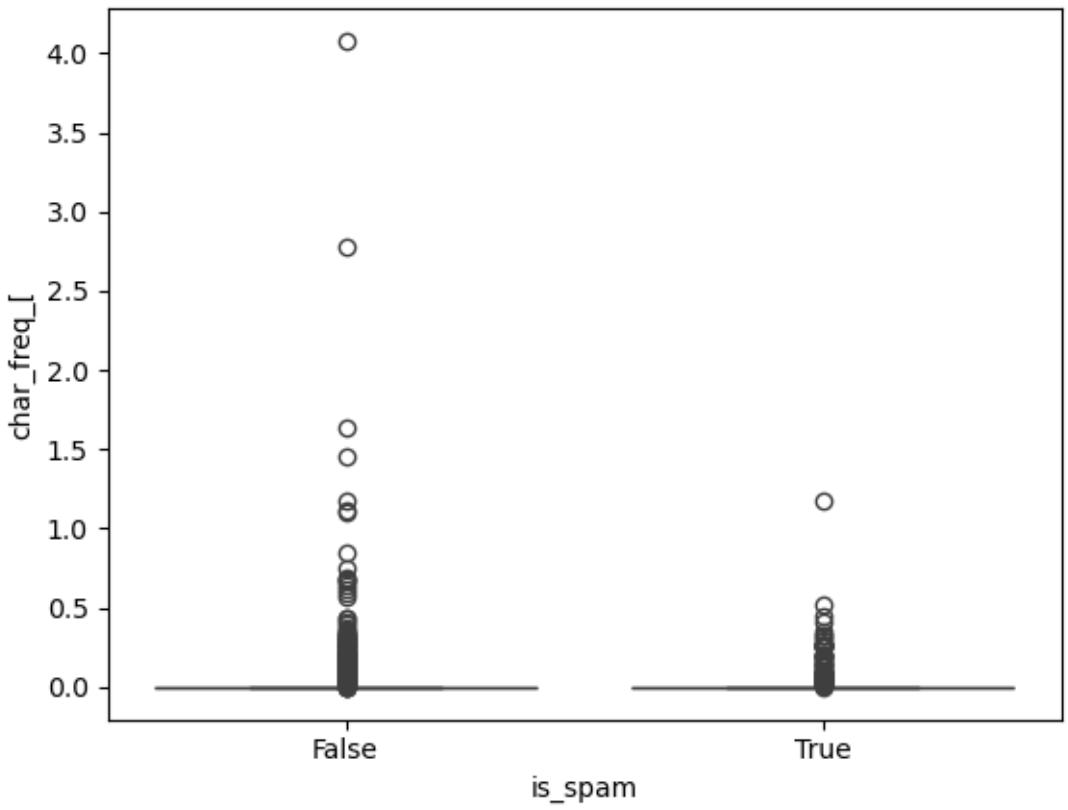
[69]:
```
#49
sns.boxplot(y="char_freq_;", x="is_spam", data=dff)
plt.show()
```



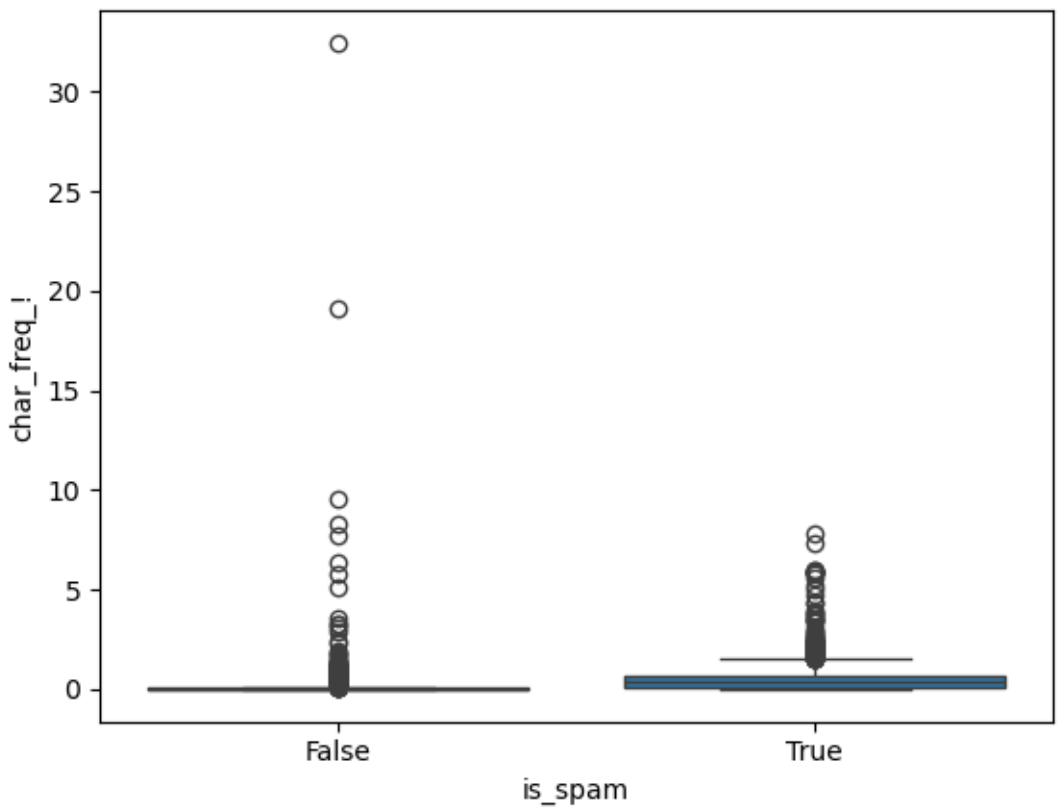
[70]:
```
#50
sns.boxplot(y="char_freq_(", x="is_spam", data=dff)
plt.show()
```



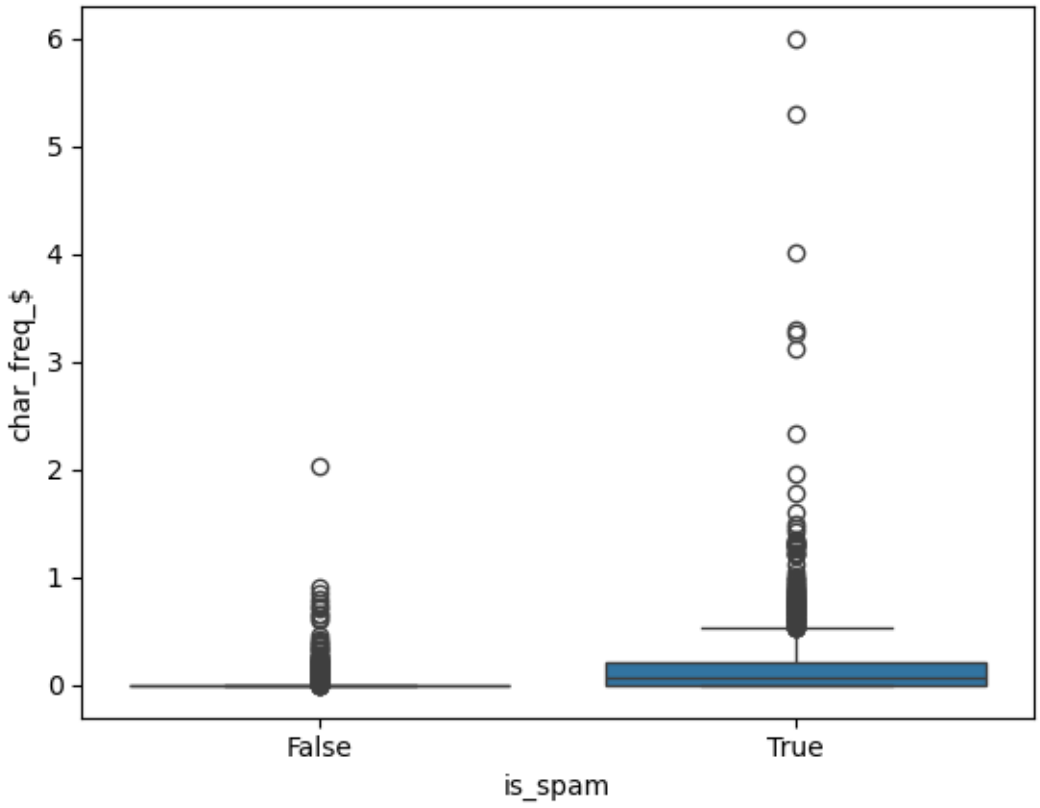
[71]:
```
#51
sns.boxplot(y="char_freq_[", x="is_spam", data=dff)
plt.show()
```



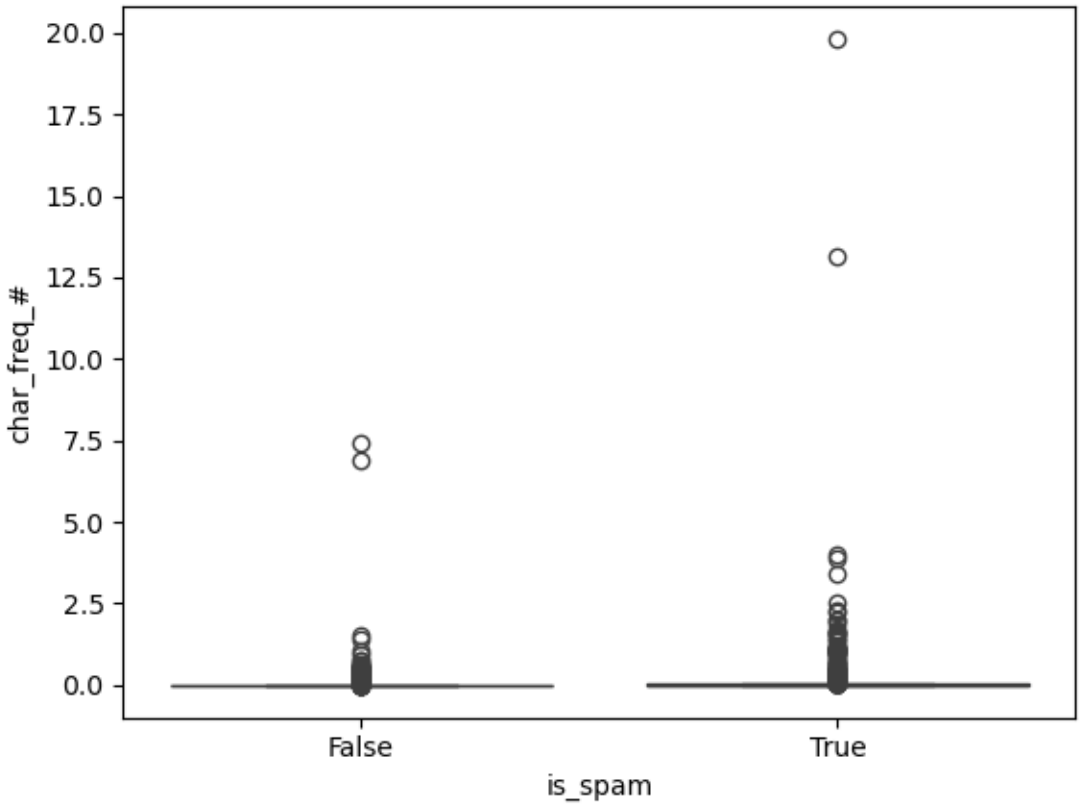
[72]:
```
#52
sns.boxplot(y="char_freq_!", x="is_spam", data=dff)
plt.show()
```



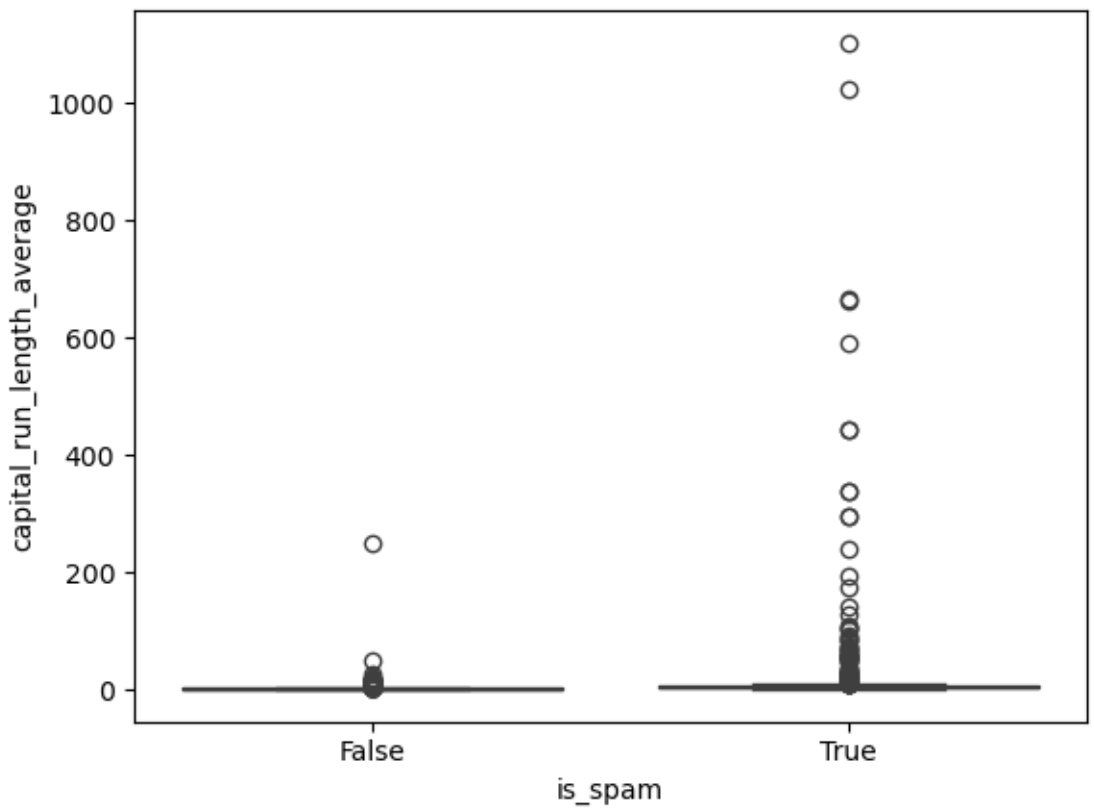
[73]:
```
#53
sns.boxplot(y="char_freq_$", x="is_spam", data=dff)
plt.show()
```



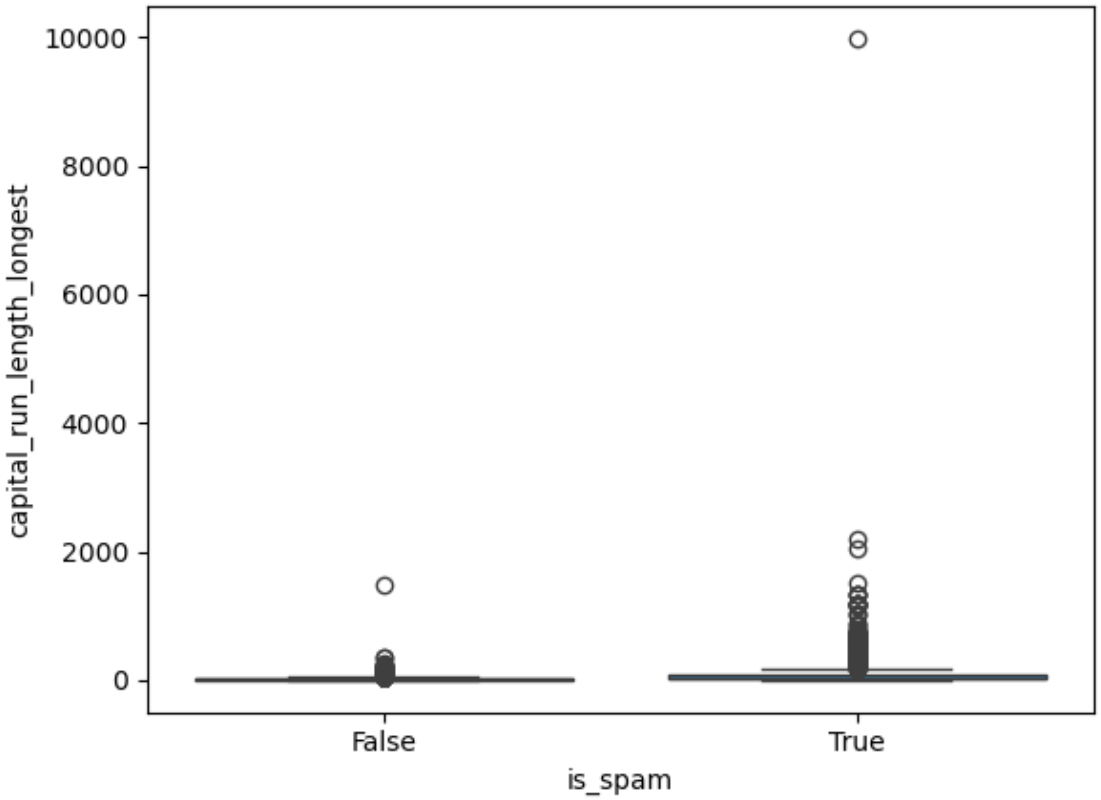
[74]:
```
#54
sns.boxplot(y="char_freq_#", x="is_spam", data=dff)
plt.show()
```



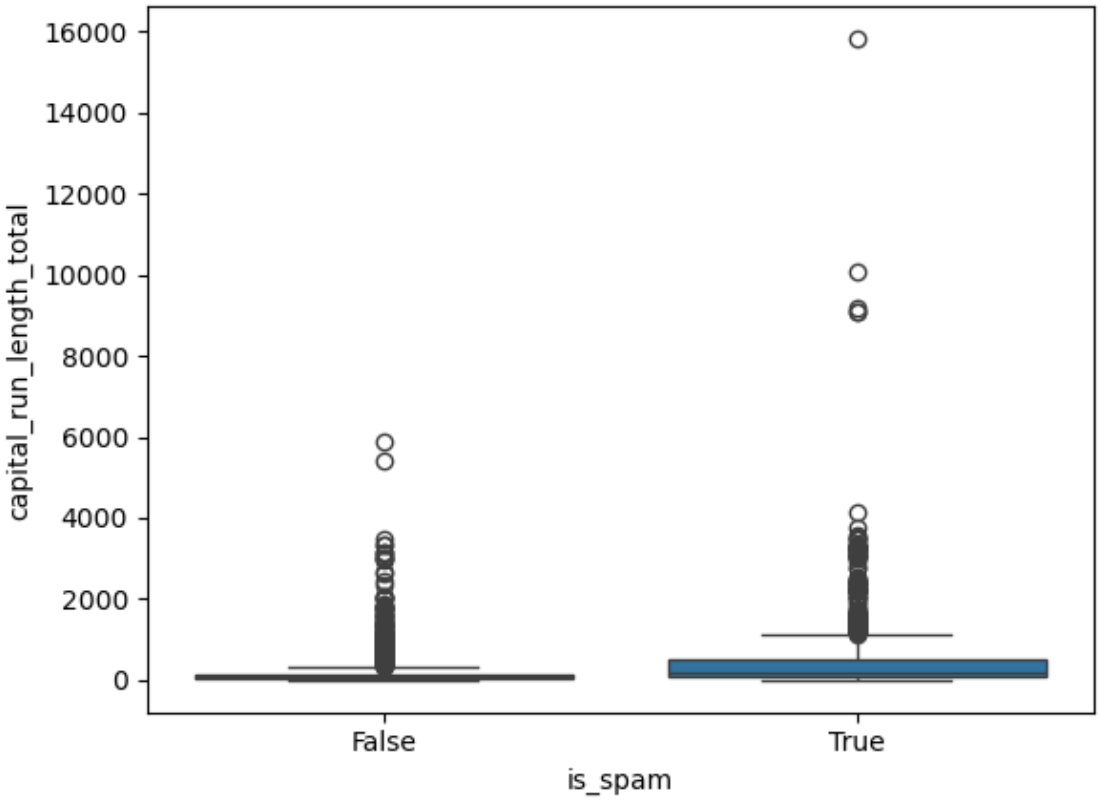
[75]:
```
#55
sns.boxplot(y="capital_run_length_average", x="is_spam", data=dff)
plt.show()
```



[76]:
```
#56
sns.boxplot(y="capital_run_length_longest", x="is_spam", data=dff)
plt.show()
```



[77]:
```
#57
sns.boxplot(y="capital_run_length_total", x="is_spam", data=dff)
plt.show()
```



2 Outlier Analysis

- Boxplots were generated for each numerical attribute against the "is_spam" column to identify the presence and distribution of outliers. A key finding emerged: the majority of outliers are associated with spam emails rather than legitimate messages.
- **Key Observation:** In this dataset, outliers represent legitimate signal rather than data quality issues. The extreme values in features such as "capital_run_length_longest" are characteristic of spam behavior patterns, not measurement errors. On the other hand, the low presence of outliers or not-so-extreme values in attributes such as "word_freq_george" indicate that the email may not be spam.
- **Decision:** Outliers will be retained in the dataset as they contain important discriminative information for classification.

3 Categorical Data

[78]:
```
import sklearn.preprocessing as preproc
```

[79]:
```
le=preproc.LabelEncoder()
```

[80]:
```
dff['is_spam'].unique()
```

[80]:
```
array([ True, False])
```

[81]:
```
dff['is_spam']=le.fit_transform(dff['is_spam'])
```

[82]:
```
dff['is_spam'].unique()
```

[82]:
```
array([1, 0])
```

- **As the file contains only one feature with categorical data, and this feature has only two values, I used the "LabelEncoder" method to simplify visualization and avoid splitting the target feature into two separate columns.**

4 Feature Selection

- **There are 58 features in this dataset. To understand the importance of all these features, I decided to apply the "VarianceThreshold" method to assess each feature's importance and possibly remove the least important ones for the dataset analysis.**

[83]:
```
dff.head()
```

[83]:

	word_freq_make	word_freq_address	word_freq_all	word_freq_3d	\
0	0.00	0.64	0.64	0.0	
1	0.21	0.28	0.50	0.0	
2	0.06	0.00	0.71	0.0	
3	0.00	0.00	0.00	0.0	
4	0.00	0.00	0.00	0.0	

	word_freq_our	word_freq_over	word_freq_remove	word_freq_internet	\
0	0.32	0.00	0.00	0.00	
1	0.14	0.28	0.21	0.07	
2	1.23	0.19	0.19	0.12	
3	0.63	0.00	0.31	0.63	
4	0.63	0.00	0.31	0.63	

	word_freq_order	word_freq_mail	…	char_freq_;	char_freq_(	\
0	0.00	0.00	…	0.00	0.000	
1	0.00	0.94	…	0.00	0.132	
2	0.64	0.25	…	0.01	0.143	
3	0.31	0.63	…	0.00	0.137	
4	0.31	0.63	…	0.00	0.135	

	char_freq_[	char_freq_!	char_freq_$	char_freq_#	\
0	0.0	0.778	0.000	0.000	
1	0.0	0.372	0.180	0.048	
2	0.0	0.276	0.184	0.010	

```
3          0.0        0.137        0.000        0.000
4          0.0        0.135        0.000        0.000

   capital_run_length_average  capital_run_length_longest  \
0                       3.756                          61
1                       5.114                         101
2                       9.821                         485
3                       3.537                          40
4                       3.537                          40

   capital_run_length_total  is_spam
0                       278        1
1                      1028        1
2                      2259        1
3                       191        1
4                       191        1

[5 rows x 58 columns]
```

[84]:
```
dff.isnull().sum()
```

[84]:
```
word_freq_make              0
word_freq_address           0
word_freq_all               0
word_freq_3d                0
word_freq_our               1
word_freq_over              0
word_freq_remove            0
word_freq_internet          0
word_freq_order             0
word_freq_mail              0
word_freq_receive           0
word_freq_will              0
word_freq_people            0
word_freq_report            0
word_freq_addresses         0
word_freq_free              0
word_freq_business          0
word_freq_email             0
word_freq_you               0
word_freq_credit            0
word_freq_your              0
word_freq_font              0
word_freq_000               1
word_freq_money             0
word_freq_hp                0
word_freq_hpl               1
```

```
word_freq_george              0
word_freq_650                 0
word_freq_lab                 0
word_freq_labs                1
word_freq_telnet              0
word_freq_857                 0
word_freq_data                0
word_freq_415                 0
word_freq_85                  0
word_freq_technology          0
word_freq_1999                0
word_freq_parts               0
word_freq_pm                  0
word_freq_direct              0
word_freq_cs                  0
word_freq_meeting             0
word_freq_original            0
word_freq_project             0
word_freq_re                  0
word_freq_edu                 0
word_freq_table               0
word_freq_conference          0
char_freq_;                   0
char_freq_(                   0
char_freq_[                   0
char_freq_!                   0
char_freq_$                   0
char_freq_#                   0
capital_run_length_average    0
capital_run_length_longest    0
capital_run_length_total      0
is_spam                       0
dtype: int64
```

[85]:
```
dff.fillna(0, inplace=True)
```

[86]:
```
dff.isnull().sum()
```

[86]:
```
word_freq_make                0
word_freq_address             0
word_freq_all                 0
word_freq_3d                  0
word_freq_our                 0
word_freq_over                0
word_freq_remove              0
word_freq_internet            0
word_freq_order               0
```

```
word_freq_mail               0
word_freq_receive            0
word_freq_will               0
word_freq_people             0
word_freq_report             0
word_freq_addresses          0
word_freq_free               0
word_freq_business           0
word_freq_email              0
word_freq_you                0
word_freq_credit             0
word_freq_your               0
word_freq_font               0
word_freq_000                0
word_freq_money              0
word_freq_hp                 0
word_freq_hpl                0
word_freq_george             0
word_freq_650                0
word_freq_lab                0
word_freq_labs               0
word_freq_telnet             0
word_freq_857                0
word_freq_data               0
word_freq_415                0
word_freq_85                 0
word_freq_technology         0
word_freq_1999               0
word_freq_parts              0
word_freq_pm                 0
word_freq_direct             0
word_freq_cs                 0
word_freq_meeting            0
word_freq_original           0
word_freq_project            0
word_freq_re                 0
word_freq_edu                0
word_freq_table              0
word_freq_conference         0
char_freq_;                  0
char_freq_(                  0
char_freq_[                  0
char_freq_!                  0
char_freq_$                  0
char_freq_#                  0
capital_run_length_average   0
capital_run_length_longest   0
```

```
capital_run_length_total      0
is_spam                       0
dtype: int64
```

[87]:
```
from sklearn.feature_selection import VarianceThreshold
from sklearn.feature_selection import SelectKBest
from sklearn.feature_selection import chi2
from sklearn.ensemble import ExtraTreesClassifier
```

[88]:
```
X=dff.drop(labels=['is_spam'], axis=1)
y=dff['is_spam']
```

[89]:
```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(
    dff.drop(labels=['is_spam'], axis=1),
    dff['is_spam'],
    test_size=0.3,
    random_state=0)

X_train.shape, X_test.shape
```

[89]:
```
((2952, 57), (1266, 57))
```

[90]:
```
var_thres=VarianceThreshold(threshold=0)
var_thres.fit(X_train)
```

[90]:
```
VarianceThreshold(threshold=0)
```

[91]:
```
var_thres.get_support()
```

[91]:
```
array([ True,  True,  True,  True,  True,  True,  True,  True,  True,
        True,  True,  True,  True,  True,  True,  True,  True,  True,
        True,  True,  True,  True,  True,  True,  True,  True,  True,
        True,  True,  True,  True,  True,  True,  True,  True,  True,
        True,  True,  True,  True,  True,  True,  True,  True,  True,
        True,  True,  True,  True,  True,  True,  True,  True,  True,
        True,  True,  True])
```

[92]:
```
sum(var_thres.get_support())
```

[92]:
```
57
```

[93]:
```
len(X_train.columns[var_thres.get_support()])
```

[93]:
```
57
```

[94]:
```
constant_columns = [column for column in X_train.columns
                    if column not in X_train.columns[var_thres.get_support()]]
```

```
print(len(constant_columns))
```

```
0
```

[95]:
```
for column in constant_columns:
    print(column)
```

[96]:
```
X_train.drop(constant_columns,axis=1)
```

[96]:

	word_freq_make	word_freq_address	word_freq_all	word_freq_3d	\
2	0.06	0.00	0.71	0.0	
272	0.00	0.00	0.00	0.0	
4126	0.00	0.00	0.00	0.0	
277	0.35	0.00	0.35	0.0	
3771	0.90	0.00	0.90	0.0	
…	…	…	…	…	
1033	0.00	0.00	0.48	0.0	
3264	0.00	0.00	0.00	0.0	
1653	0.71	0.35	0.71	0.0	
2607	0.00	0.00	0.87	0.0	
2732	0.00	0.00	0.13	0.0	

	word_freq_our	word_freq_over	word_freq_remove	word_freq_internet	\
2	1.23	0.19	0.19	0.12	
272	0.00	0.00	0.00	1.05	
4126	0.00	0.00	0.00	0.00	
277	0.35	0.70	0.35	1.41	
3771	0.00	0.00	0.00	0.00	
…	…	…	…	…	
1033	0.96	0.00	0.48	0.00	
3264	0.00	0.00	0.00	0.00	
1653	1.79	0.00	0.00	0.00	
2607	0.00	0.00	0.00	0.00	
2732	0.00	0.00	0.00	0.00	

	word_freq_order	word_freq_mail	…	word_freq_conference	char_freq_;	\
2	0.64	0.25	…	0.0	0.01	
272	0.00	0.00	…	0.0	0.00	
4126	0.00	0.00	…	0.0	0.00	
277	0.00	0.00	…	0.0	0.00	
3771	0.00	0.00	…	0.0	0.00	
…	…	…	…	…	…	
1033	0.00	0.00	…	0.0	0.00	
3264	0.00	0.00	…	0.0	0.00	
1653	0.00	0.35	…	0.0	0.00	
2607	0.00	0.87	…	0.0	0.00	
2732	0.13	0.00	…	0.0	0.00	

```
      char_freq_(  char_freq_[  char_freq_!  char_freq_$  char_freq_#  \
2           0.143          0.0        0.276        0.184         0.01
272         0.000          0.0        0.702        0.263         0.00
4126        0.000          0.0        0.000        0.000         0.00
277         0.000          0.0        0.411        0.000         0.00
3771        0.000          0.0        0.000        0.000         0.00
…               …            …            …            …            …
1033        0.073          0.0        0.515        0.957         0.00
3264        0.354          0.0        0.000        0.000         0.00
1653        0.061          0.0        0.000        0.000         0.00
2607        0.608          0.0        0.000        0.000         0.00
2732        0.072          0.0        0.024        0.000         0.00

      capital_run_length_average  capital_run_length_longest  \
2                          9.821                         485
272                        6.487                          47
4126                       3.043                          15
277                        2.917                          60
3771                       1.727                           5
…                              …                           …
1033                       6.833                          78
3264                       2.187                           5
1653                       8.086                         153
2607                       2.941                          11
2732                       1.666                           8

      capital_run_length_total
2                         2259
272                        266
4126                        70
277                        213
3771                        19
…                            …
1033                       328
3264                        35
1653                       186
2607                       100
2732                       190

[2952 rows x 57 columns]
```

[97]:
```
bf=SelectKBest(score_func=chi2, k=10)
best=bf.fit(X,y)
best
```

[97]:
```
SelectKBest(score_func=<function chi2 at 0x15b372020>)
```

[98]:
```
best.scores_
```

[98]:
```
array([6.09582174e+01, 7.85465128e+01, 1.15029013e+02, 3.88878218e+02,
       3.25846687e+02, 1.50148616e+02, 6.34266154e+02, 2.65013257e+02,
       1.80279322e+02, 1.27624391e+02, 1.87677767e+02, 1.12530996e+00,
       5.77915348e+01, 2.37764085e+01, 1.86728008e+02, 8.26434993e+02,
       4.03194102e+02, 2.33902418e+02, 5.07893031e+02, 4.39473009e+02,
       1.07218808e+03, 3.23504656e+02, 5.49047309e+02, 3.63522516e+02,
       1.49700203e+03, 7.24452312e+02, 1.06104813e+03, 2.53982697e+02,
       2.70128186e+02, 2.55109980e+02, 1.69486871e+02, 1.22067630e+02,
       2.04401384e+02, 1.17090400e+02, 2.62106714e+02, 1.33225516e+02,
       2.13610723e+02, 1.84530820e+01, 1.67783516e+02, 2.66474316e+01,
       1.15129790e+02, 3.81324443e+02, 9.87288755e+01, 2.03135483e+02,
       3.34955021e+02, 4.46143544e+02, 9.76540279e+00, 8.56692453e+01,
       2.97959535e+01, 2.46649549e+01, 1.33588375e+01, 5.85161975e+02,
       3.41083589e+02, 8.66864344e+01, 1.04782286e+04, 1.33843343e+05,
       2.99403024e+05])
```

[99]:
```
dfscores=pd.DataFrame(best.scores_)
dfscores
```

[99]:

	0
0	60.958217
1	78.546513
2	115.029013
3	388.878218
4	325.846687
5	150.148616
6	634.266154
7	265.013257
8	180.279322
9	127.624391
10	187.677767
11	1.125310
12	57.791535
13	23.776409
14	186.728008
15	826.434993
16	403.194102
17	233.902418
18	507.893031
19	439.473009
20	1072.188077
21	323.504656
22	549.047309
23	363.522516
24	1497.002034

```
25     724.452312
26    1061.048130
27     253.982697
28     270.128186
29     255.109980
30     169.486871
31     122.067630
32     204.401384
33     117.090400
34     262.106714
35     133.225516
36     213.610723
37      18.453082
38     167.783516
39      26.647432
40     115.129790
41     381.324443
42      98.728875
43     203.135483
44     334.955021
45     446.143544
46       9.765403
47      85.669245
48      29.795954
49      24.664955
50      13.358838
51     585.161975
52     341.083589
53      86.686434
54   10478.228617
55  133843.342837
56  299403.023979
```

[100]:
```
dfcolumns=pd.DataFrame(X.columns)
dfcolumns
```

[100]:

	0
0	word_freq_make
1	word_freq_address
2	word_freq_all
3	word_freq_3d
4	word_freq_our
5	word_freq_over
6	word_freq_remove
7	word_freq_internet
8	word_freq_order
9	word_freq_mail

```
10           word_freq_receive
11              word_freq_will
12            word_freq_people
13            word_freq_report
14         word_freq_addresses
15              word_freq_free
16          word_freq_business
17             word_freq_email
18               word_freq_you
19            word_freq_credit
20              word_freq_your
21              word_freq_font
22               word_freq_000
23             word_freq_money
24                word_freq_hp
25               word_freq_hpl
26            word_freq_george
27               word_freq_650
28               word_freq_lab
29              word_freq_labs
30            word_freq_telnet
31               word_freq_857
32              word_freq_data
33               word_freq_415
34                word_freq_85
35        word_freq_technology
36              word_freq_1999
37             word_freq_parts
38                word_freq_pm
39            word_freq_direct
40                word_freq_cs
41           word_freq_meeting
42          word_freq_original
43           word_freq_project
44                word_freq_re
45               word_freq_edu
46             word_freq_table
47        word_freq_conference
48                  char_freq_;
49                  char_freq_(
50                  char_freq_[
51                  char_freq_!
52                  char_freq_$
53                  char_freq_#
54  capital_run_length_average
55  capital_run_length_longest
56    capital_run_length_total
```

[101]:
```
feature_score=pd.concat([dfcolumns, dfscores],axis=1)
feature_score
```

[101]:

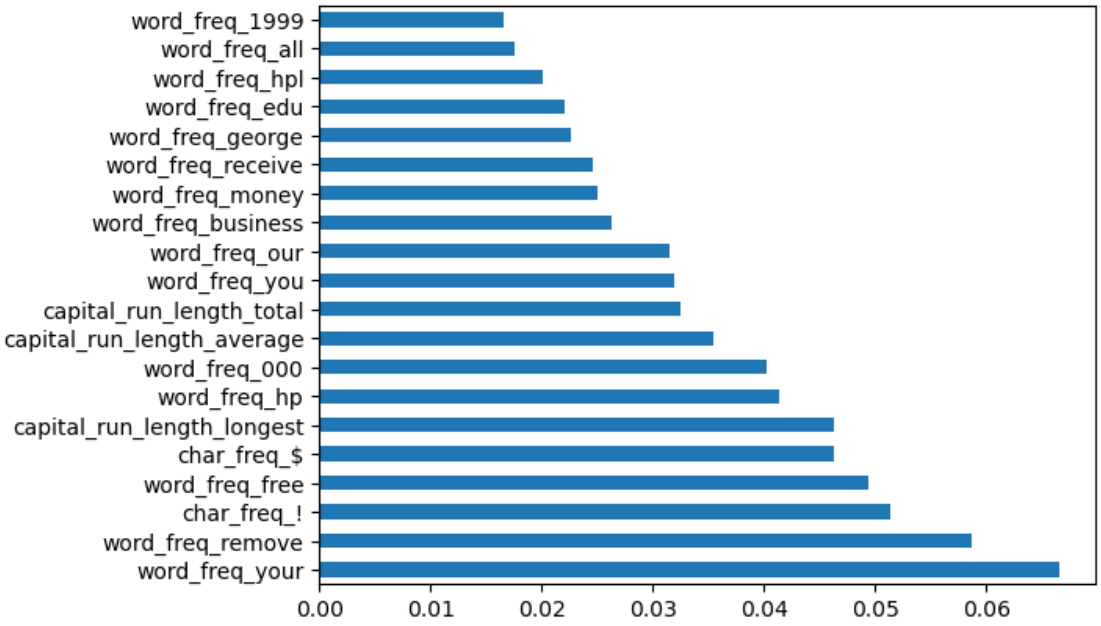
	0	0
0	word_freq_make	60.958217
1	word_freq_address	78.546513
2	word_freq_all	115.029013
3	word_freq_3d	388.878218
4	word_freq_our	325.846687
5	word_freq_over	150.148616
6	word_freq_remove	634.266154
7	word_freq_internet	265.013257
8	word_freq_order	180.279322
9	word_freq_mail	127.624391
10	word_freq_receive	187.677767
11	word_freq_will	1.125310
12	word_freq_people	57.791535
13	word_freq_report	23.776409
14	word_freq_addresses	186.728008
15	word_freq_free	826.434993
16	word_freq_business	403.194102
17	word_freq_email	233.902418
18	word_freq_you	507.893031
19	word_freq_credit	439.473009
20	word_freq_your	1072.188077
21	word_freq_font	323.504656
22	word_freq_000	549.047309
23	word_freq_money	363.522516
24	word_freq_hp	1497.002034
25	word_freq_hpl	724.452312
26	word_freq_george	1061.048130
27	word_freq_650	253.982697
28	word_freq_lab	270.128186
29	word_freq_labs	255.109980
30	word_freq_telnet	169.486871
31	word_freq_857	122.067630
32	word_freq_data	204.401384
33	word_freq_415	117.090400
34	word_freq_85	262.106714
35	word_freq_technology	133.225516
36	word_freq_1999	213.610723
37	word_freq_parts	18.453082
38	word_freq_pm	167.783516
39	word_freq_direct	26.647432
40	word_freq_cs	115.129790
41	word_freq_meeting	381.324443
42	word_freq_original	98.728875

```
43           word_freq_project     203.135483
44                word_freq_re     334.955021
45               word_freq_edu     446.143544
46             word_freq_table       9.765403
47       word_freq_conference      85.669245
48                  char_freq_;      29.795954
49                  char_freq_(      24.664955
50                  char_freq_[      13.358838
51                  char_freq_!     585.161975
52                  char_freq_$     341.083589
53                  char_freq_#      86.686434
54  capital_run_length_average   10478.228617
55  capital_run_length_longest  133843.342837
56    capital_run_length_total  299403.023979
```

[102]:
```
model = ExtraTreesClassifier()
model.fit(X,y)
print(model.feature_importances_)
```

```
[0.00788407 0.00988733 0.01750831 0.00261485 0.0315779  0.01556381
 0.05868941 0.01584767 0.0119473  0.01052141 0.02454189 0.01412783
 0.00676486 0.00429792 0.00963375 0.04934997 0.02634156 0.01591259
 0.0319353  0.01150992 0.06654919 0.00646355 0.04027238 0.02509328
 0.04139295 0.02014232 0.02268567 0.01024372 0.00607762 0.00988629
 0.00516758 0.00298775 0.00666974 0.00276192 0.0058474  0.0070121
 0.01659461 0.00144887 0.00632174 0.00324168 0.00363005 0.01204206
 0.00517986 0.00599465 0.01612032 0.02211373 0.00089859 0.00353109
 0.00874605 0.01518281 0.0045975  0.05138367 0.04633859 0.00663514
 0.03551564 0.04627639 0.03249586]
```

[103]:
```
feat_importances = pd.Series(model.feature_importances_, index=X.columns)
feat_importances.nlargest(20).plot(kind='barh')
plt.show()
```



[104]:
```
feat_importances.nlargest(20)
```

[104]:
```
word_freq_your              0.066549
word_freq_remove            0.058689
char_freq_!                 0.051384
word_freq_free              0.049350
char_freq_$                 0.046339
capital_run_length_longest  0.046276
word_freq_hp                0.041393
word_freq_000               0.040272
capital_run_length_average  0.035516
capital_run_length_total    0.032496
word_freq_you               0.031935
word_freq_our               0.031578
word_freq_business          0.026342
word_freq_money             0.025093
word_freq_receive           0.024542
word_freq_george            0.022686
word_freq_edu               0.022114
word_freq_hpl               0.020142
word_freq_all               0.017508
word_freq_1999              0.016595
dtype: float64
```

- **After applying the "VarianceThreshold method", we see that there is no reason to remove any feature from the analysis, since they present at least some minimal variation in their values, which may be important for the analysis results.**

- **I also applied the "Feature Importance" method to identify the features most related to the target "is_spam".**

5 EDA

- **The EDA will be based on the analysis of the most important features in relation to the target feature "is_spam".**
- **I chose to start the EDA with a "heatmap" of the most important variables to observe the correlation between them and a "countplot" of the "is_spam" variable to observe the distribution between spam and non-spam emails.**
- **In this EDA, "horizontal bar charts" were chosen to observe the frequency of use of each variable, and "boxplots" to observe the distribution of each variable, including outliers.**

[105]:
```
feat_importances.nlargest(20)
```

[105]:
```
word_freq_your              0.066549
word_freq_remove            0.058689
char_freq_!                 0.051384
word_freq_free              0.049350
char_freq_$                 0.046339
capital_run_length_longest  0.046276
word_freq_hp                0.041393
word_freq_000               0.040272
capital_run_length_average  0.035516
capital_run_length_total    0.032496
word_freq_you               0.031935
word_freq_our               0.031578
word_freq_business          0.026342
word_freq_money             0.025093
word_freq_receive           0.024542
word_freq_george            0.022686
word_freq_edu               0.022114
word_freq_hpl               0.020142
word_freq_all               0.017508
word_freq_1999              0.016595
dtype: float64
```

[106]:
```
heat=dff[['word_freq_remove','char_freq_$','char_freq_!
 ↪','word_freq_your','word_freq_000', 'word_freq_free',␣
 ↪'capital_run_length_longest', 'word_freq_hp','capital_run_length_average',␣
 ↪'word_freq_you', 'word_freq_our',␣
 ↪'capital_run_length_total','word_freq_receive', 'word_freq_business',␣
 ↪'word_freq_george', 'word_freq_edu', 'word_freq_money', 'word_freq_hpl',␣
 ↪'word_freq_all', 'word_freq_1999', 'is_spam']]
```

[107]:
```
heat
```

[107]:
```
      word_freq_remove  char_freq_$  char_freq_!  word_freq_your  \
0                 0.00        0.000        0.778            0.96
1                 0.21        0.180        0.372            1.59
2                 0.19        0.184        0.276            0.51
3                 0.31        0.000        0.137            0.31
4                 0.31        0.000        0.135            0.31
…                    …            …            …               …
4213              0.00        0.000        0.000            0.00
4214              0.00        0.000        0.353            2.00
4215              0.00        0.000        0.000            0.30
4216              0.00        0.000        0.000            0.32
4217              0.00        0.000        0.125            0.65

      word_freq_000  word_freq_free  capital_run_length_longest  word_freq_hp  \
0              0.00            0.32                          61           0.0
1              0.43            0.14                         101           0.0
2              1.16            0.06                         485           0.0
3              0.00            0.31                          40           0.0
4              0.00            0.31                          40           0.0
…                 …               …                           …             …
4213           0.00            0.00                           3           0.0
4214           0.00            0.00                           4           0.0
4215           0.00            0.00                           6           0.0
4216           0.00            0.00                           5           0.0
4217           0.00            0.00                           5           0.0

      capital_run_length_average  word_freq_you  …  \
0                          3.756           1.93  …
1                          5.114           3.47  …
2                          9.821           1.36  …
3                          3.537           3.18  …
4                          3.537           3.18  …
…                              …              …  …
4213                       1.142           0.62  …
4214                       1.555           6.00  …
4215                       1.404           1.50  …
4216                       1.147           1.93  …
4217                       1.250           4.60  …

      capital_run_length_total  word_freq_receive  word_freq_business  \
0                          278               0.00                0.00
1                         1028               0.21                0.07
2                         2259               0.38                0.06
3                          191               0.31                0.00
4                          191               0.31                0.00
```

```
…                          …                …                    …
4213                      88             0.00                 0.00
4214                      14             0.00                 0.00
4215                     118             0.00                 0.00
4216                      78             0.00                 0.00
4217                      40             0.00                 0.00

      word_freq_george  word_freq_edu  word_freq_money  word_freq_hpl  \
0                  0.0           0.00             0.00            0.0
1                  0.0           0.00             0.43            0.0
2                  0.0           0.06             0.06            0.0
3                  0.0           0.00             0.00            0.0
4                  0.0           0.00             0.00            0.0
…                  …             …                …               …
4213               0.0           0.31             0.00            0.0
4214               0.0           2.00             0.00            0.0
4215               0.0           1.20             0.00            0.0
4216               0.0           0.32             0.00            0.0
4217               0.0           0.65             0.00            0.0

      word_freq_all  word_freq_1999  is_spam
0              0.64            0.00        1
1              0.50            0.07        1
2              0.71            0.00        1
3              0.00            0.00        1
4              0.00            0.00        1
…              …               …           …
4213           0.62            0.00        0
4214           0.00            0.00        0
4215           0.30            0.00        0
4216           0.00            0.00        0
4217           0.65            0.00        0

[4218 rows x 21 columns]
```

[108]:
```
corrdf=heat.corr()
corrdf
```

[108]:
```
                          word_freq_remove  char_freq_$  char_freq_!  \
word_freq_remove                  1.000000     0.067377     0.051158
char_freq_$                       0.067377     1.000000     0.137163
char_freq_!                       0.051158     0.137163     1.000000
word_freq_your                    0.144024     0.149227     0.087284
word_freq_000                     0.068567     0.315574     0.065894
word_freq_free                    0.136012     0.044794     0.108630
capital_run_length_longest        0.050944     0.188814     0.072745
word_freq_hp                     -0.094805    -0.092242    -0.095505
```

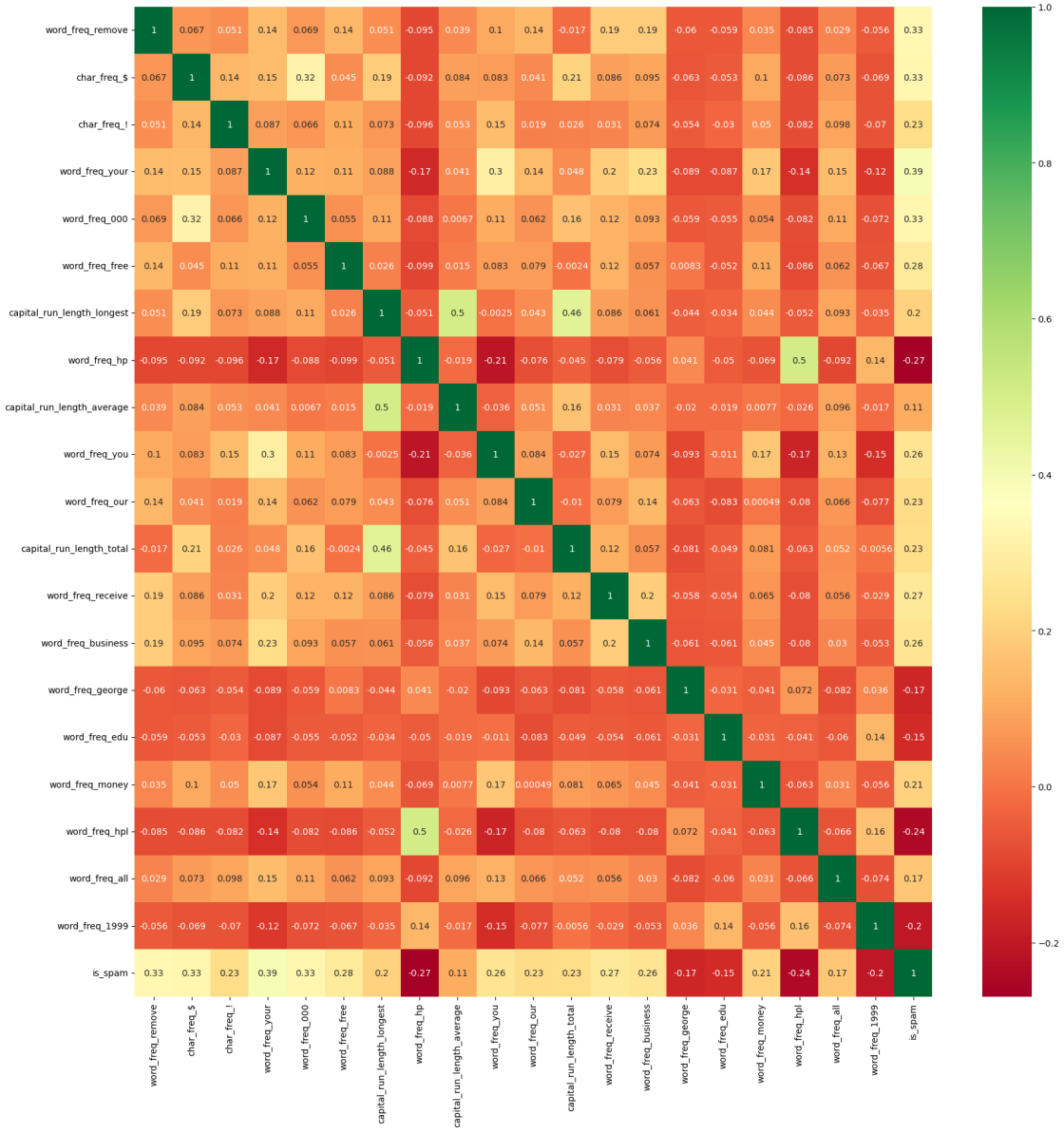
```
capital_run_length_average          0.039235     0.083628     0.052747
word_freq_you                       0.102964     0.082599     0.147573
word_freq_our                       0.136154     0.040887     0.019455
capital_run_length_total           -0.016819     0.206803     0.025713
word_freq_receive                   0.186921     0.086407     0.030675
word_freq_business                  0.189774     0.094732     0.073585
word_freq_george                   -0.060159    -0.062881    -0.054321
word_freq_edu                      -0.059022    -0.053063    -0.030487
word_freq_money                     0.034642     0.103406     0.050323
word_freq_hpl                      -0.084507    -0.085693    -0.081682
word_freq_all                       0.028843     0.072985     0.097543
word_freq_1999                     -0.055736    -0.068673    -0.069705
is_spam                             0.334611     0.327265     0.234258

                           word_freq_your  word_freq_000  word_freq_free  \
word_freq_remove                 0.144024       0.068567        0.136012
char_freq_$                      0.149227       0.315574        0.044794
char_freq_!                      0.087284       0.065894        0.108630
word_freq_your                   1.000000       0.119551        0.114641
word_freq_000                    0.119551       1.000000        0.054644
word_freq_free                   0.114641       0.054644        1.000000
capital_run_length_longest       0.087729       0.107316        0.025650
word_freq_hp                    -0.165004      -0.087719       -0.098860
capital_run_length_average       0.041163       0.006709        0.014897
word_freq_you                    0.298074       0.111517        0.082505
word_freq_our                    0.143454       0.062346        0.079411
capital_run_length_total         0.047639       0.157384       -0.002421
word_freq_receive                0.195218       0.123183        0.123185
word_freq_business               0.233699       0.093468        0.057238
word_freq_george                -0.089485      -0.059065        0.008319
word_freq_edu                   -0.086606      -0.054533       -0.051560
word_freq_money                  0.172188       0.054104        0.112283
word_freq_hpl                   -0.143231      -0.082142       -0.085988
word_freq_all                    0.148492       0.106943        0.062135
word_freq_1999                  -0.122329      -0.071963       -0.066698
is_spam                          0.394666       0.325757        0.279739

                           capital_run_length_longest  word_freq_hp  \
word_freq_remove                             0.050944     -0.094805
char_freq_$                                  0.188814     -0.092242
char_freq_!                                  0.072745     -0.095505
word_freq_your                               0.087729     -0.165004
word_freq_000                                0.107316     -0.087719
word_freq_free                               0.025650     -0.098860
capital_run_length_longest                   1.000000     -0.051364
word_freq_hp                                -0.051364      1.000000
capital_run_length_average                   0.499469     -0.018966
```

```
word_freq_you                          -0.002487     -0.214265
word_freq_our                           0.043176     -0.076493
capital_run_length_total                0.464500     -0.045045
word_freq_receive                       0.086325     -0.079100
word_freq_business                      0.061132     -0.055804
word_freq_george                       -0.043846      0.041204
word_freq_edu                          -0.034274     -0.049873
word_freq_money                         0.044261     -0.068997
word_freq_hpl                          -0.051790      0.504716
word_freq_all                           0.092723     -0.092342
word_freq_1999                         -0.035073      0.135049
is_spam                                 0.203842     -0.269272

                         capital_run_length_average  word_freq_you  …  \
word_freq_remove                           0.039235       0.102964  …
char_freq_$                                0.083628       0.082599  …
char_freq_!                                0.052747       0.147573  …
word_freq_your                             0.041163       0.298074  …
word_freq_000                              0.006709       0.111517  …
word_freq_free                             0.014897       0.082505  …
capital_run_length_longest                 0.499469      -0.002487  …
word_freq_hp                              -0.018966      -0.214265  …
capital_run_length_average                 1.000000      -0.036363  …
word_freq_you                             -0.036363       1.000000  …
word_freq_our                              0.050925       0.083773  …
capital_run_length_total                   0.162661      -0.026687  …
word_freq_receive                          0.031400       0.146673  …
word_freq_business                         0.036822       0.074441  …
word_freq_george                          -0.020330      -0.093418  …
word_freq_edu                             -0.018945      -0.010873  …
word_freq_money                            0.007678       0.174105  …
word_freq_hpl                             -0.025832      -0.174687  …
word_freq_all                              0.095768       0.127309  …
word_freq_1999                            -0.017065      -0.146522  …
is_spam                                    0.110371       0.257006  …

                         capital_run_length_total  word_freq_receive  \
word_freq_remove                        -0.016819           0.186921
char_freq_$                              0.206803           0.086407
char_freq_!                              0.025713           0.030675
word_freq_your                           0.047639           0.195218
word_freq_000                            0.157384           0.123183
word_freq_free                          -0.002421           0.123185
capital_run_length_longest               0.464500           0.086325
word_freq_hp                            -0.045045          -0.079100
capital_run_length_average               0.162661           0.031400
word_freq_you                           -0.026687           0.146673
```

```
word_freq_our                          -0.010135           0.079380
capital_run_length_total                1.000000           0.122395
word_freq_receive                       0.122395           1.000000
word_freq_business                      0.057227           0.195712
word_freq_george                       -0.081076          -0.058044
word_freq_edu                          -0.049005          -0.054098
word_freq_money                         0.081382           0.065071
word_freq_hpl                          -0.062604          -0.080347
word_freq_all                           0.052273           0.055881
word_freq_1999                         -0.005606          -0.029147
is_spam                                 0.232431           0.272786

                          word_freq_business  word_freq_george  \
word_freq_remove                    0.189774         -0.060159
char_freq_$                         0.094732         -0.062881
char_freq_!                         0.073585         -0.054321
word_freq_your                      0.233699         -0.089485
word_freq_000                       0.093468         -0.059065
word_freq_free                      0.057238          0.008319
capital_run_length_longest          0.061132         -0.043846
word_freq_hp                       -0.055804          0.041204
capital_run_length_average          0.036822         -0.020330
word_freq_you                       0.074441         -0.093418
word_freq_our                       0.138086         -0.063393
capital_run_length_total            0.057227         -0.081076
word_freq_receive                   0.195712         -0.058044
word_freq_business                  1.000000         -0.061379
word_freq_george                   -0.061379          1.000000
word_freq_edu                      -0.060694         -0.030924
word_freq_money                     0.045190         -0.041468
word_freq_hpl                      -0.079796          0.072187
word_freq_all                       0.030150         -0.082093
word_freq_1999                     -0.053170          0.035633
is_spam                             0.260902         -0.167943

                          word_freq_edu  word_freq_money  word_freq_hpl  \
word_freq_remove              -0.059022         0.034642      -0.084507
char_freq_$                   -0.053063         0.103406      -0.085693
char_freq_!                   -0.030487         0.050323      -0.081682
word_freq_your                -0.086606         0.172188      -0.143231
word_freq_000                 -0.054533         0.054104      -0.082142
word_freq_free                -0.051560         0.112283      -0.085988
capital_run_length_longest    -0.034274         0.044261      -0.051790
word_freq_hp                  -0.049873        -0.068997       0.504716
capital_run_length_average    -0.018945         0.007678      -0.025832
word_freq_you                 -0.010873         0.174105      -0.174687
word_freq_our                 -0.083135         0.000487      -0.079541
```

```
capital_run_length_total        -0.049005         0.081382      -0.062604
word_freq_receive               -0.054098         0.065071      -0.080347
word_freq_business              -0.060694         0.045190      -0.079796
word_freq_george                -0.030924        -0.041468       0.072187
word_freq_edu                    1.000000        -0.031284      -0.040651
word_freq_money                 -0.031284         1.000000      -0.062812
word_freq_hpl                   -0.040651        -0.062812       1.000000
word_freq_all                   -0.060335         0.031351      -0.066130
word_freq_1999                   0.135220        -0.055622       0.161250
is_spam                         -0.152471         0.205111      -0.241885

                           word_freq_all  word_freq_1999   is_spam
word_freq_remove                0.028843       -0.055736  0.334611
char_freq_$                     0.072985       -0.068673  0.327265
char_freq_!                     0.097543       -0.069705  0.234258
word_freq_your                  0.148492       -0.122329  0.394666
word_freq_000                   0.106943       -0.071963  0.325757
word_freq_free                  0.062135       -0.066698  0.279739
capital_run_length_longest      0.092723       -0.035073  0.203842
word_freq_hp                   -0.092342        0.135049 -0.269272
capital_run_length_average      0.095768       -0.017065  0.110371
word_freq_you                   0.127309       -0.146522  0.257006
word_freq_our                   0.066091       -0.077396  0.230496
capital_run_length_total        0.052273       -0.005606  0.232431
word_freq_receive               0.055881       -0.029147  0.272786
word_freq_business              0.030150       -0.053170  0.260902
word_freq_george               -0.082093        0.035633 -0.167943
word_freq_edu                  -0.060335        0.135220 -0.152471
word_freq_money                 0.031351       -0.055622  0.205111
word_freq_hpl                  -0.066130        0.161250 -0.241885
word_freq_all                   1.000000       -0.074008  0.172860
word_freq_1999                 -0.074008        1.000000 -0.200856
is_spam                         0.172860       -0.200856  1.000000

[21 rows x 21 columns]
```

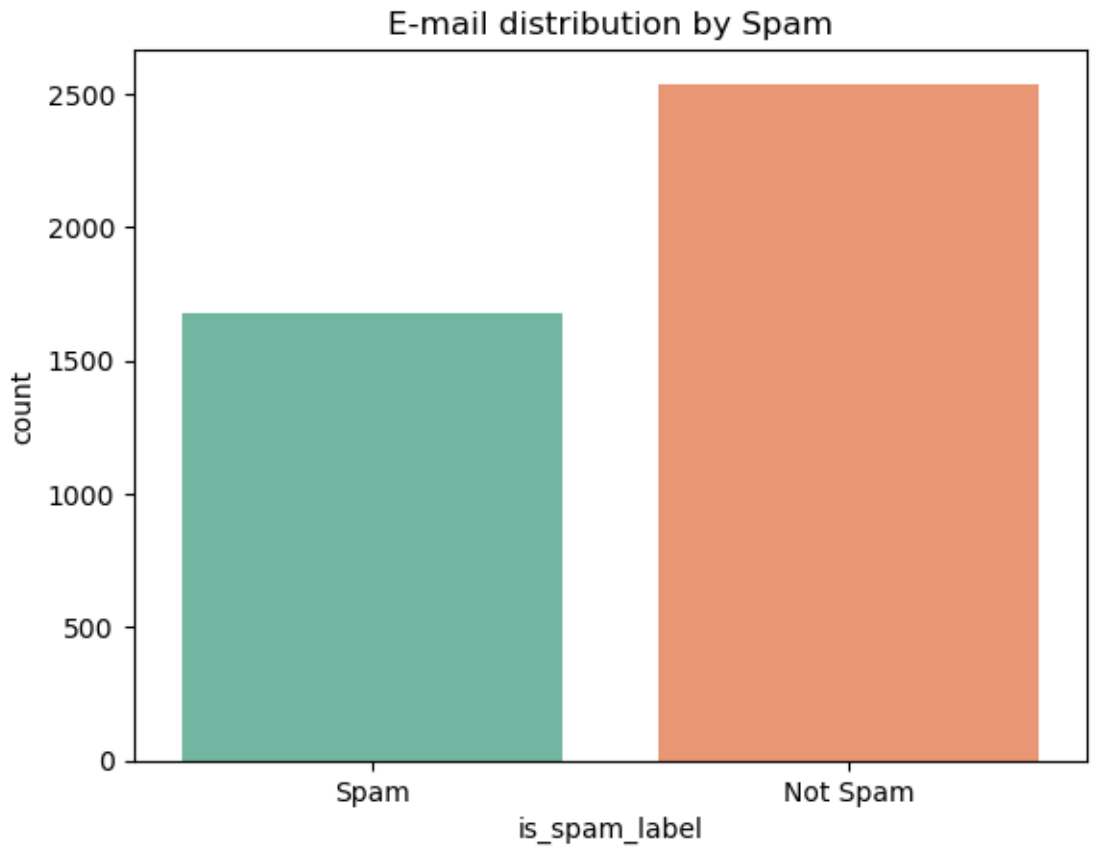
[109]:
```
corrmat = heat.corr()
top_corr_features = corrmat.index
plt.figure(figsize=(20,20))
g=sns.heatmap(dff[top_corr_features].corr(),annot=True,cmap="RdYlGn")
```



- **Objective:** Explore relationships between the most important numeric variables.
- **Why this plot:** Heatmaps summarize correlations effectively.
- **Learning:** Words that are more common tend to be more frequent in emails classified as spam.

[110]:
```
dff['is_spam_label']=dff['is_spam'].map({0: "Not Spam", 1: "Spam"})
sns.countplot(x="is_spam_label", data=dff, palette="Set2")
plt.title("E-mail distribution by Spam")
plt.show()
```

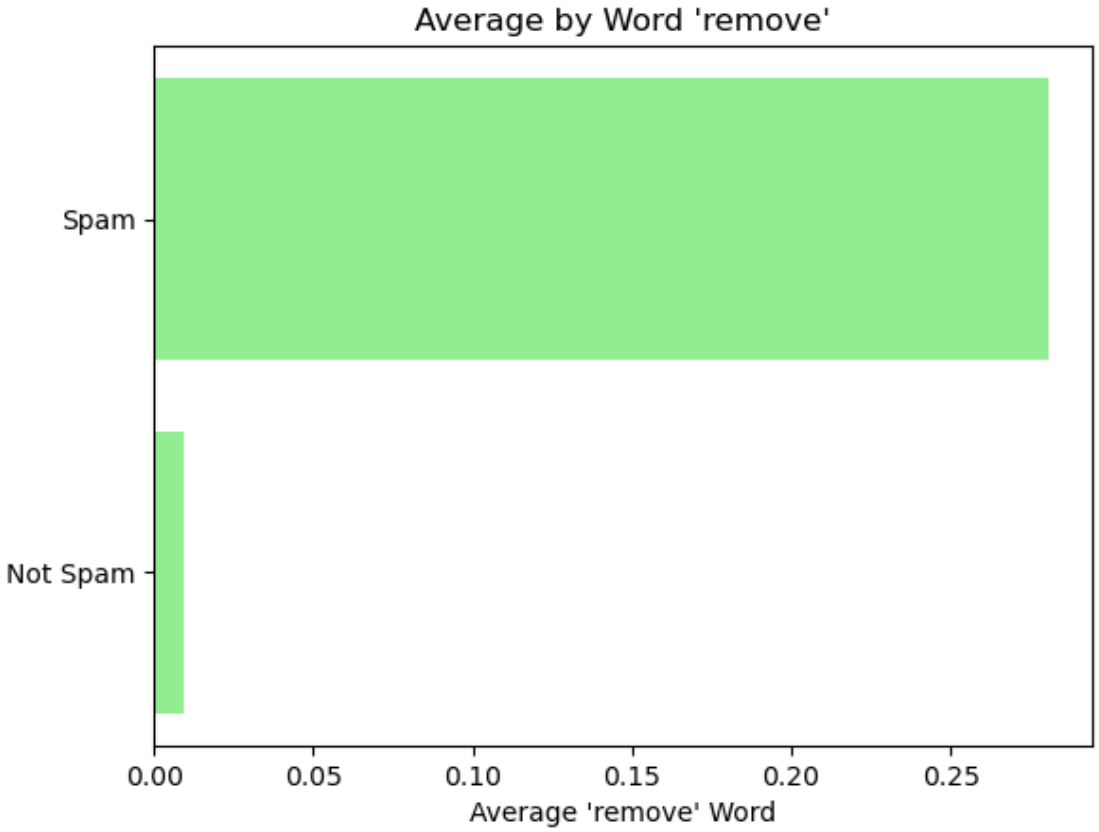
```
/var/folders/n9/69dc0cws2sz4qzt9yk7vc1j40000gn/T/ipykernel_4927/2905436564.py:2:
FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in
v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same
effect.

  sns.countplot(x="is_spam_label", data=dff, palette="Set2")
```



- **Objective:** To compare how many emails are spam or not.
- **Why this plot:** Countplots are ideal for categorical comparisons.
- **Learning:** There are more not spam emaisl than spam.

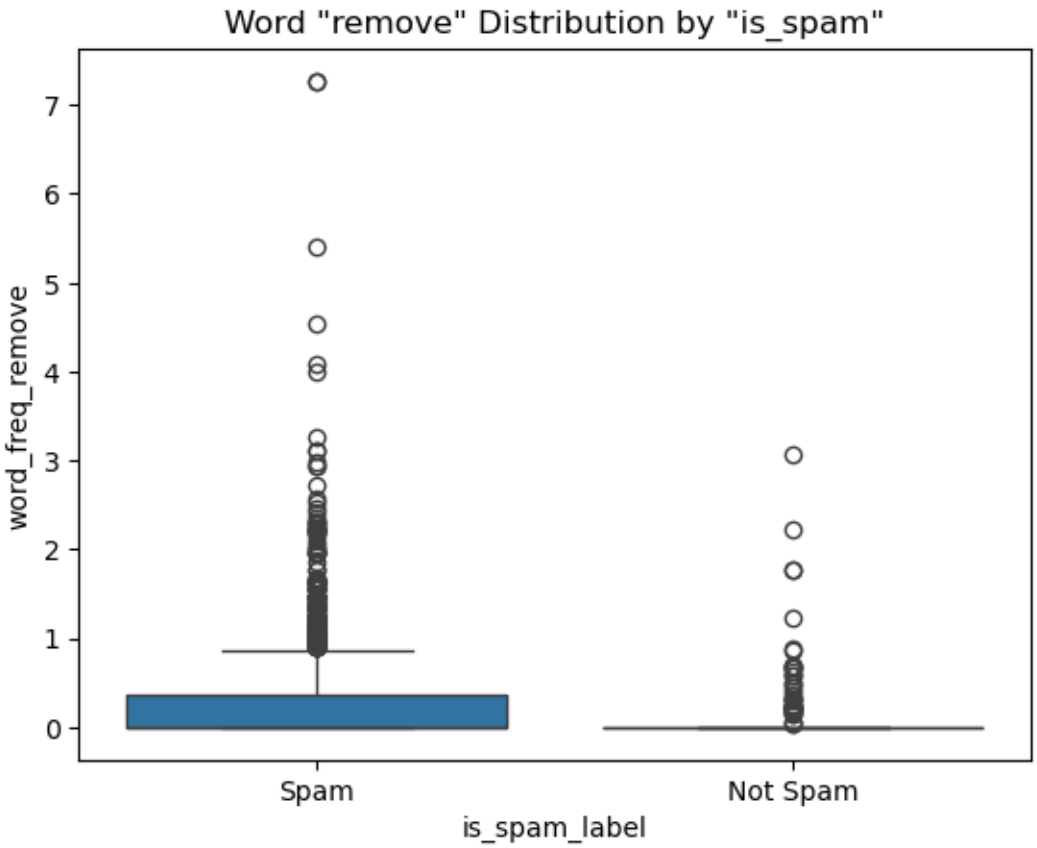
[111]:
```
#1
avg_remove = dff.groupby("is_spam")["word_freq_remove"].mean().sort_values()
labels = avg_remove.index.map({0: "Not Spam", 1: "Spam"})
plt.barh(labels, avg_remove.values, color="lightgreen")
plt.title("Average by Word 'remove'")
```

```
plt.xlabel("Average 'remove' Word")
plt.show()
```



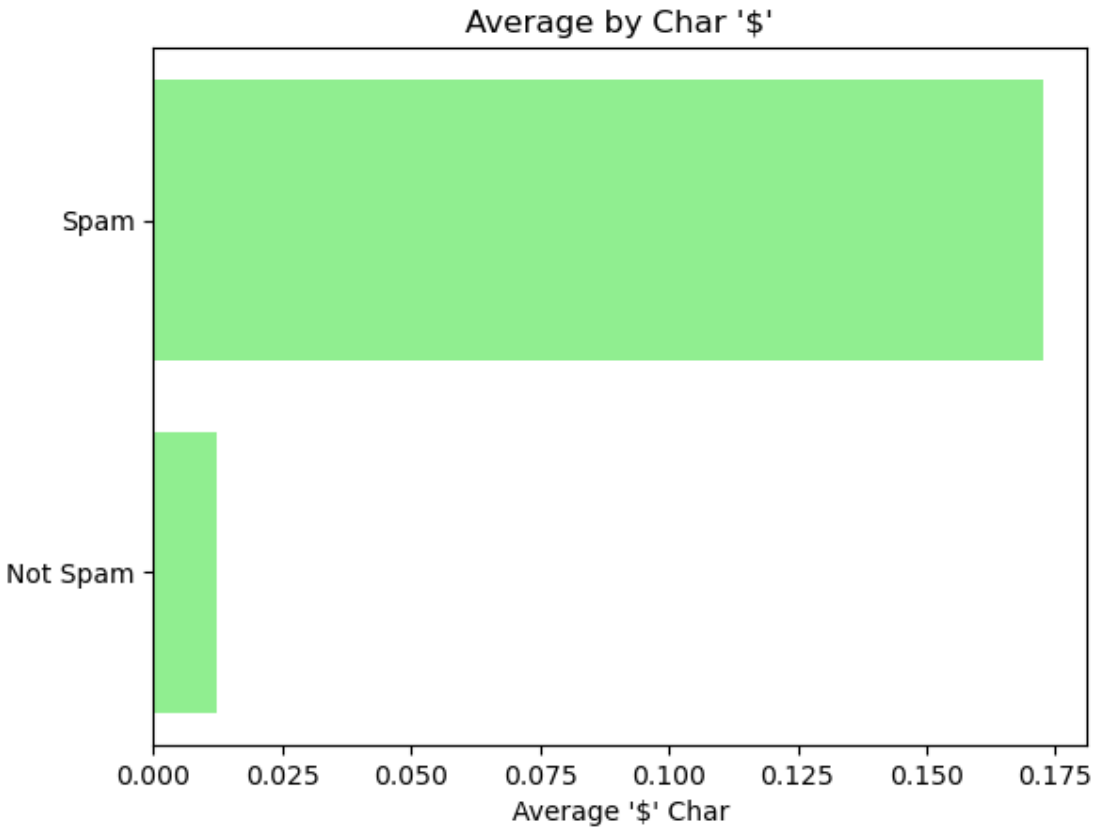
- **Objective:** Compare average word 'remove' usage in the spam or not spam emails.
- **Why this plot:** Horizontal bars are easier for categorical labels.
- **Learning:** The word 'remove' it's frequently used in spam emails.

[112]:
```
dff['is_spam_label']=dff['is_spam'].map({0: "Not Spam", 1: "Spam"})
sns.boxplot(y="word_freq_remove", x="is_spam_label", data=dff)
plt.title('Word "remove" Distribution by "is_spam"')
plt.show()
```



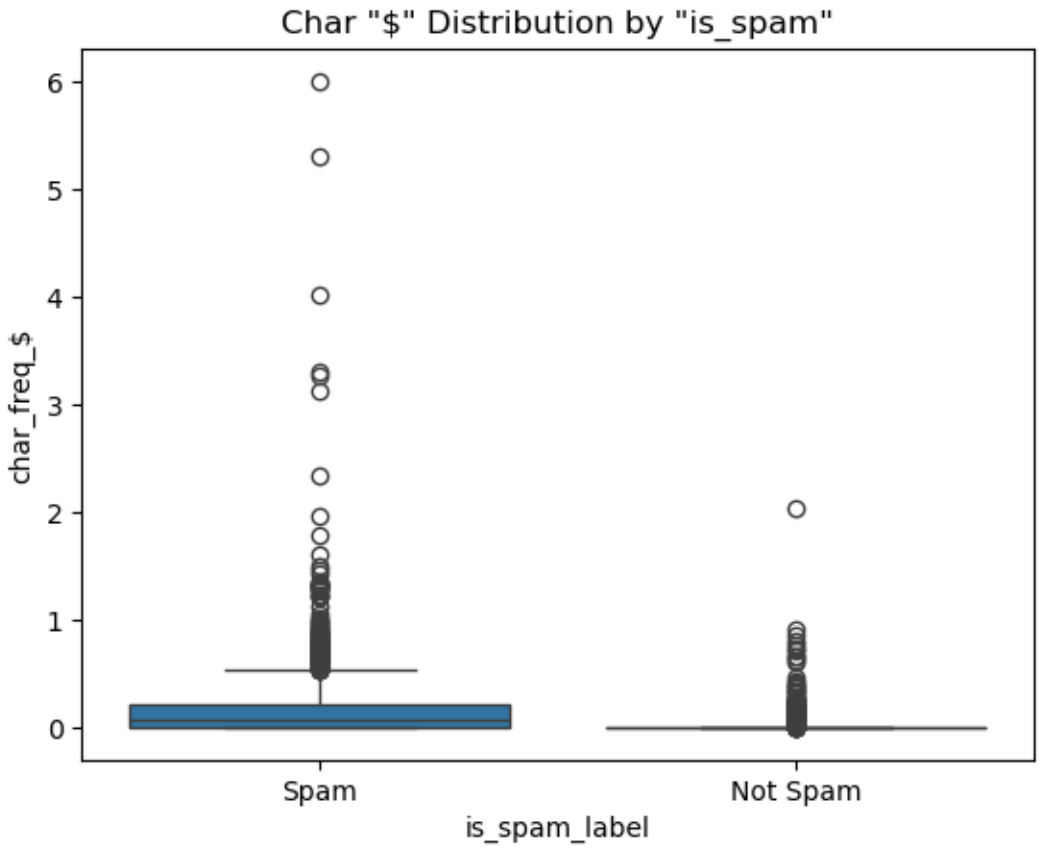
- **Objective:** Examine the word 'remove' distribution across different spam or not epam emails.
- **Why this plot:** Boxplots show spread, median, and outliers effectively.
- **Learning:** The word 'remove' is normally found in non-spam emails too, but it is more frequently used in spam emails, in some cases excessively as shown by the outliers.

[113]:
```
#2
avg_remove = dff.groupby("is_spam")["char_freq_$"].mean().sort_values()
labels = avg_remove.index.map({0: "Not Spam", 1: "Spam"})
plt.barh(labels, avg_remove.values, color="lightgreen")
plt.title("Average by Char '$'")
plt.xlabel("Average '$' Char")
plt.show()
```



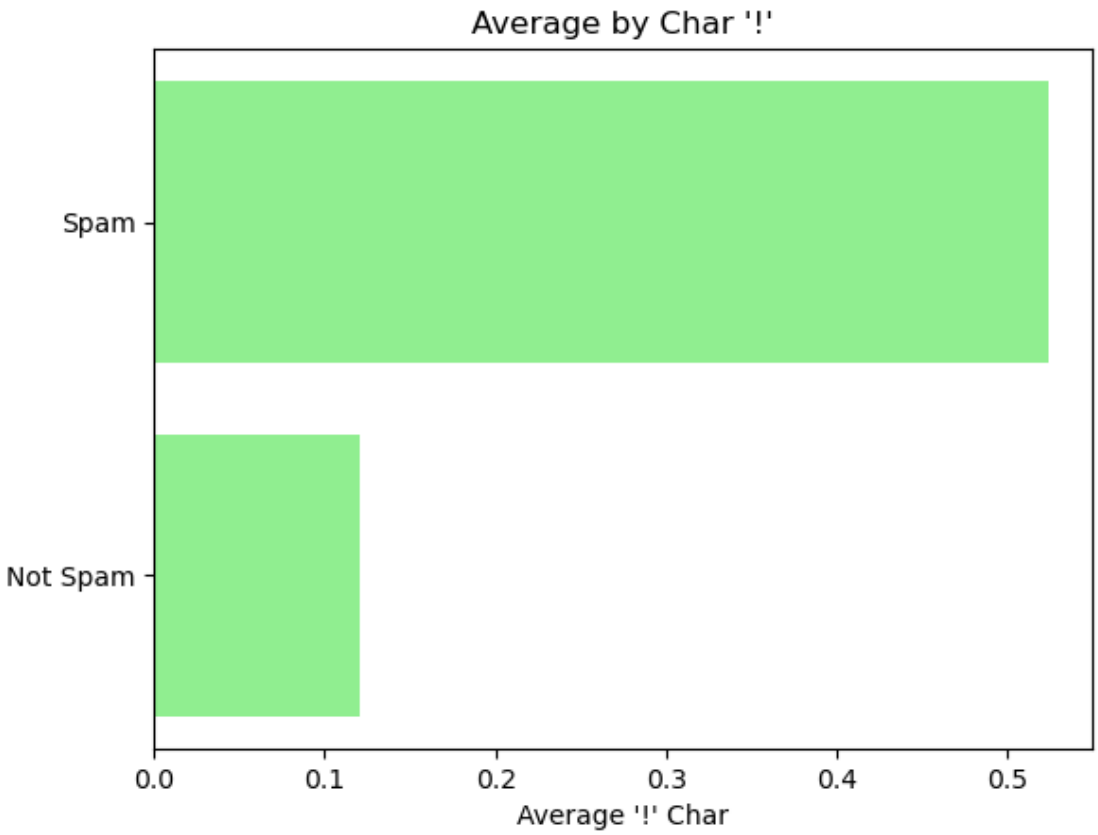
- **Objective:** Compare average char '$' usage in the spam or not spam emails.
- **Why this plot:** Horizontal bars are easier for categorical labels.
- **Learning:** The char it's frequently used in spam emails.

[114]:
```
dff['is_spam_label']=dff['is_spam'].map({0: "Not Spam", 1: "Spam"})
sns.boxplot(y="char_freq_$", x="is_spam_label", data=dff)
plt.title('Char "$" Distribution by "is_spam"')
plt.show()
```



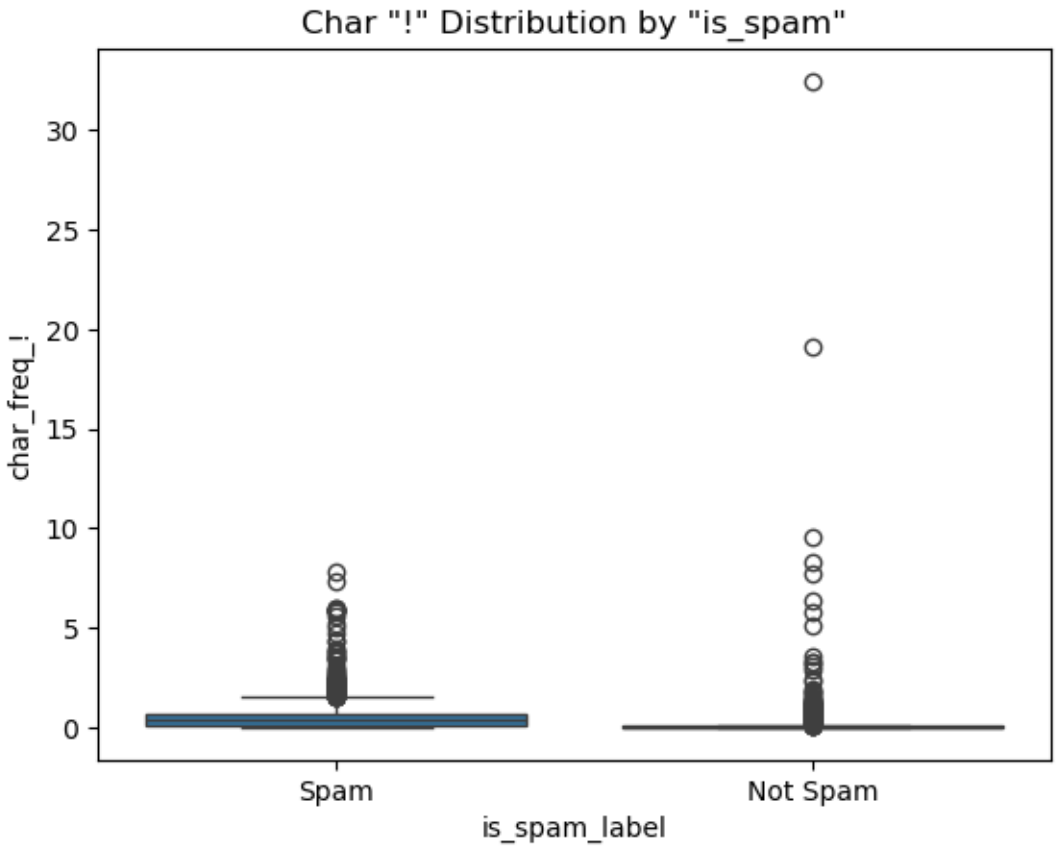
- **Objective:** Examine the char '$' distribution across different spam or not epam emails.
- **Why this plot:** Boxplots show spread, median, and outliers effectively.
- **Learning:** The char can be found in non-spam emails too, but it is more frequently used in spam emails, in some cases excessively as shown by the outliers.

[115]:
```
#3
avg_remove = dff.groupby("is_spam")["char_freq_!"].mean().sort_values()
labels = avg_remove.index.map({0: "Not Spam", 1: "Spam"})
plt.barh(labels, avg_remove.values, color="lightgreen")
plt.title("Average by Char '!'")
plt.xlabel("Average '!' Char")
plt.show()
```



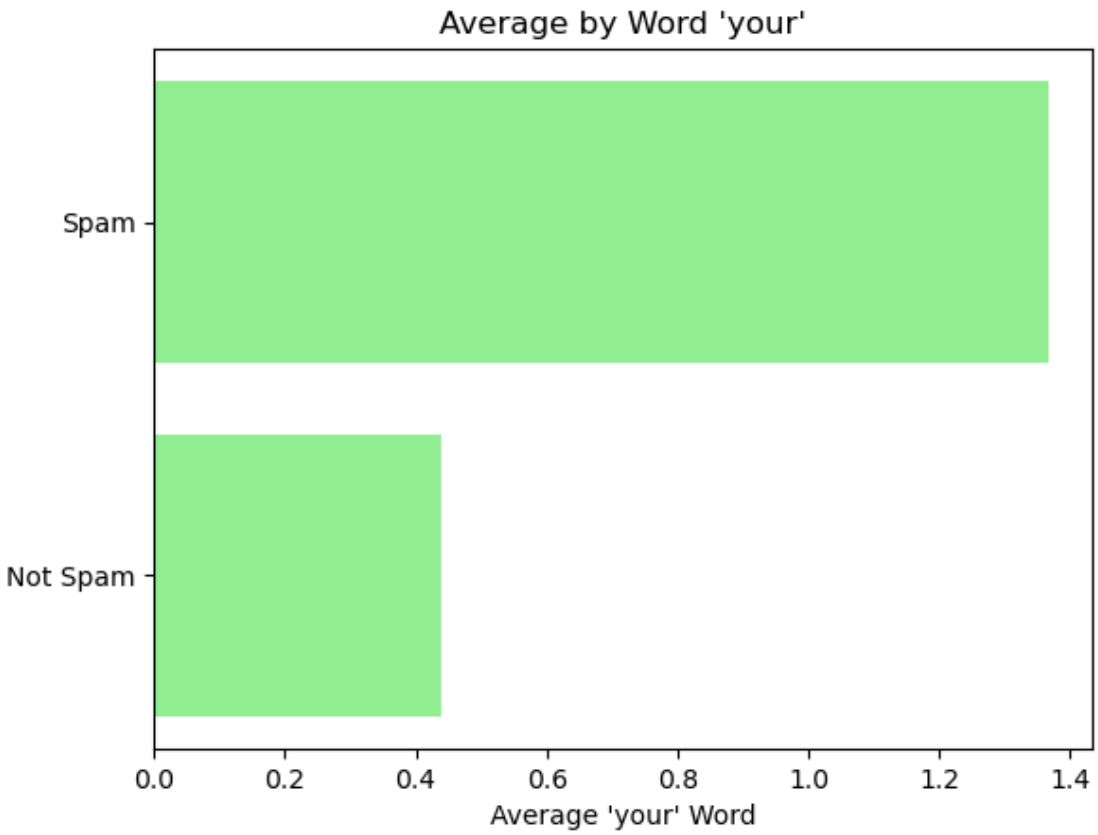
- **Objective:** Compare average char '!' usage in the spam or not spam emails.
- **Why this plot:** Horizontal bars are easier for categorical labels.
- **Learning:** The char '!' it's frequently used in spam emails.

[116]:
```
dff['is_spam_label']=dff['is_spam'].map({0: "Not Spam", 1: "Spam"})
sns.boxplot(y="char_freq_!", x="is_spam_label", data=dff)
plt.title('Char "!" Distribution by "is_spam"')
plt.show()
```



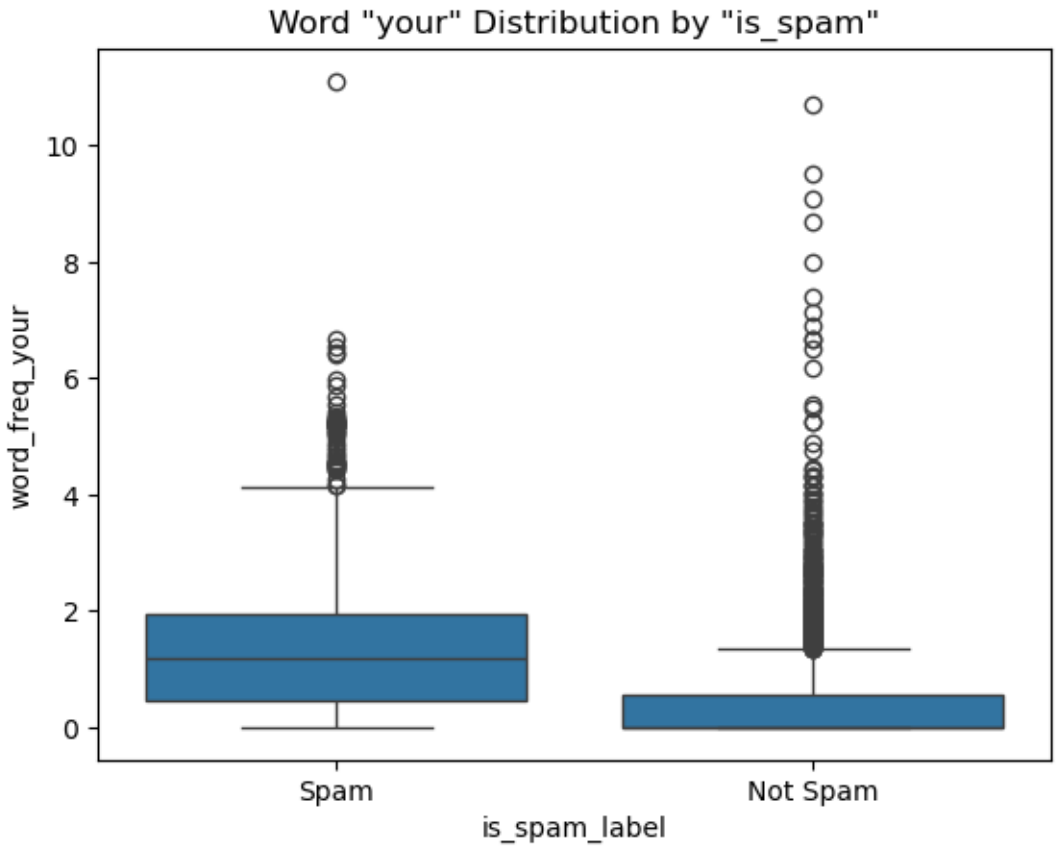
- **Objective:** Examine the mark '!' distribution across different spam or not epam emails.
- **Why this plot:** Boxplots show spread, median, and outliers effectively.
- **Learning:** The exclamation mark '!' is more frequently used in spam emails, but it is normally found in non-spam emails too, exaggerated in some cases as shown by the outliers, indicating caution when evaluating if the email is spam or not solely by the presence of the mark in the email, showing it is necessary to cross this information with other variables to confirm if the email is spam or not.

[117]:
```
#4
avg_remove = dff.groupby("is_spam")["word_freq_your"].mean().sort_values()
labels = avg_remove.index.map({0: "Not Spam", 1: "Spam"})
plt.barh(labels, avg_remove.values, color="lightgreen")
plt.title("Average by Word 'your'")
plt.xlabel("Average 'your' Word")
plt.show()
```



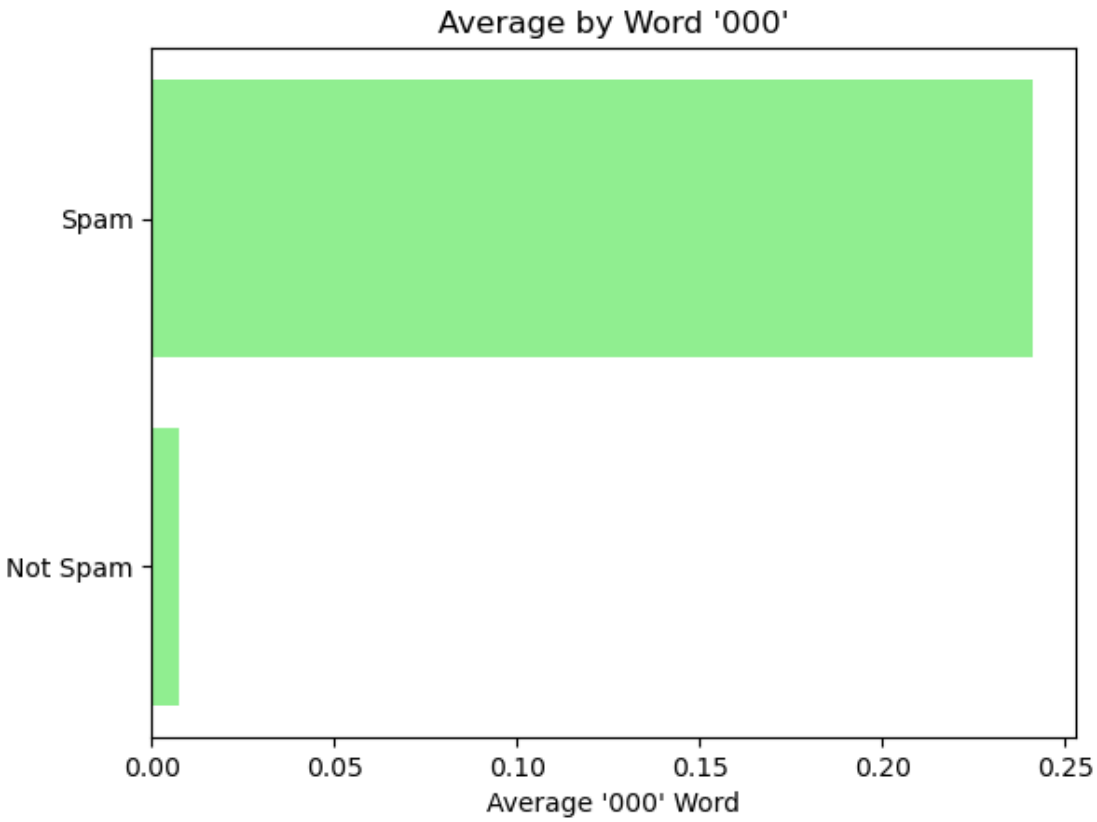
- **Objective:** Compare average word ‘your’ usage in the spam or not spam emails.
- **Why this plot:** Horizontal bars are easier for categorical labels.
- **Learning:** The word ‘your’ it’s frequently used in spam emails.

[118]:
```
dff['is_spam_label']=dff['is_spam'].map({0: "Not Spam", 1: "Spam"})
sns.boxplot(y="word_freq_your", x="is_spam_label", data=dff)
plt.title('Word "your" Distribution by "is_spam"')
plt.show()
```



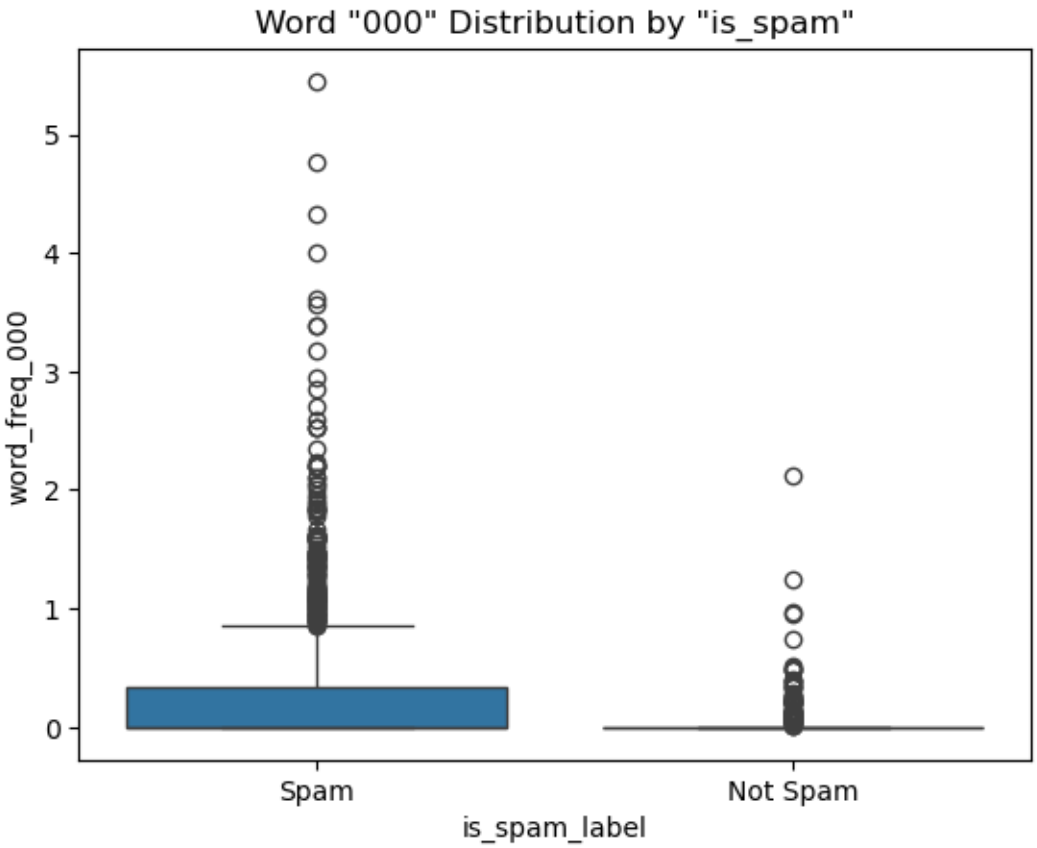
- **Objective:** Examine the word 'your' distribution across different spam or not epam emails.
- **Why this plot:** Boxplots show spread, median, and outliers effectively.
- **Learning:** The word 'your' is commonly found in both cases, the graph shows that the word is more common in spam emails, but this word is common in the day-to-day life of all people, which justifies its strong presence in both scenarios.

[119]:
```
#5
avg_remove = dff.groupby("is_spam")["word_freq_000"].mean().sort_values()
labels = avg_remove.index.map({0: "Not Spam", 1: "Spam"})
plt.barh(labels, avg_remove.values, color="lightgreen")
plt.title("Average by Word '000'")
plt.xlabel("Average '000' Word")
plt.show()
```



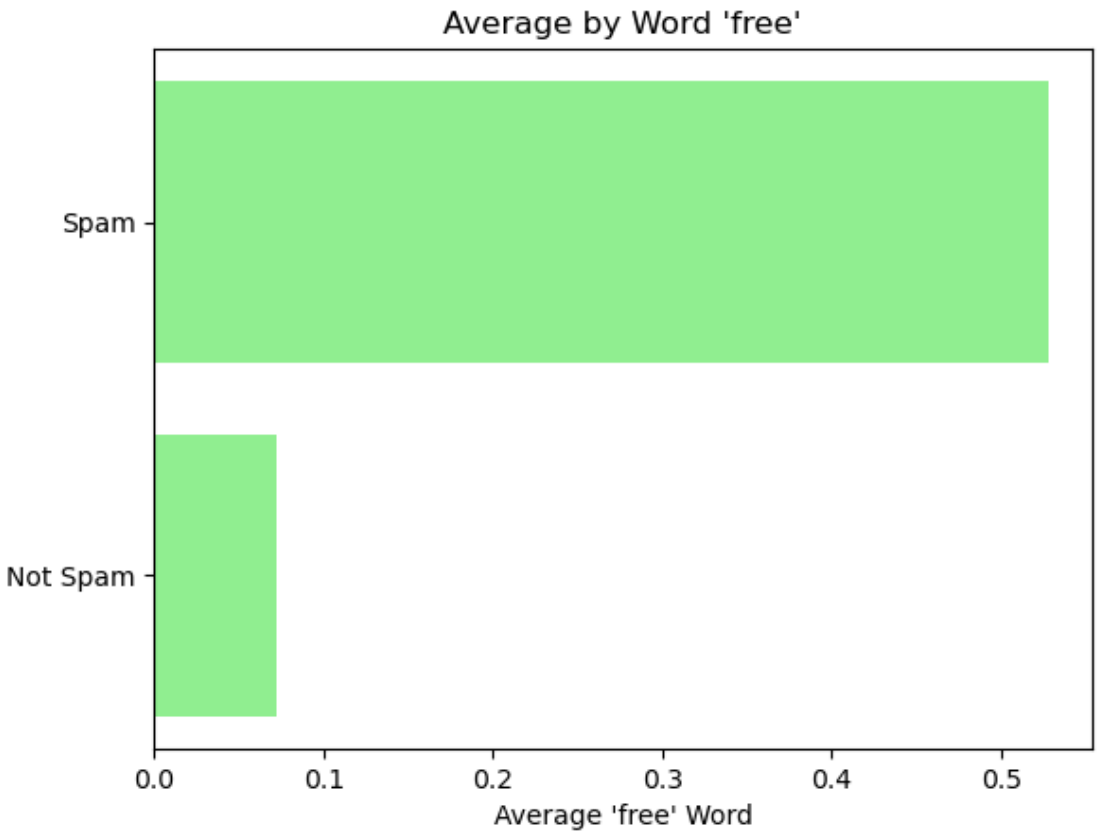
- **Objective:** Compare average word '000' usage in the spam or not spam emails.
- **Why this plot:** Horizontal bars are easier for categorical labels.
- **Learning:** The word '000' it's frequently used in spam emails.

[120]:
```
dff['is_spam_label']=dff['is_spam'].map({0: "Not Spam", 1: "Spam"})
sns.boxplot(y="word_freq_000", x="is_spam_label", data=dff)
plt.title('Word "000" Distribution by "is_spam"')
plt.show()
```



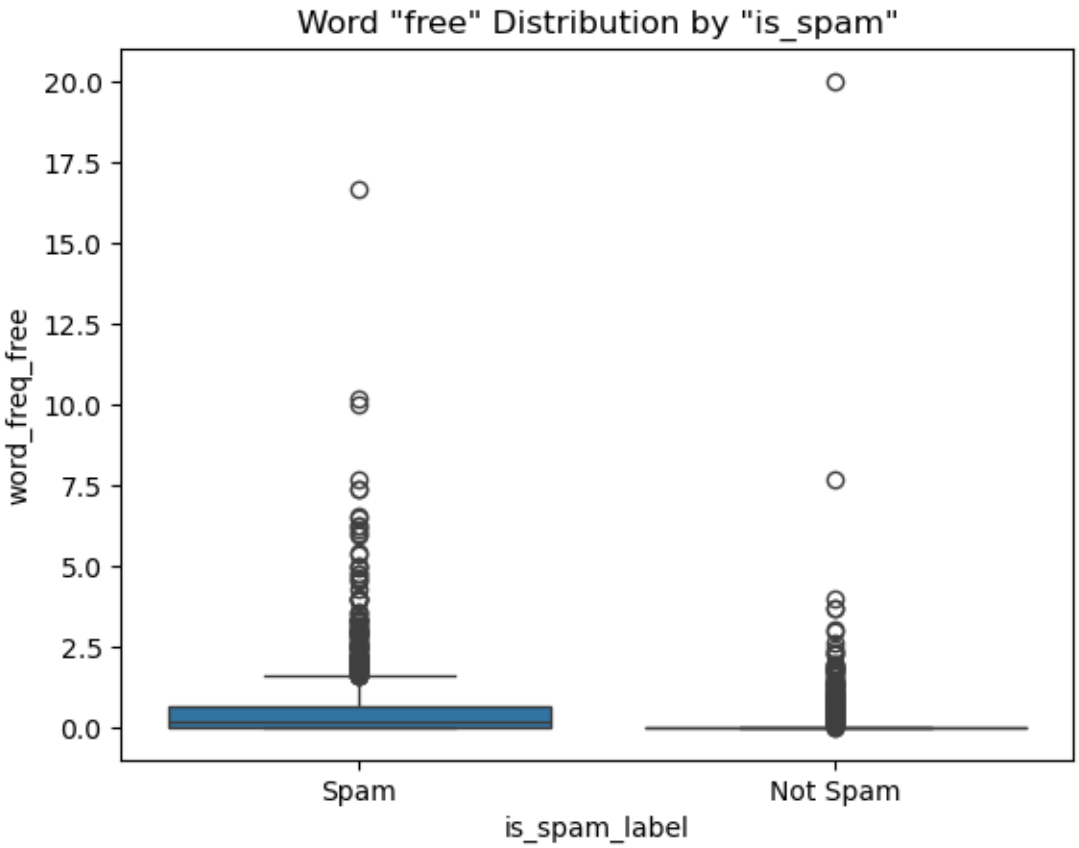
- **Objective:** Examine the word '000' distribution across different spam or not epam emails.
- **Why this plot:** Boxplots show spread, median, and outliers effectively.
- **Learning:** The sequence number '000' is normally found in non-spam emails too, but it is more frequently used in spam emails, in some cases excessively as shown by the outliers, indicating the most usage of these numbers in spam emails.

[121]:
```
#6
avg_remove = dff.groupby("is_spam")["word_freq_free"].mean().sort_values()
labels = avg_remove.index.map({0: "Not Spam", 1: "Spam"})
plt.barh(labels, avg_remove.values, color="lightgreen")
plt.title("Average by Word 'free'")
plt.xlabel("Average 'free' Word")
plt.show()
```



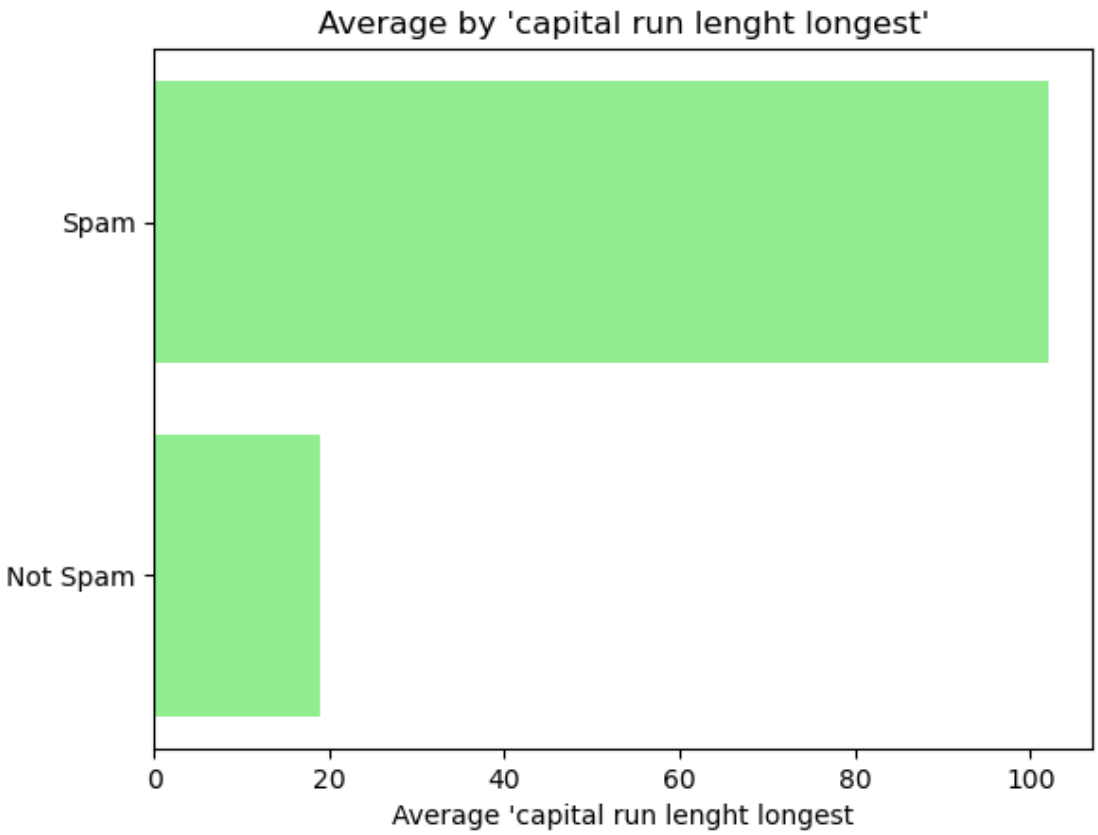
- **Objective:** Compare average word ‘free’ usage in the spam or not spam emails.
- **Why this plot:** Horizontal bars are easier for categorical labels.
- **Learning:** The word ‘free’ it’s frequently used in spam emails.

[122]:
```
dff['is_spam_label']=dff['is_spam'].map({0: "Not Spam", 1: "Spam"})
sns.boxplot(y="word_freq_free", x="is_spam_label", data=dff)
plt.title('Word "free" Distribution by "is_spam"')
plt.show()
```



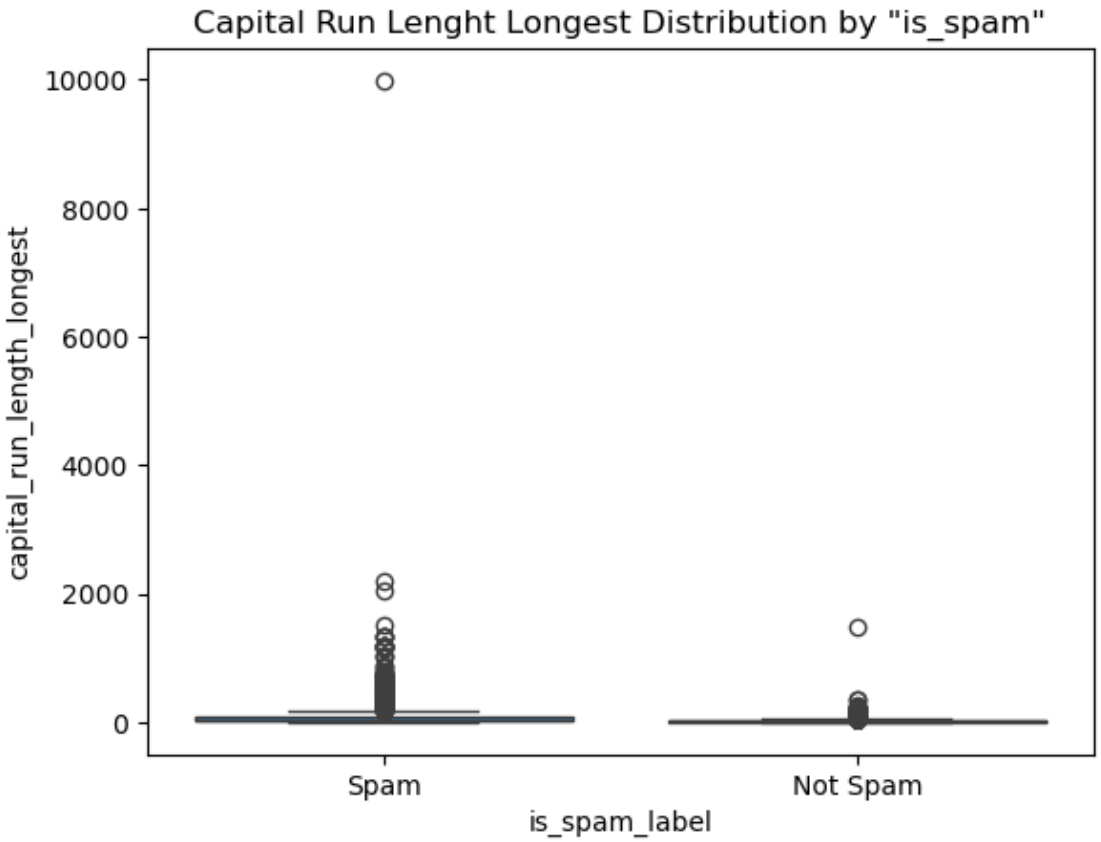
- **Objective:** Examine the word 'free' distribution across different spam or not epam emails.
- **Why this plot:** Boxplots show spread, median, and outliers effectively.
- **Learning:** This is another example of a word found frequently in both cases, the graphic shows it is used more often in spam emails, yet it is common in everyday language, which explains its presence in both scenarios.

[123]:
```
#7
avg_remove = dff.groupby("is_spam")["capital_run_length_longest"].mean().
 ↪sort_values()
labels = avg_remove.index.map({0: "Not Spam", 1: "Spam"})
plt.barh(labels, avg_remove.values, color="lightgreen")
plt.title("Average by 'capital run lenght longest'")
plt.xlabel("Average 'capital run lenght longest")
plt.show()
```



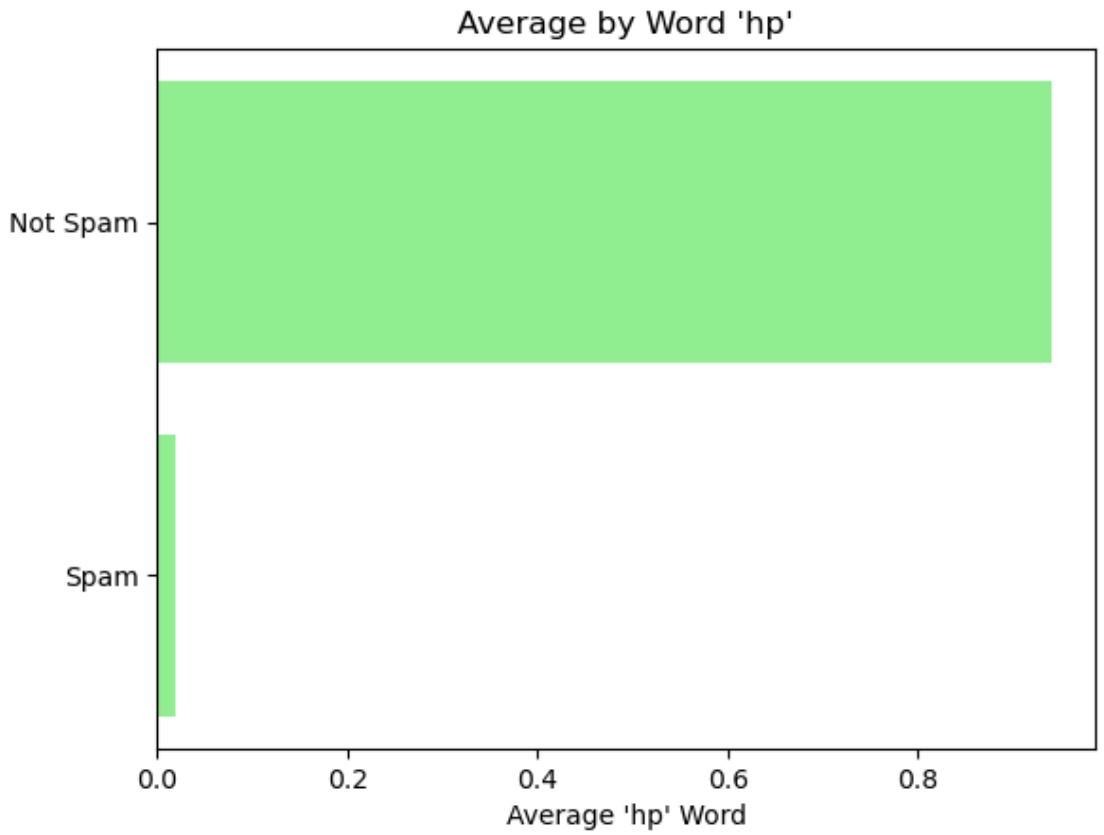
- **Objective:** Compare the longest capital run length usage in the spam or not spam emails.
- **Why this plot:** Horizontal bars are easier for categorical labels.
- **Learning:** The longest capital run length it's frequently founded in spam emails.

[124]:
```
dff['is_spam_label']=dff['is_spam'].map({0: "Not Spam", 1: "Spam"})
sns.boxplot(y="capital_run_length_longest", x="is_spam_label", data=dff)
plt.title('Capital Run Lenght Longest Distribution by "is_spam"')
plt.show()
```



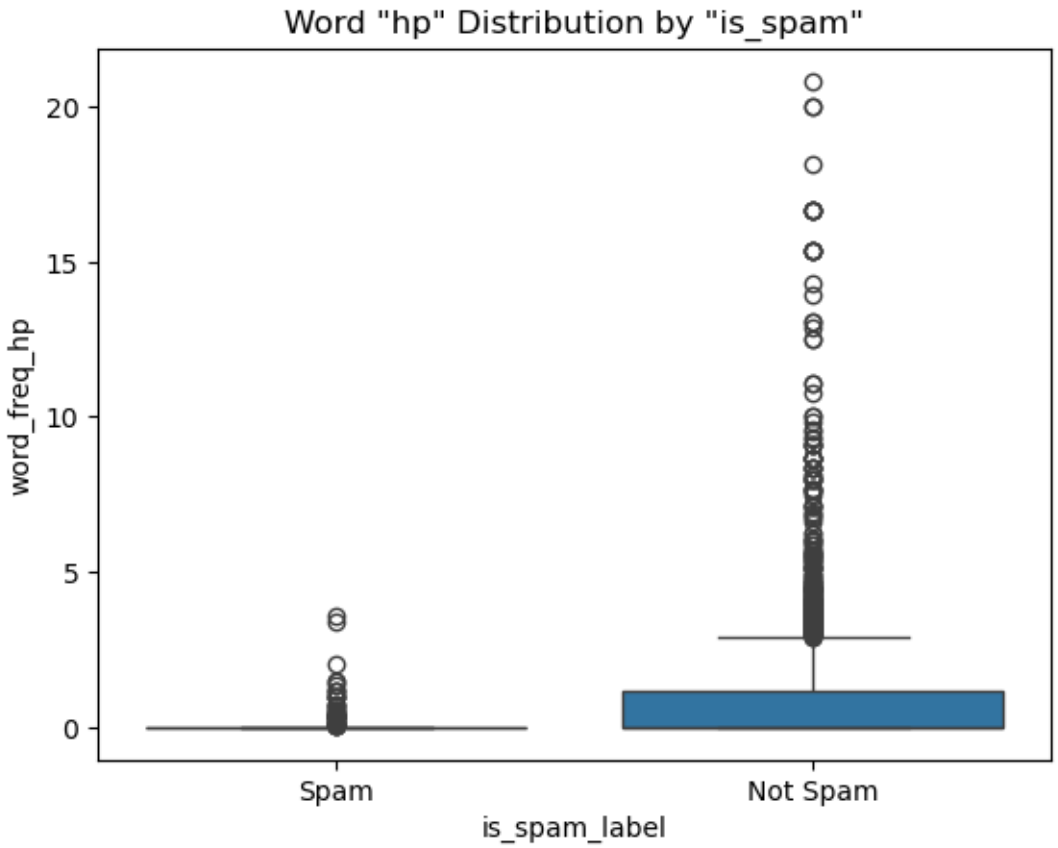
- **Objective:** Examine the longest capital run length distribution across different spam or not epam emails.
- **Why this plot:** Boxplots show spread, median, and outliers effectively.
- **Learning:** The use of words with uppercase letters in emails is not common in any scenario, as shown by the graph, this type of writing is normally found in spam, indicating that the presence of many words with uppercase letters is a spam indicator, especially consecutively.

[125]:
```
#8
avg_remove = dff.groupby("is_spam")["word_freq_hp"].mean().sort_values()
labels = avg_remove.index.map({0: "Not Spam", 1: "Spam"})
plt.barh(labels, avg_remove.values, color="lightgreen")
plt.title("Average by Word 'hp'")
plt.xlabel("Average 'hp' Word")
plt.show()
```



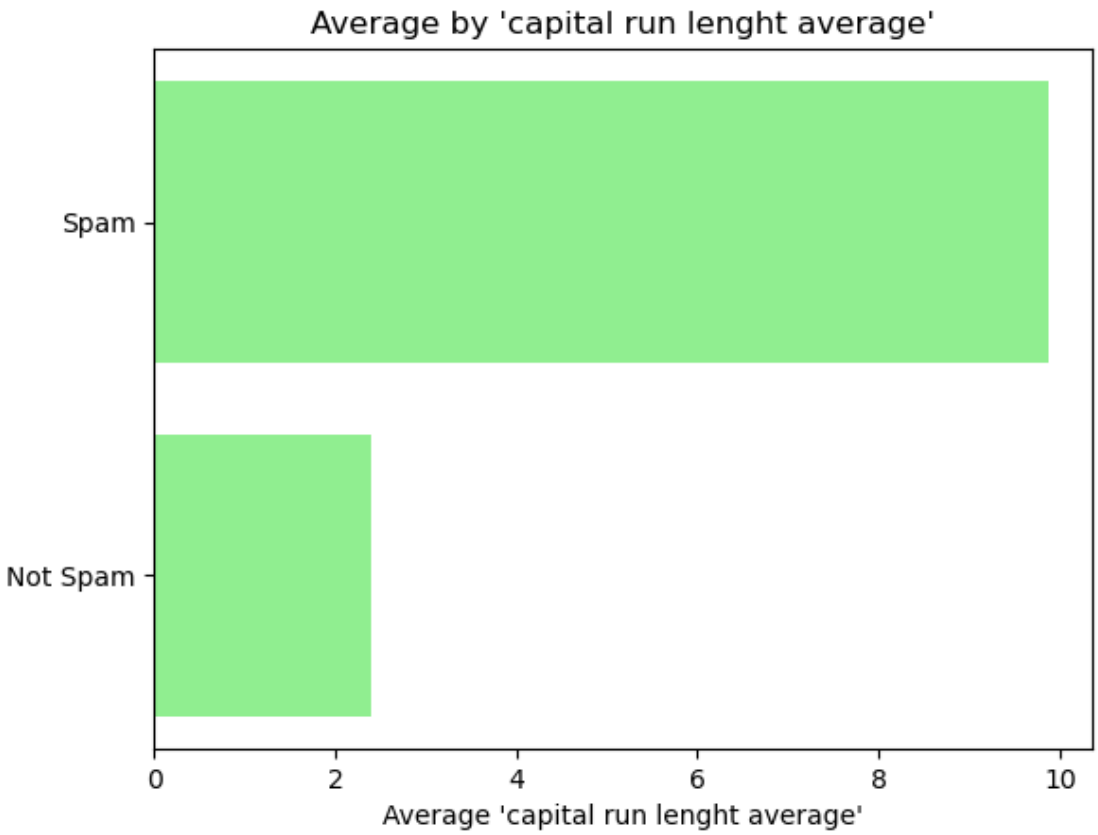
- **Objective:** Compare average word 'hp' usage in the spam or not spam emails.
- **Why this plot:** Horizontal bars are easier for categorical labels.
- **Learning:** The word 'hp' it's frequently used in non-spam emails.

[126]:
```
dff['is_spam_label']=dff['is_spam'].map({0: "Not Spam", 1: "Spam"})
sns.boxplot(y="word_freq_hp", x="is_spam_label", data=dff)
plt.title('Word "hp" Distribution by "is_spam"')
plt.show()
```



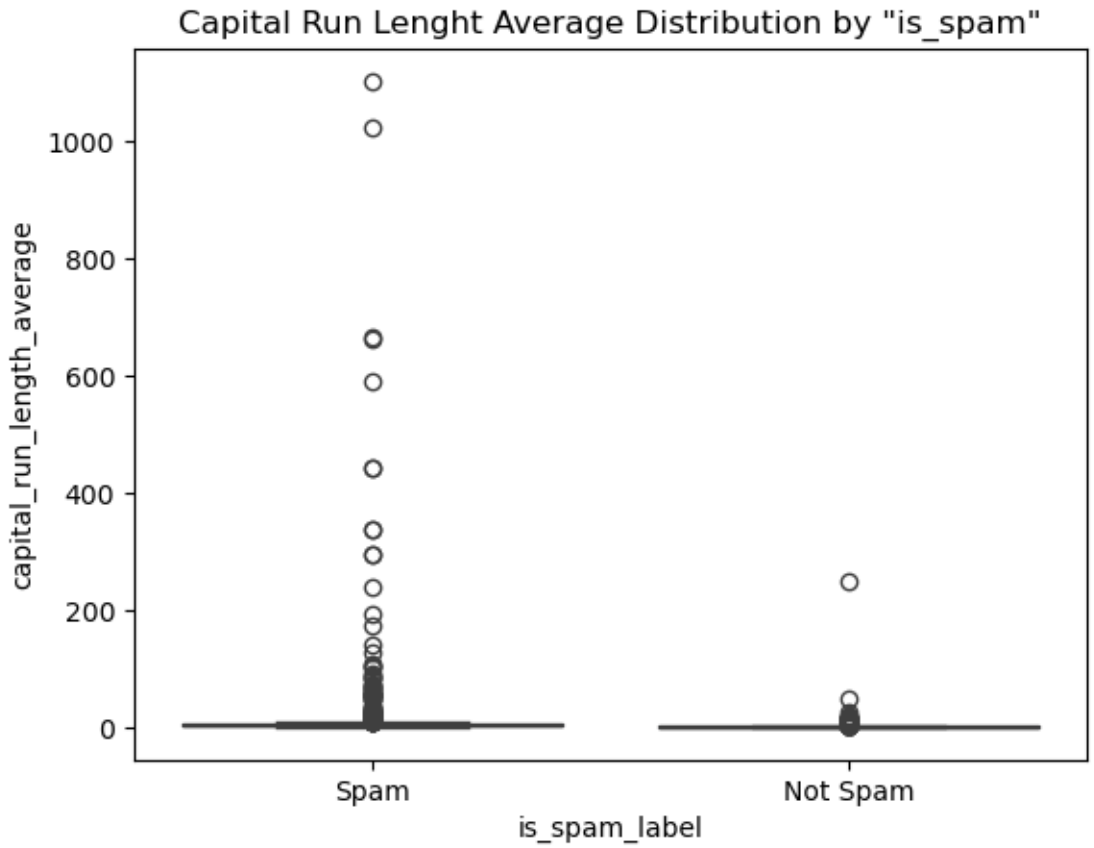
- **Objective:** Examine the word 'remove' distribution across different spam or not epam emails.
- **Why this plot:** Boxplots show spread, median, and outliers effectively.
- **Learning:** The word 'hp' is frequently used in non-spam emails, in some cases excessively as shown by the outliers.

[127]:
```
#9
avg_remove = dff.groupby("is_spam")["capital_run_length_average"].mean().
 ↪sort_values()
labels = avg_remove.index.map({0: "Not Spam", 1: "Spam"})
plt.barh(labels, avg_remove.values, color="lightgreen")
plt.title("Average by 'capital run lenght average'")
plt.xlabel("Average 'capital run lenght average'")
plt.show()
```



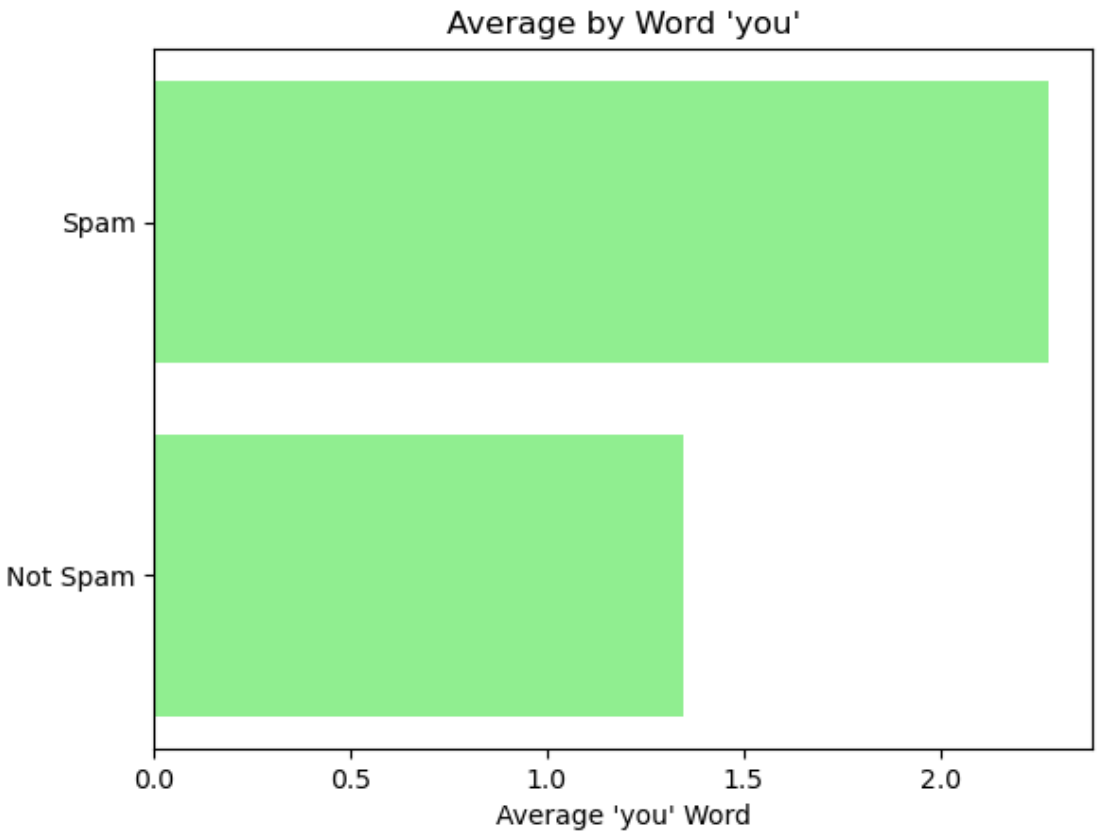
- **Objective:** Compare the capital run lenght average usage in the spam or not spam emails.
- **Why this plot:** Horizontal bars are easier for categorical labels.
- **Learning:** The capital run lenght average it's frequently founded in spam emails.

[128]:
```
dff['is_spam_label']=dff['is_spam'].map({0: "Not Spam", 1: "Spam"})
sns.boxplot(y="capital_run_length_average", x="is_spam_label", data=dff)
plt.title('Capital Run Lenght Average Distribution by "is_spam"')
plt.show()
```



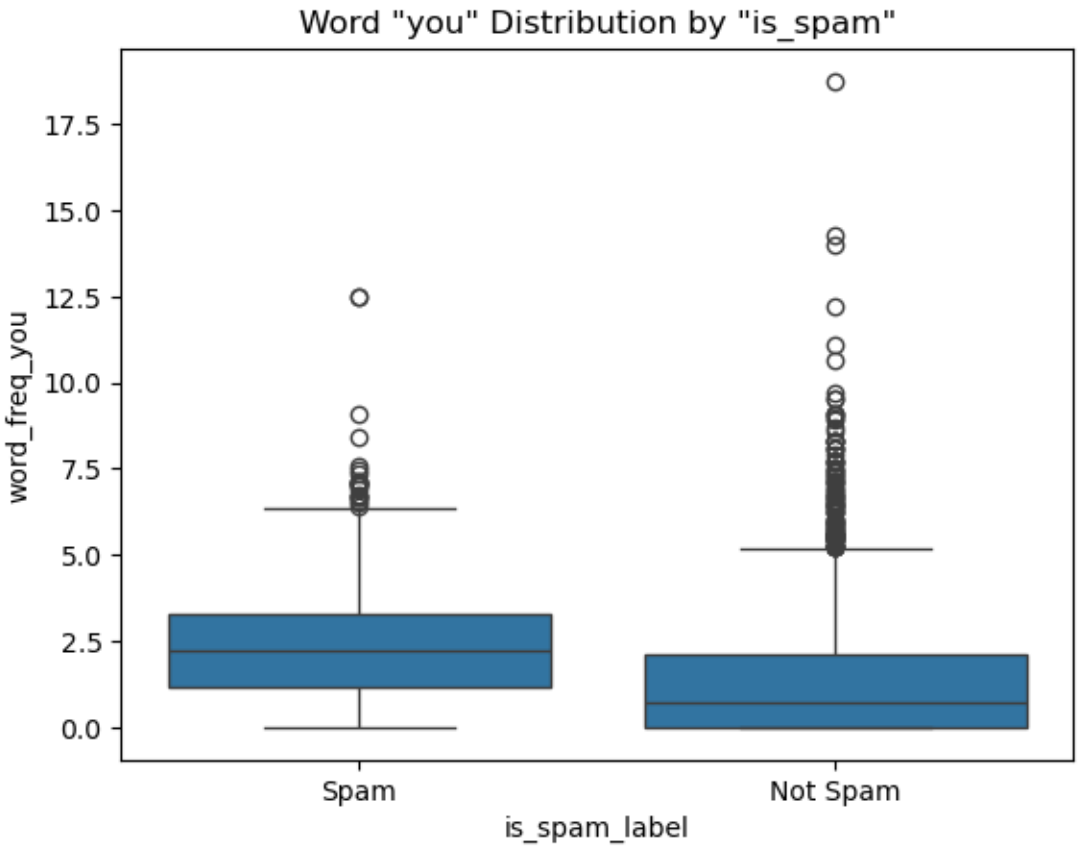
- **Objective:** Examine the capital run lenght average distribution across different spam or not epam emails.
- **Why this plot:** Boxplots show spread, median, and outliers effectively.
- **Learning:** This is another example of how the presence of words in uppercase letters in an email indicates it is spam, they are frequently founded in spam emails rather than non-spam emails.

[129]:
```
#10
avg_remove = dff.groupby("is_spam")["word_freq_you"].mean().sort_values()
labels = avg_remove.index.map({0: "Not Spam", 1: "Spam"})
plt.barh(labels, avg_remove.values, color="lightgreen")
plt.title("Average by Word 'you'")
plt.xlabel("Average 'you' Word")
plt.show()
```



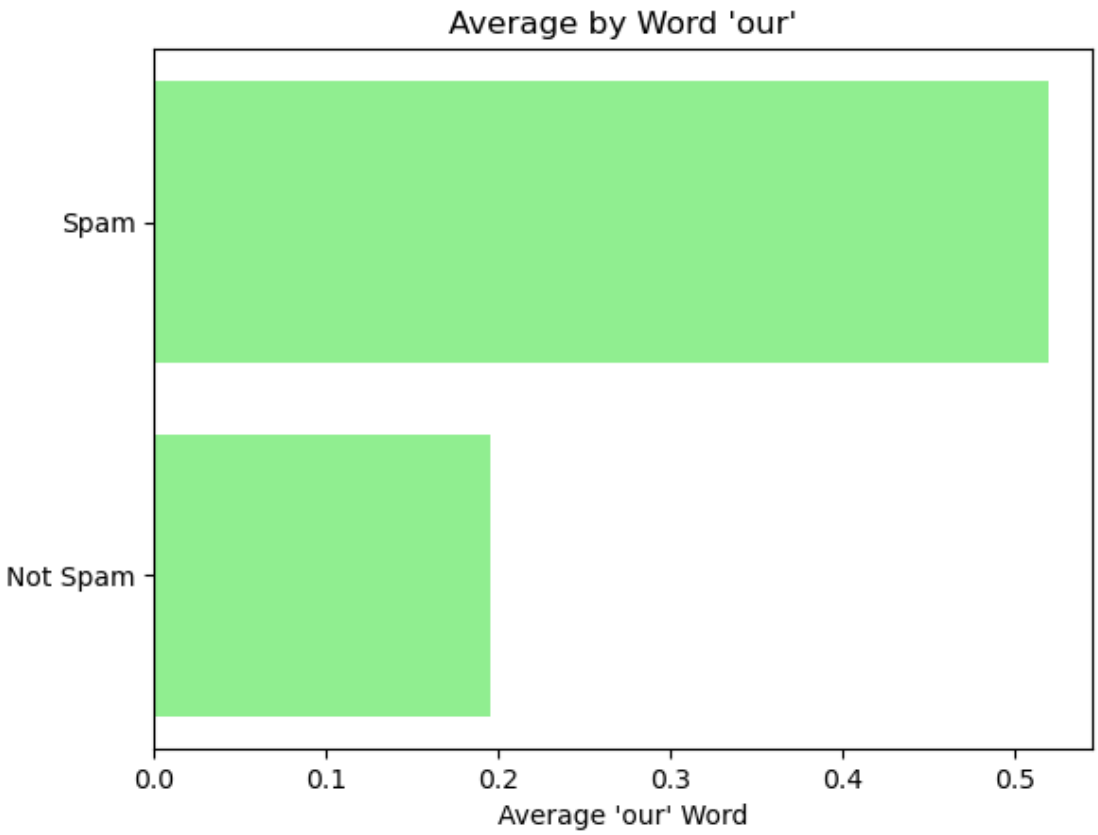
- **Objective:** Compare average word ‘you’ usage in the spam or not spam emails.
- **Why this plot:** Horizontal bars are easier for categorical labels.
- **Learning:** The word ‘you’ it’s frequently used in spam and non-spam emails.

[130]:
```
dff['is_spam_label']=dff['is_spam'].map({0: "Not Spam", 1: "Spam"})
sns.boxplot(y="word_freq_you", x="is_spam_label", data=dff)
plt.title('Word "you" Distribution by "is_spam"')
plt.show()
```



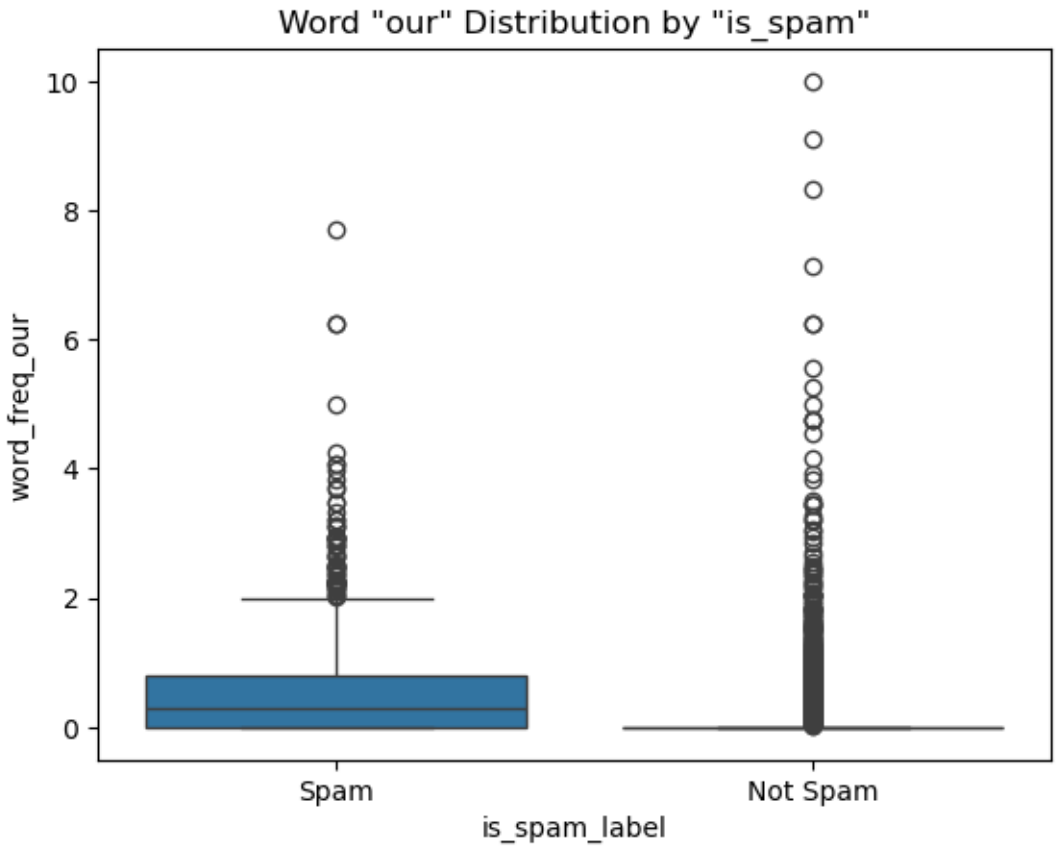
- **Objective:** Examine the word 'you' distribution across different spam or not epam emails.
- **Why this plot:** Boxplots show spread, median, and outliers effectively.
- **Learning:** This is another example of a word common in people's everyday language, normally found in both scenarios, indicating caution in using this variable to determine if it is spam or not, although more frequently used in spam emails in this example, it is also easily found in non-spam emails, in some cases even with greater frequency than in a spam email.

[131]:
```
#11
avg_remove = dff.groupby("is_spam")["word_freq_our"].mean().sort_values()
labels = avg_remove.index.map({0: "Not Spam", 1: "Spam"})
plt.barh(labels, avg_remove.values, color="lightgreen")
plt.title("Average by Word 'our'")
plt.xlabel("Average 'our' Word")
plt.show()
```



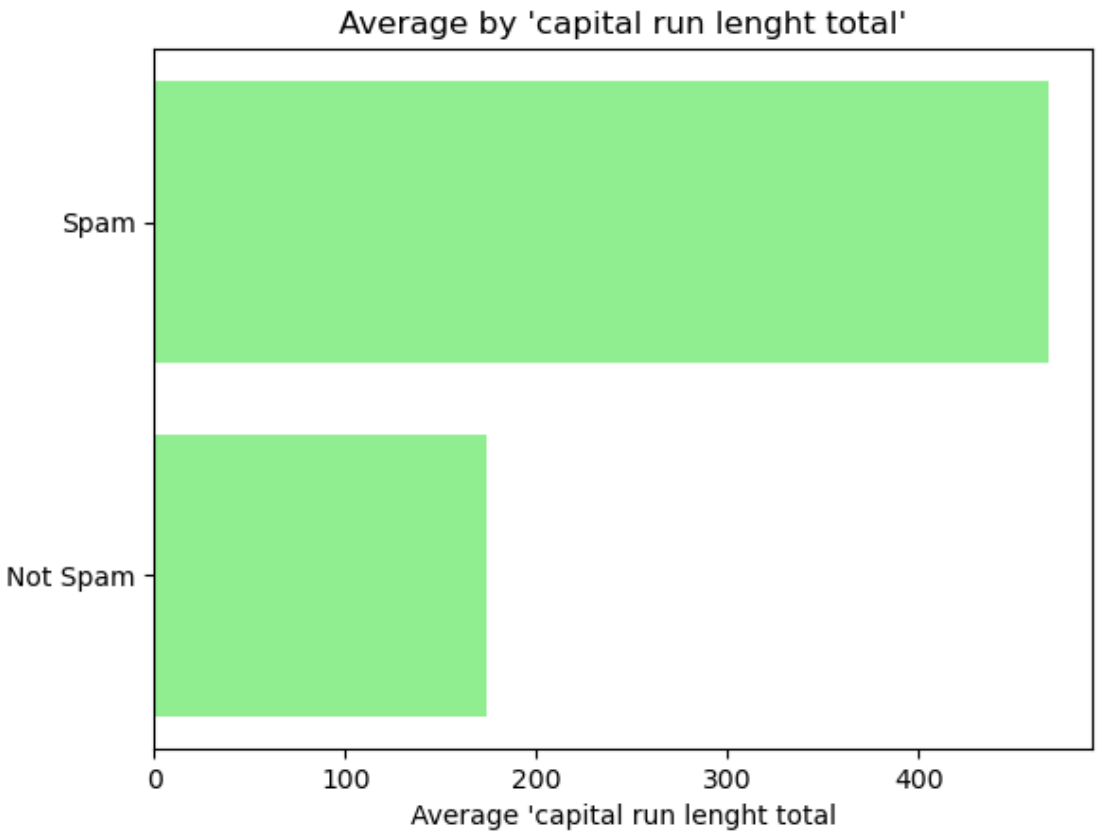
- **Objective:** Compare average word ‘our’ usage in the spam or not spam emails.
- **Why this plot:** Horizontal bars are easier for categorical labels.
- **Learning:** The word ‘our’ it’s frequently used in spam emails.

[132]:
```
dff['is_spam_label']=dff['is_spam'].map({0: "Not Spam", 1: "Spam"})
sns.boxplot(y="word_freq_our", x="is_spam_label", data=dff)
plt.title('Word "our" Distribution by "is_spam"')
plt.show()
```



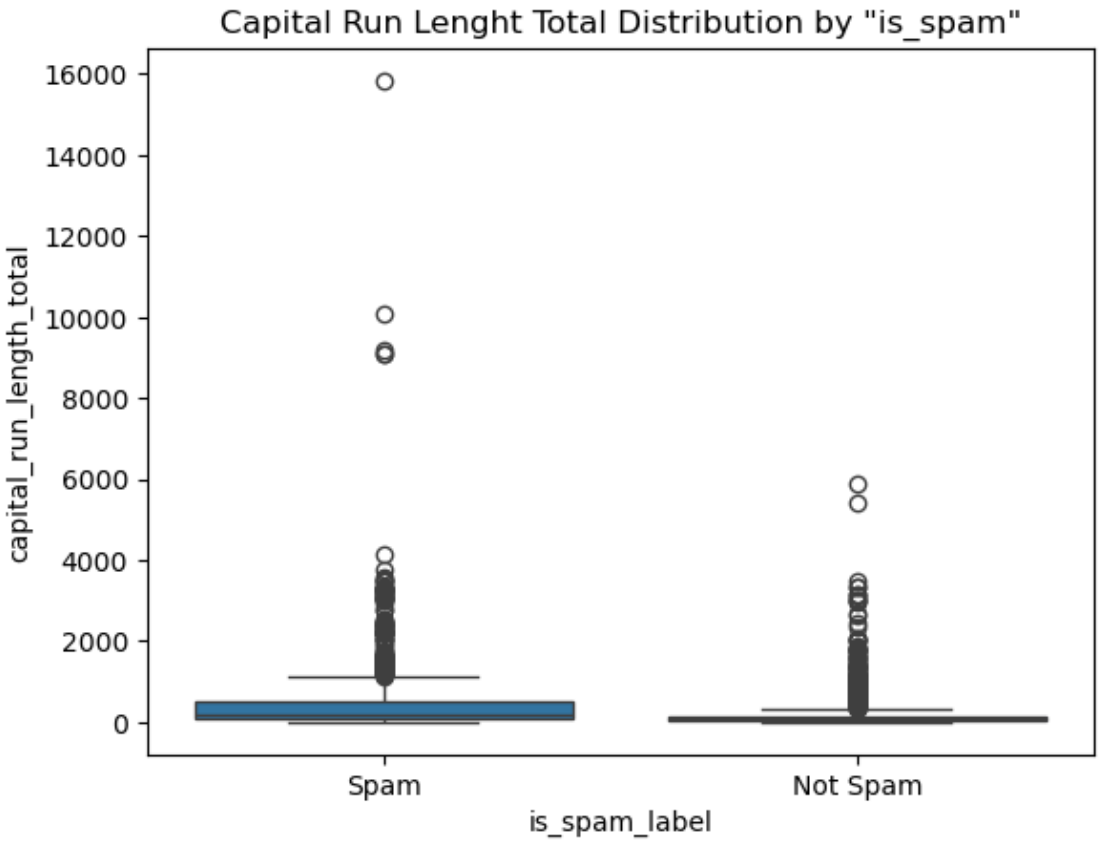
- **Objective:** Examine the word 'our' distribution across different spam or not epam emails.
- **Why this plot:** Boxplots show spread, median, and outliers effectively.
- **Learning:** Again, a word common in everyone's everyday language, commonly found in both scenarios and once more indicating caution in using this variable to determine if it is spam or not. The graph shows a more common presence in spam, but in certain cases it is also heavily used in non-spam emails.

[133]:
```
#12
avg_remove = dff.groupby("is_spam")["capital_run_length_total"].mean().
 ↪sort_values()
labels = avg_remove.index.map({0: "Not Spam", 1: "Spam"})
plt.barh(labels, avg_remove.values, color="lightgreen")
plt.title("Average by 'capital run lenght total'")
plt.xlabel("Average 'capital run lenght total")
plt.show()
```



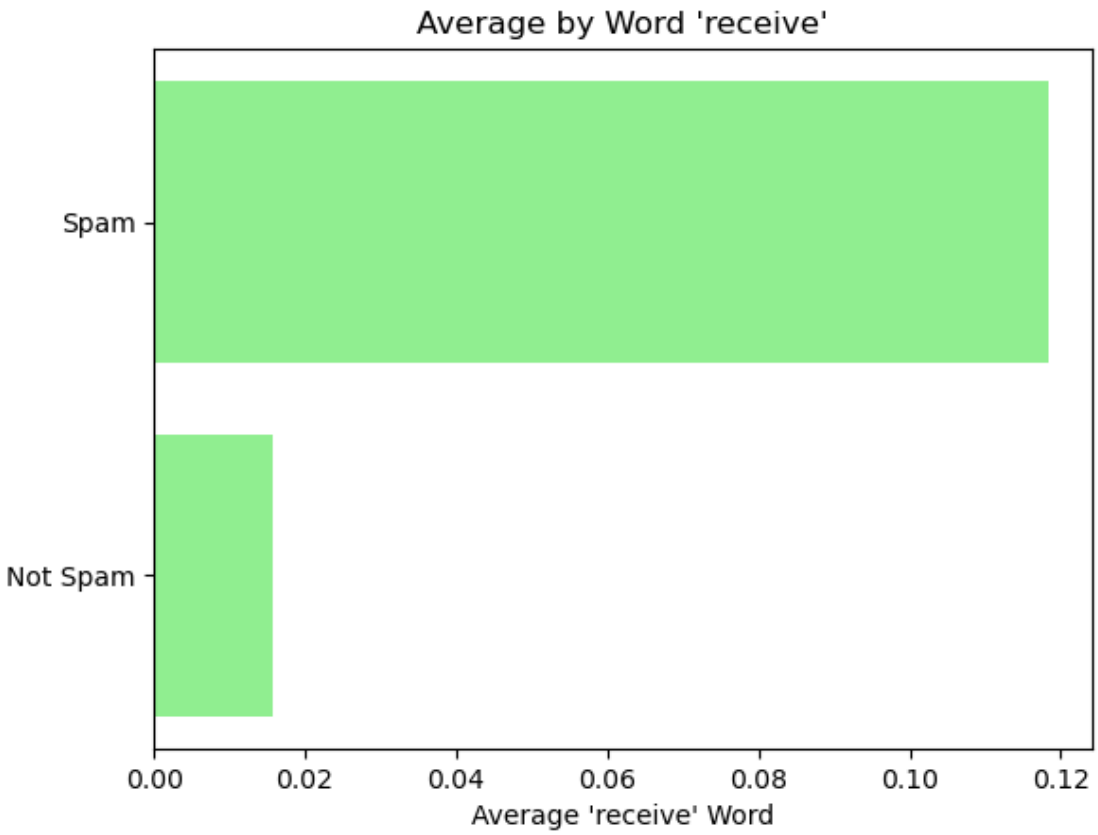
- **Objective:** Compare he capital run lenght total usage in the spam or not spam emails.
- **Why this plot:** Horizontal bars are easier for categorical labels.
- **Learning:** The capital run lenght total it's frequently used in spam emails.

[134]:
```
dff['is_spam_label']=dff['is_spam'].map({0: "Not Spam", 1: "Spam"})
sns.boxplot(y="capital_run_length_total", x="is_spam_label", data=dff)
plt.title('Capital Run Lenght Total Distribution by "is_spam"')
plt.show()
```



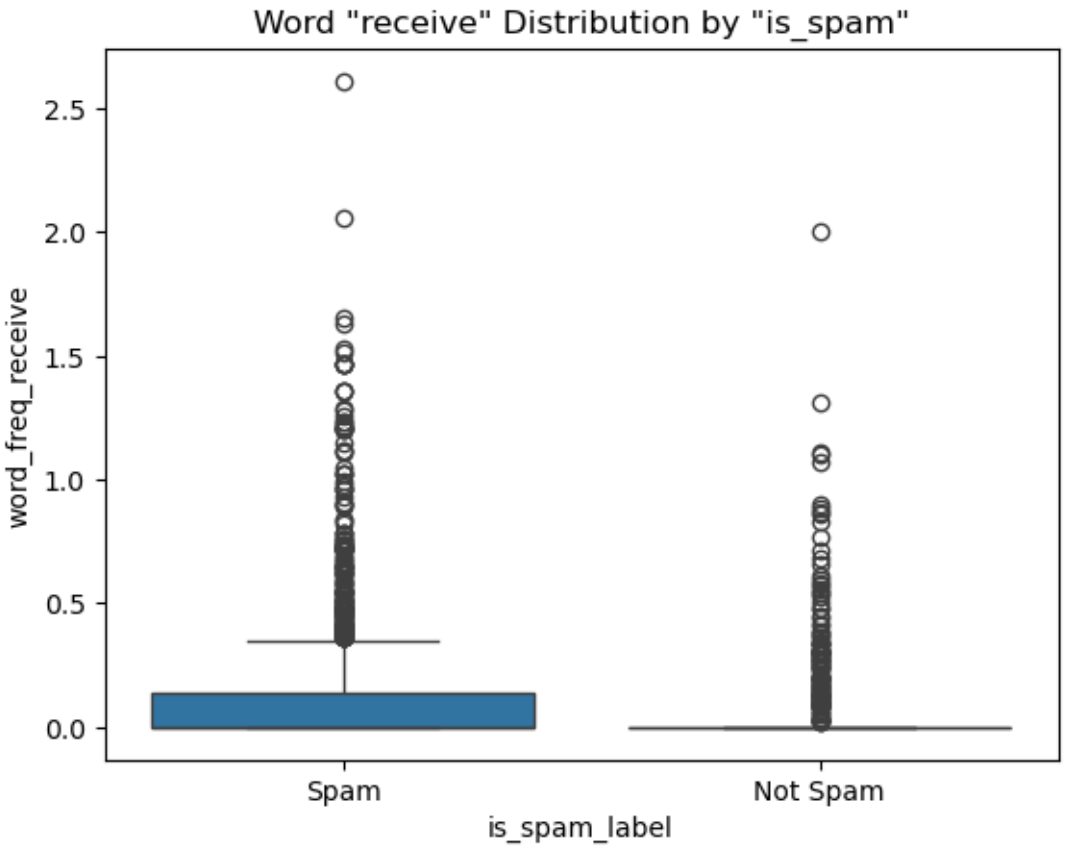
- **Objective:** Examine the capital run lenght total distribution across different spam or not epam emails.
- **Why this plot:** Boxplots show spread, median, and outliers effectively.
- **Learning:** In this case, we compare the total number of words in uppercase letters, which is somewhat less uncommon to find in emails, but once again indicating that a high presence of words in uppercase letters suggests the email may be spam.

[135]:
```
#13
avg_remove = dff.groupby("is_spam")["word_freq_receive"].mean().sort_values()
labels = avg_remove.index.map({0: "Not Spam", 1: "Spam"})
plt.barh(labels, avg_remove.values, color="lightgreen")
plt.title("Average by Word 'receive'")
plt.xlabel("Average 'receive' Word")
plt.show()
```



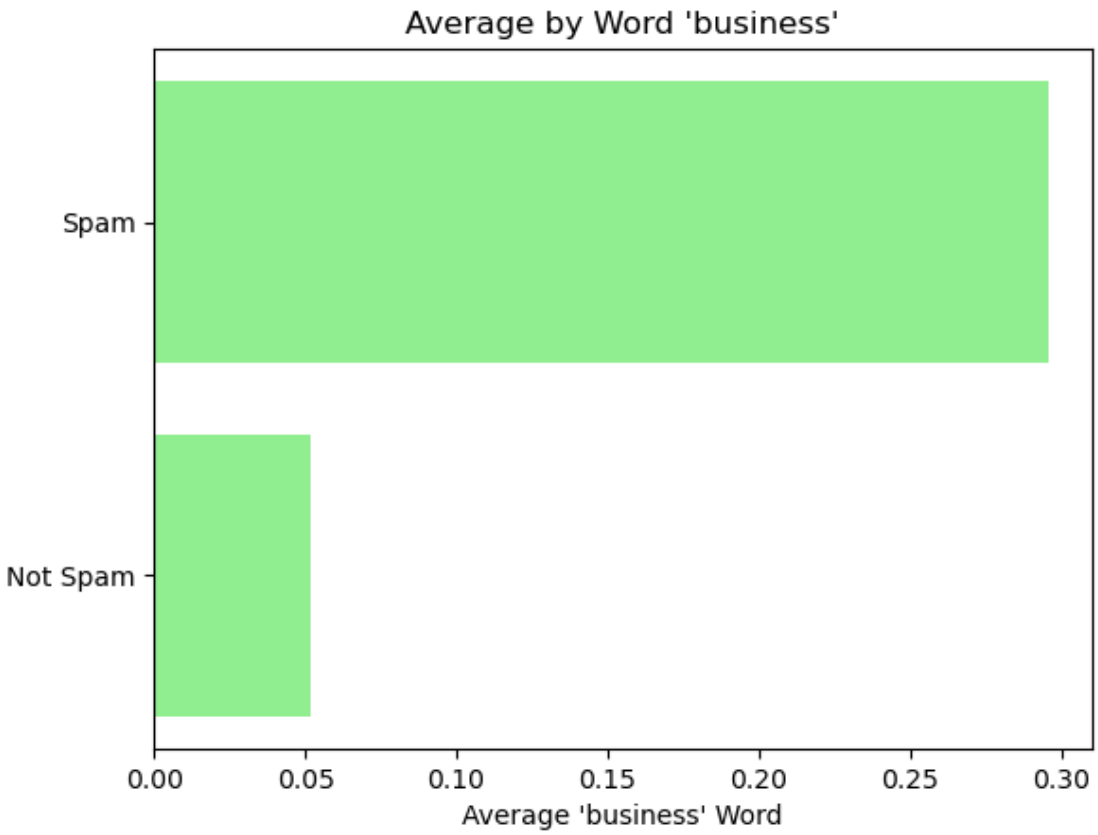
- **Objective:** Compare average word ‘receive’ usage in the spam or not spam emails.
- **Why this plot:** Horizontal bars are easier for categorical labels.
- **Learning:** The word ‘receive’ it’s frequently used in spam emails.

[136]:
```
dff['is_spam_label']=dff['is_spam'].map({0: "Not Spam", 1: "Spam"})
sns.boxplot(y="word_freq_receive", x="is_spam_label", data=dff)
plt.title('Word "receive" Distribution by "is_spam"')
plt.show()
```



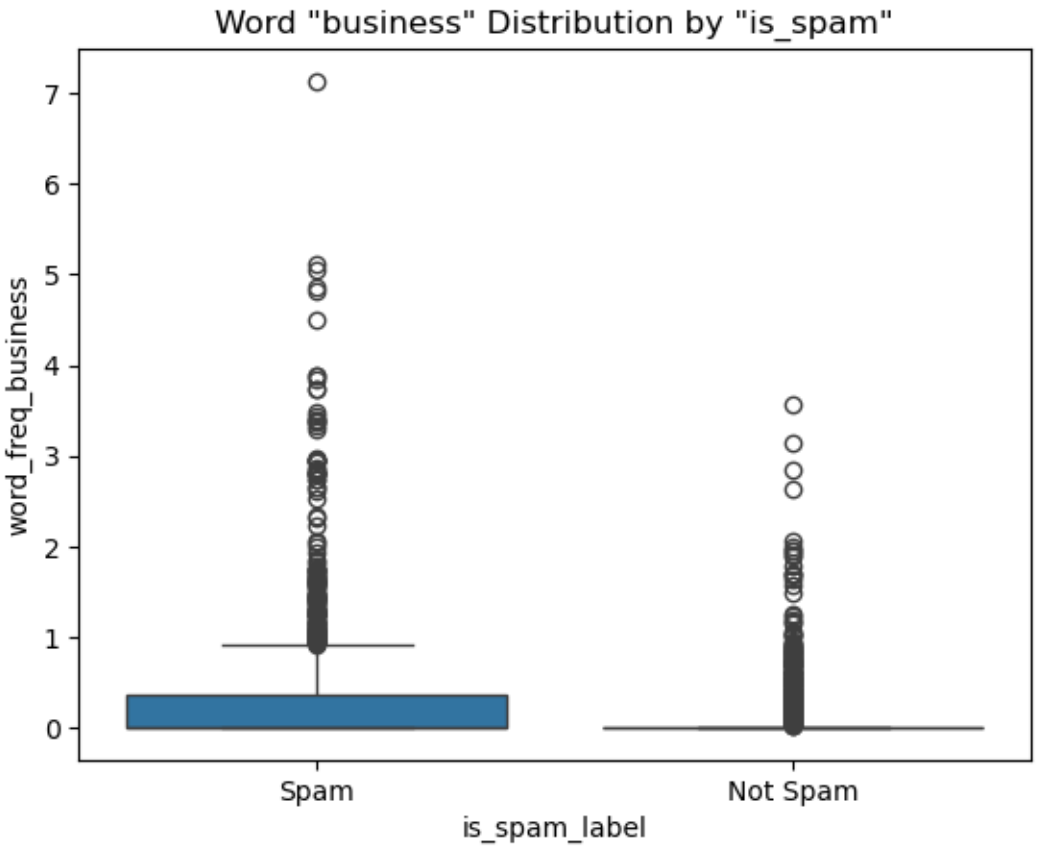
- **Objective:** Examine the word 'remove' distribution across different spam or not epam emails.
- **Why this plot:** Boxplots show spread, median, and outliers effectively.
- **Learning:** The word 'receive' is normally found in non-spam emails too, but it is more frequently used in spam emails, dispite both scenarios have excessively usage of the word in some cases.

[137]:
```
#14
avg_remove = dff.groupby("is_spam")["word_freq_business"].mean().sort_values()
labels = avg_remove.index.map({0: "Not Spam", 1: "Spam"})
plt.barh(labels, avg_remove.values, color="lightgreen")
plt.title("Average by Word 'business'")
plt.xlabel("Average 'business' Word")
plt.show()
```



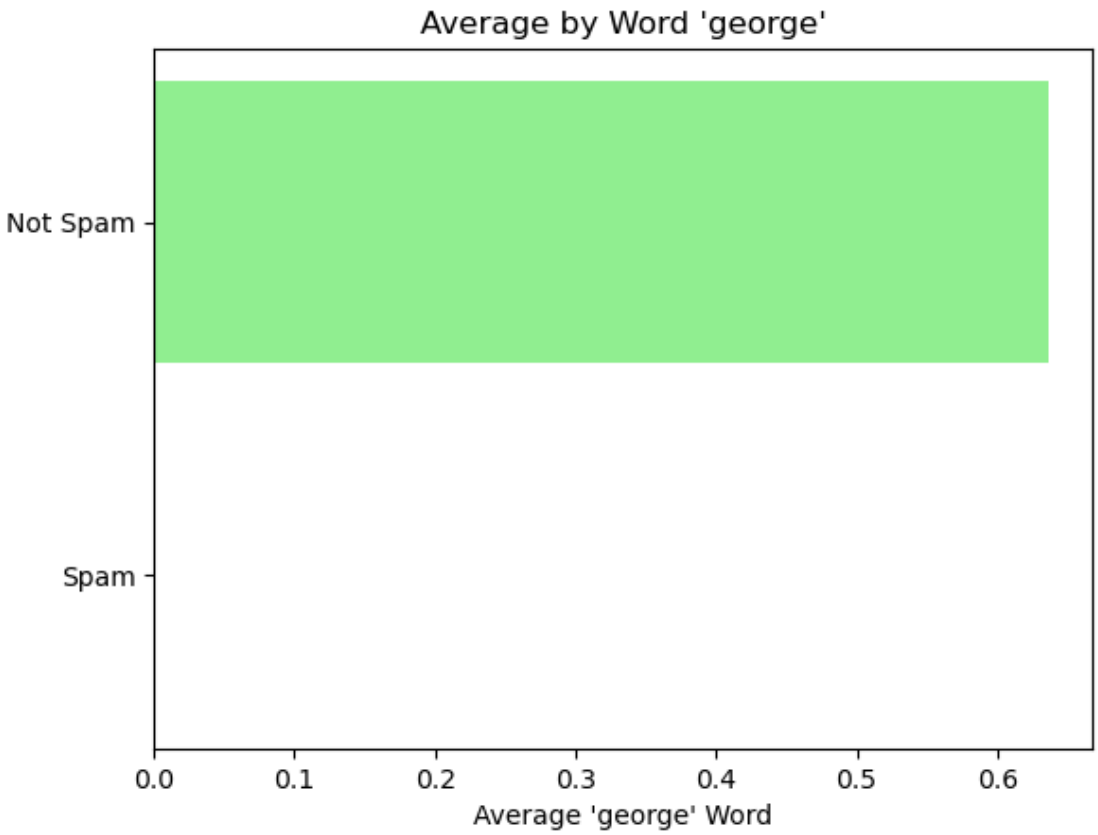
- **Objective:** Compare average word 'business' usage in the spam or not spam emails.
- **Why this plot:** Horizontal bars are easier for categorical labels.
- **Learning:** The word 'business' it's frequently used in spam emails.

[138]:
```
dff['is_spam_label']=dff['is_spam'].map({0: "Not Spam", 1: "Spam"})
sns.boxplot(y="word_freq_business", x="is_spam_label", data=dff)
plt.title('Word "business" Distribution by "is_spam"')
plt.show()
```



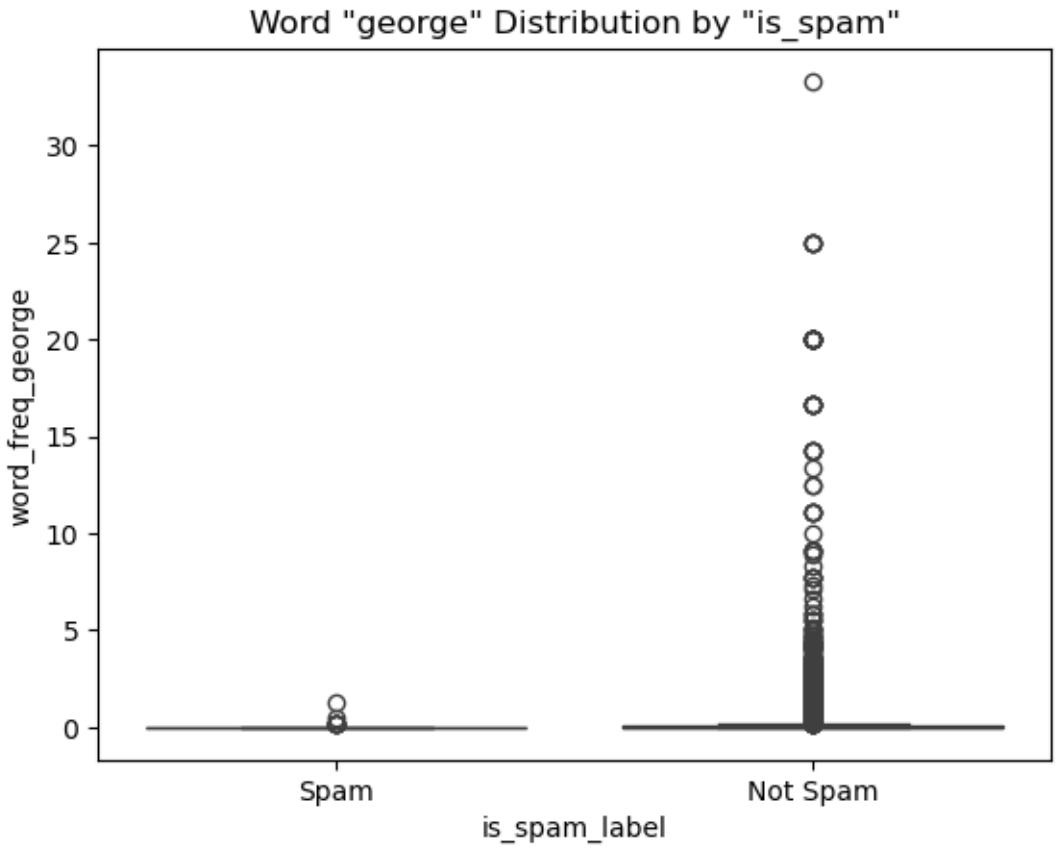
- **Objective:** Examine the word 'business' distribution across different spam or not epam emails.
- **Why this plot:** Boxplots show spread, median, and outliers effectively.
- **Learning:** The word 'business' is normally found in non-spam emails too, but it is more frequently used in spam emails.

[139]:
```
#15
avg_remove = dff.groupby("is_spam")["word_freq_george"].mean().sort_values()
labels = avg_remove.index.map({0: "Not Spam", 1: "Spam"})
plt.barh(labels, avg_remove.values, color="lightgreen")
plt.title("Average by Word 'george'")
plt.xlabel("Average 'george' Word")
plt.show()
```



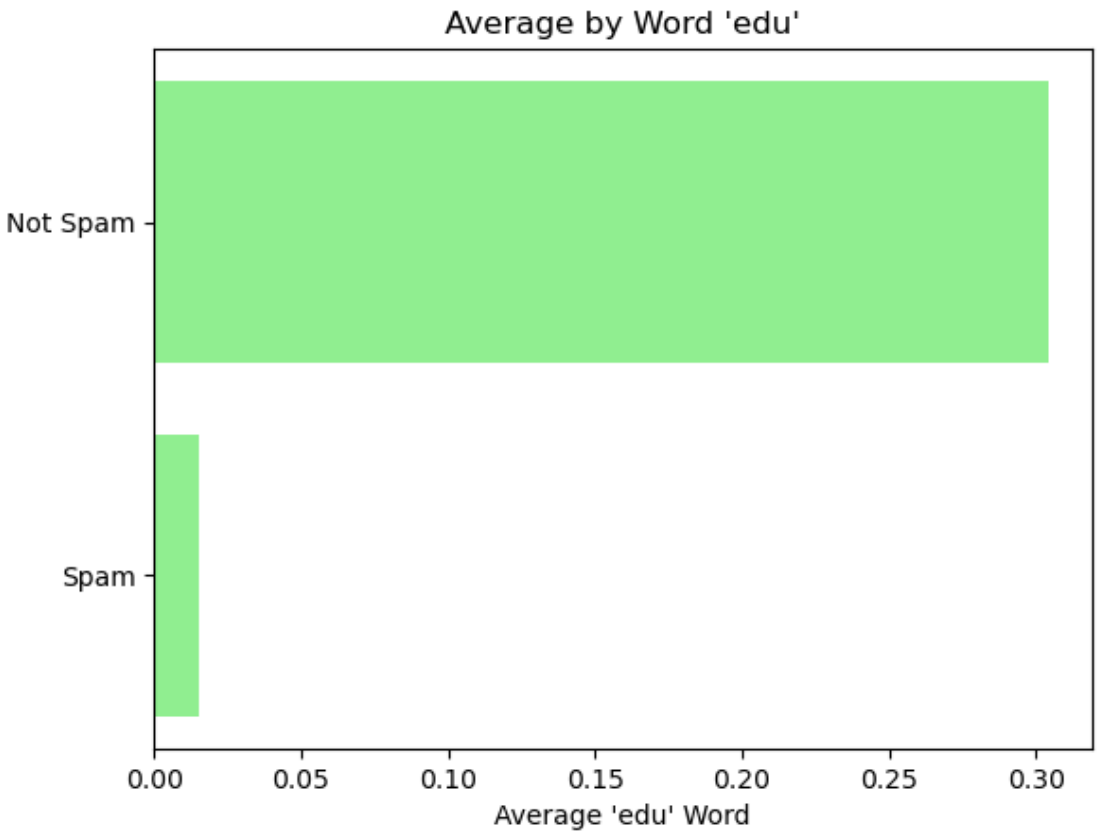
- **Objective:** Compare average word 'george' usage in the spam or not spam emails.
- **Why this plot:** Horizontal bars are easier for categorical labels.
- **Learning:** The word 'george' it's frequently used in non-spam emails.

[140]:
```
dff['is_spam_label']=dff['is_spam'].map({0: "Not Spam", 1: "Spam"})
sns.boxplot(y="word_freq_george", x="is_spam_label", data=dff)
plt.title('Word "george" Distribution by "is_spam"')
plt.show()
```



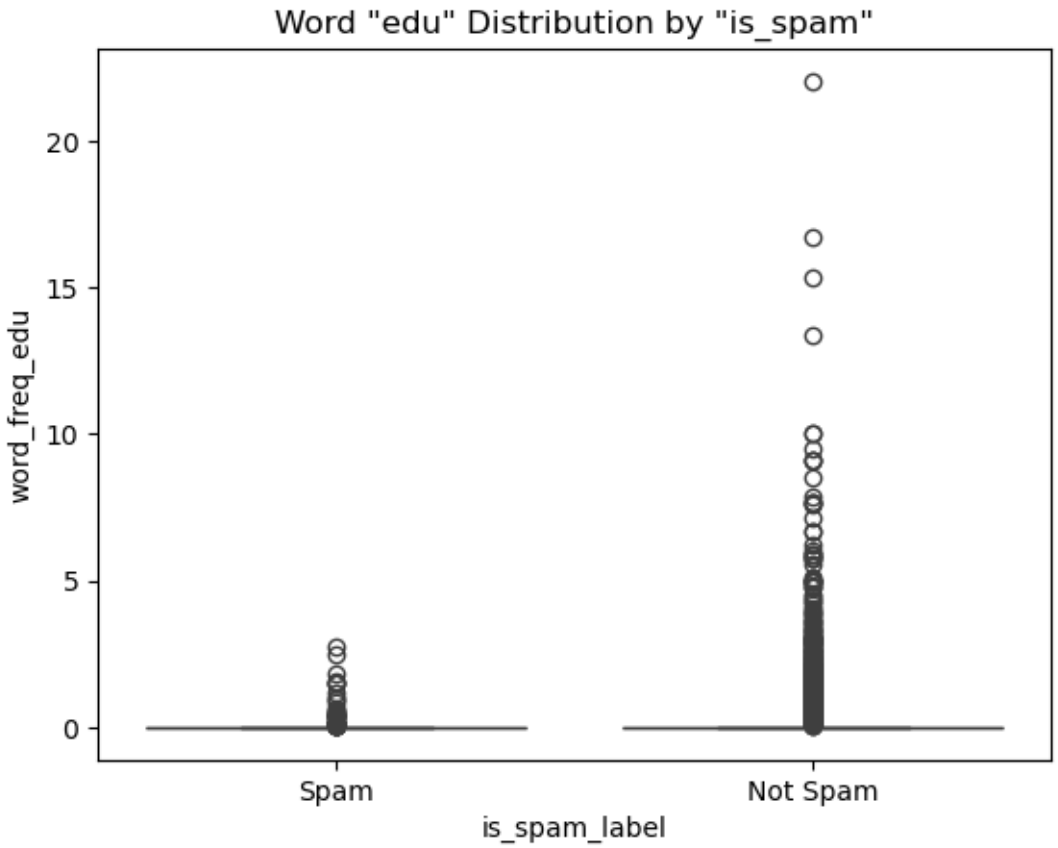
- **Objective:** Examine the word 'george' distribution across different spam or not epam emails.
- **Why this plot:** Boxplots show spread, median, and outliers effectively.
- **Learning:** The word 'george' is rarely used in people's daily lives, which justifies its low presence in spam emails, showing it to be a strong indicator for determining whether the email is spam or not.

[141]:
```
#16
avg_remove = dff.groupby("is_spam")["word_freq_edu"].mean().sort_values()
labels = avg_remove.index.map({0: "Not Spam", 1: "Spam"})
plt.barh(labels, avg_remove.values, color="lightgreen")
plt.title("Average by Word 'edu'")
plt.xlabel("Average 'edu' Word")
plt.show()
```



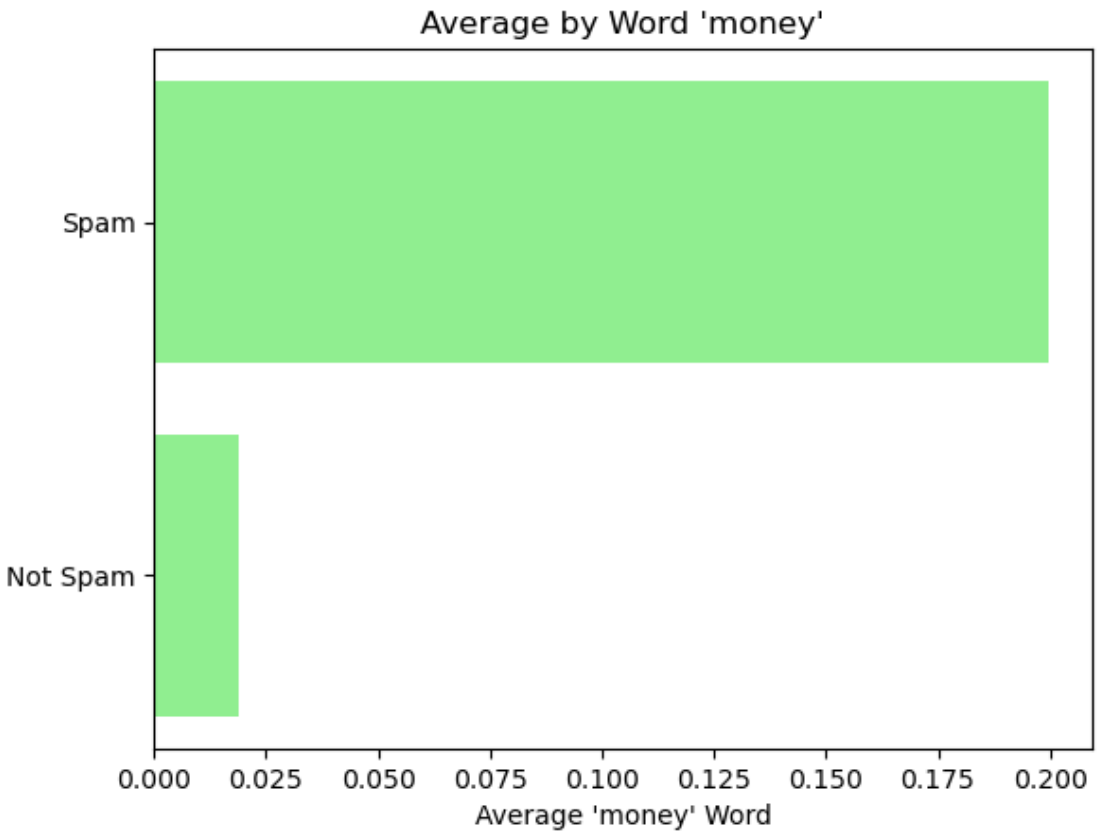
- **Objective:** Compare average word ‘edu’ usage in the spam or not spam emails.
- **Why this plot:** Horizontal bars are easier for categorical labels.
- **Learning:** The word ‘edu’ it’s frequently used in non-spam emails.

[142]:
```
dff['is_spam_label']=dff['is_spam'].map({0: "Not Spam", 1: "Spam"})
sns.boxplot(y="word_freq_edu", x="is_spam_label", data=dff)
plt.title('Word "edu" Distribution by "is_spam"')
plt.show()
```



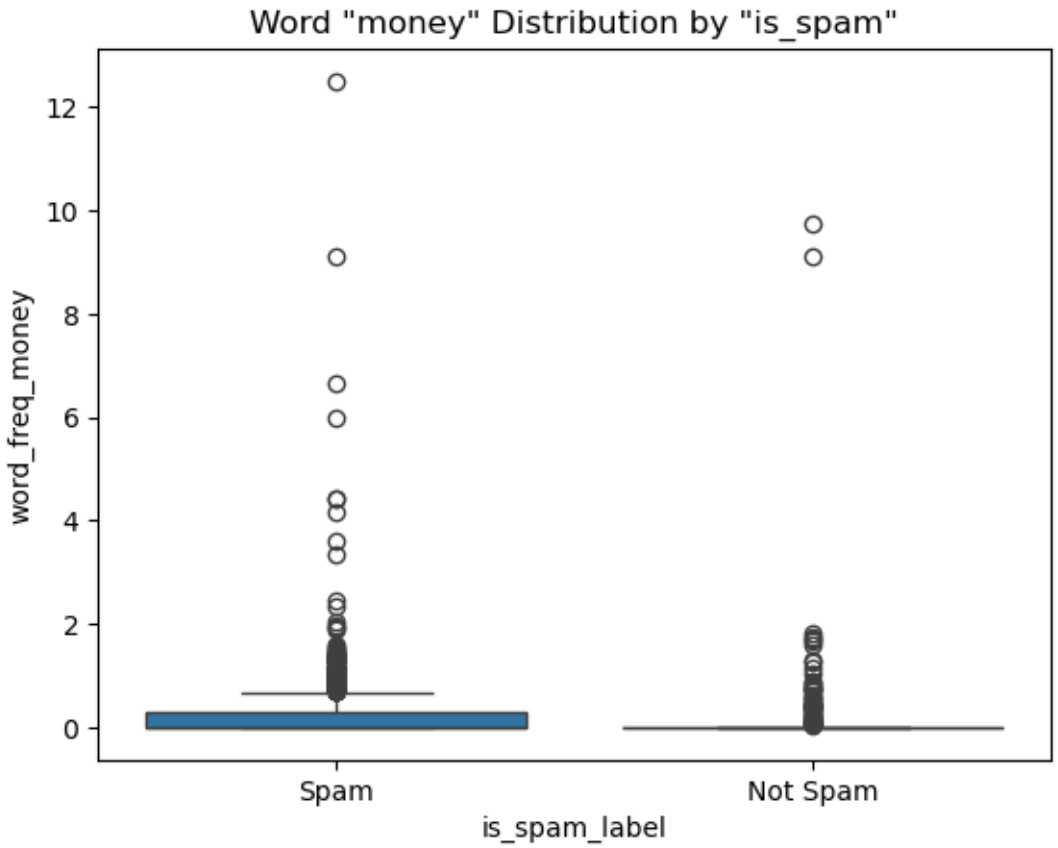
- **Objective:** Examine the word 'edu' distribution across different spam or not epam emails.
- **Why this plot:** Boxplots show spread, median, and outliers effectively.
- **Learning:** The word 'edu' is normally found in non-spam emails, with a very low presence in spam emails, showing it to be a strong indicator for determining whether the email is spam or not.

[143]:
```
#17
avg_remove = dff.groupby("is_spam")["word_freq_money"].mean().sort_values()
labels = avg_remove.index.map({0: "Not Spam", 1: "Spam"})
plt.barh(labels, avg_remove.values, color="lightgreen")
plt.title("Average by Word 'money'")
plt.xlabel("Average 'money' Word")
plt.show()
```



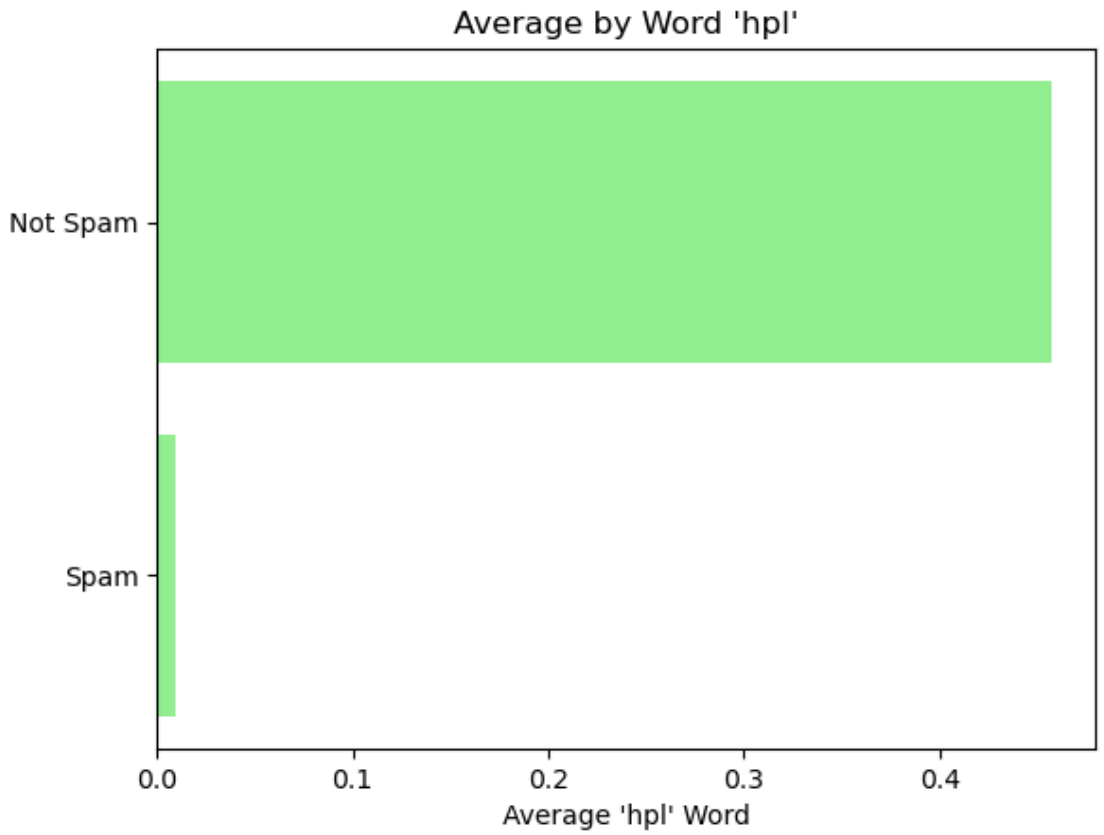
- **Objective:** Compare average word ‘money’ usage in the spam or not spam emails.
- **Why this plot:** Horizontal bars are easier for categorical labels.
- **Learning:** The word ‘money’ it’s frequently used in spam emails.

[144]:
```
dff['is_spam_label']=dff['is_spam'].map({0: "Not Spam", 1: "Spam"})
sns.boxplot(y="word_freq_money", x="is_spam_label", data=dff)
plt.title('Word "money" Distribution by "is_spam"')
plt.show()
```



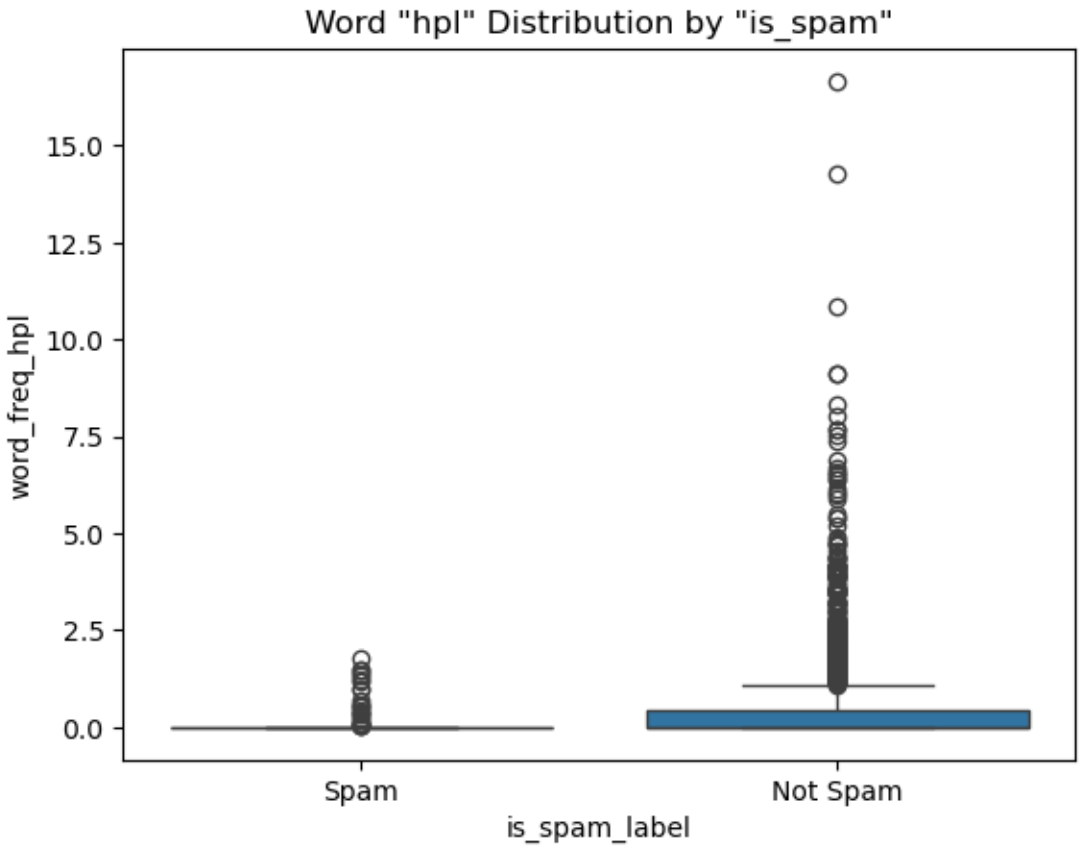
- **Objective:** Examine the word 'money' distribution across different spam or not epam emails.
- **Why this plot:** Boxplots show spread, median, and outliers effectively.
- **Learning:** The word 'money' is normally found spam emails, Even though it is not uncommon to find it in non-spam emails, it is frequently used in spam.

[145]:
```
#18
avg_remove = dff.groupby("is_spam")["word_freq_hpl"].mean().sort_values()
labels = avg_remove.index.map({0: "Not Spam", 1: "Spam"})
plt.barh(labels, avg_remove.values, color="lightgreen")
plt.title("Average by Word 'hpl'")
plt.xlabel("Average 'hpl' Word")
plt.show()
```



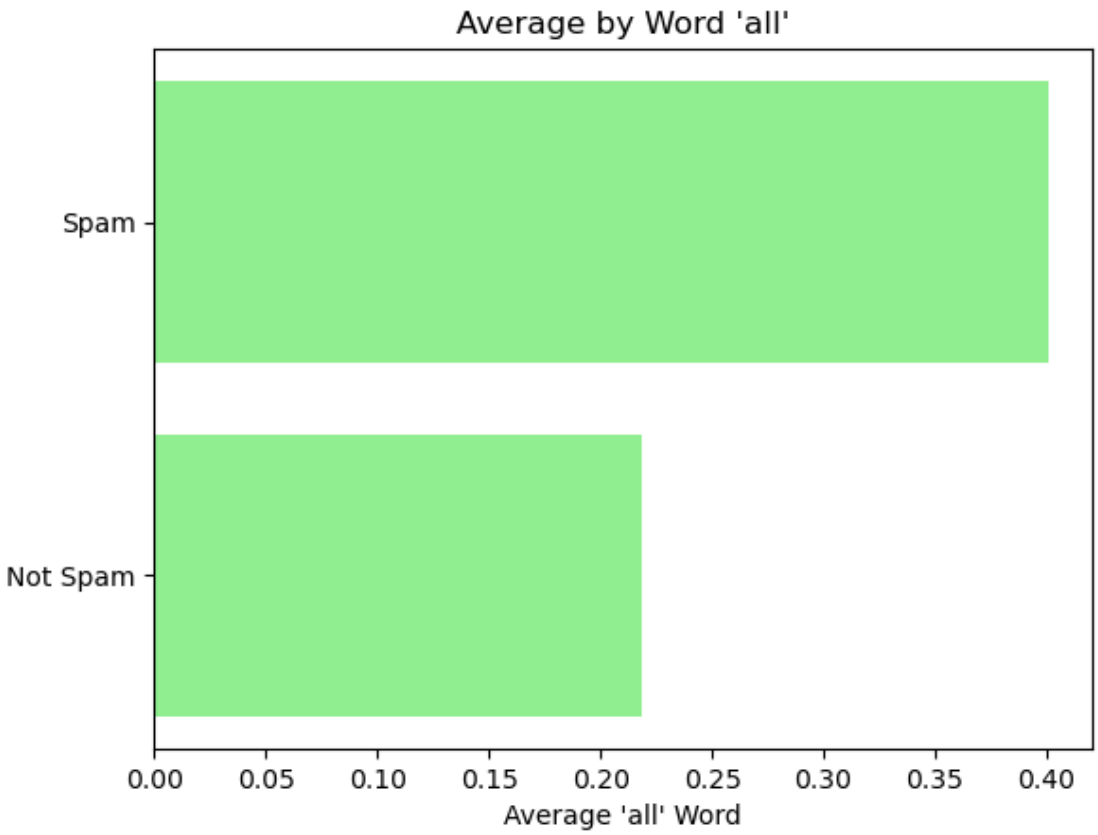
- **Objective:** Compare average word 'hpl' usage in the spam or not spam emails.
- **Why this plot:** Horizontal bars are easier for categorical labels.
- **Learning:** The word 'hpl' it's frequently used in non-spam emails.

[146]:
```
dff['is_spam_label']=dff['is_spam'].map({0: "Not Spam", 1: "Spam"})
sns.boxplot(y="word_freq_hpl", x="is_spam_label", data=dff)
plt.title('Word "hpl" Distribution by "is_spam"')
plt.show()
```



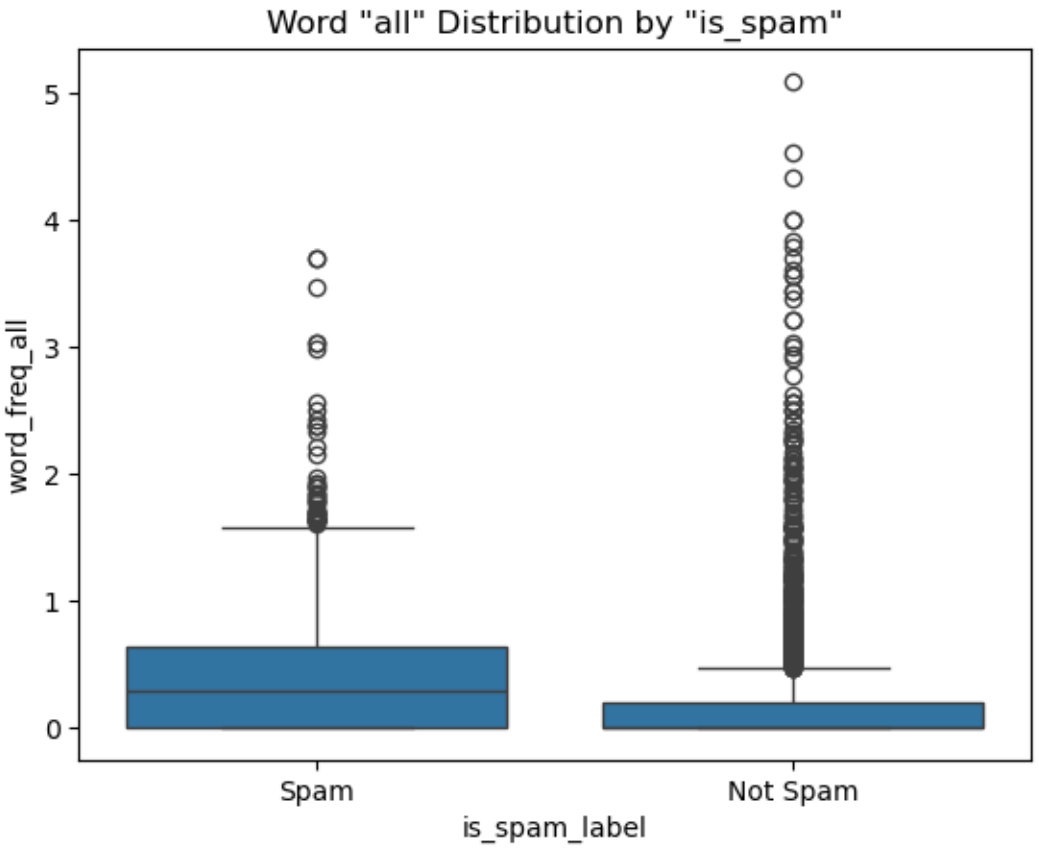
- **Objective:** Examine the word 'hpl' distribution across different spam or not epam emails.
- **Why this plot:** Boxplots show spread, median, and outliers effectively.
- **Learning:** The word 'hpl' is also not commonly used in daily life, which justifies its low presence in spam emails, showing it to be another strong indicator for determining whether it is spam or not.

[147]:
```
#19
avg_remove = dff.groupby("is_spam")["word_freq_all"].mean().sort_values()
labels = avg_remove.index.map({0: "Not Spam", 1: "Spam"})
plt.barh(labels, avg_remove.values, color="lightgreen")
plt.title("Average by Word 'all'")
plt.xlabel("Average 'all' Word")
plt.show()
```



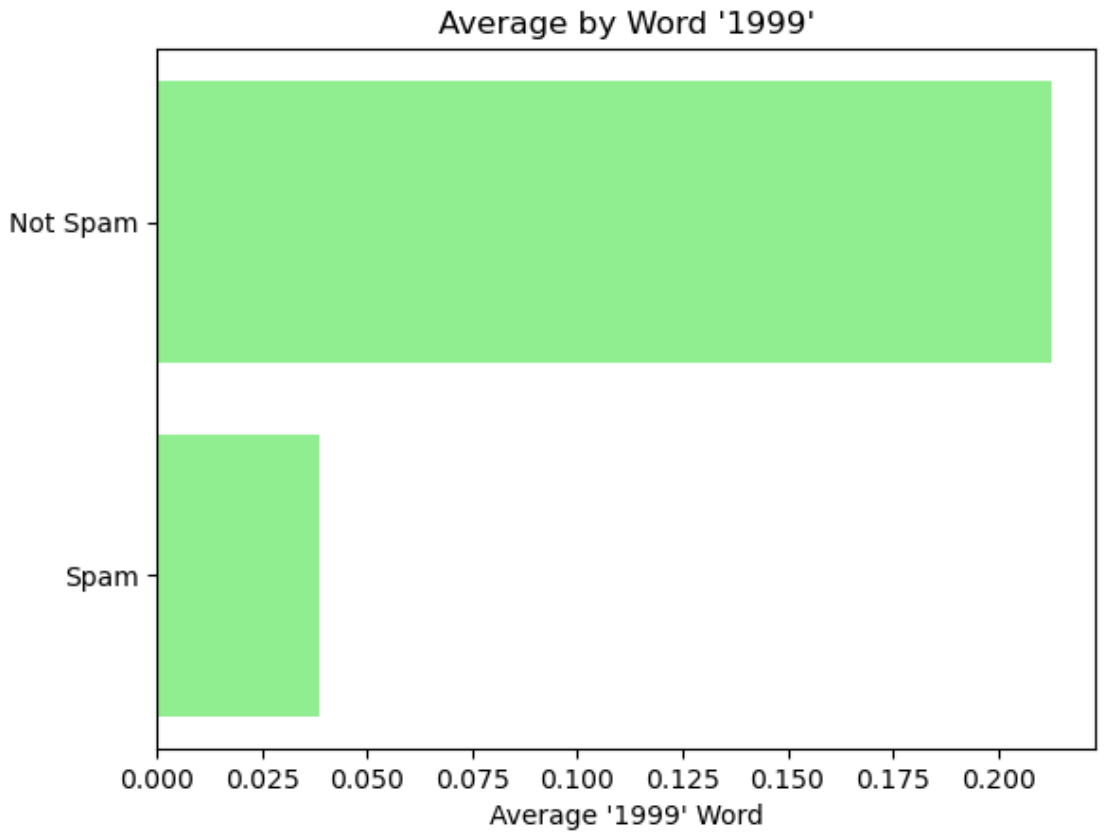
- **Objective:** Compare average word ‘all’ usage in the spam or not spam emails.
- **Why this plot:** Horizontal bars are easier for categorical labels.
- **Learning:** The word ‘all’ it’s frequently used in both spam and non-spam emails.

[148]:
```
dff['is_spam_label']=dff['is_spam'].map({0: "Not Spam", 1: "Spam"})
sns.boxplot(y="word_freq_all", x="is_spam_label", data=dff)
plt.title('Word "all" Distribution by "is_spam"')
plt.show()
```



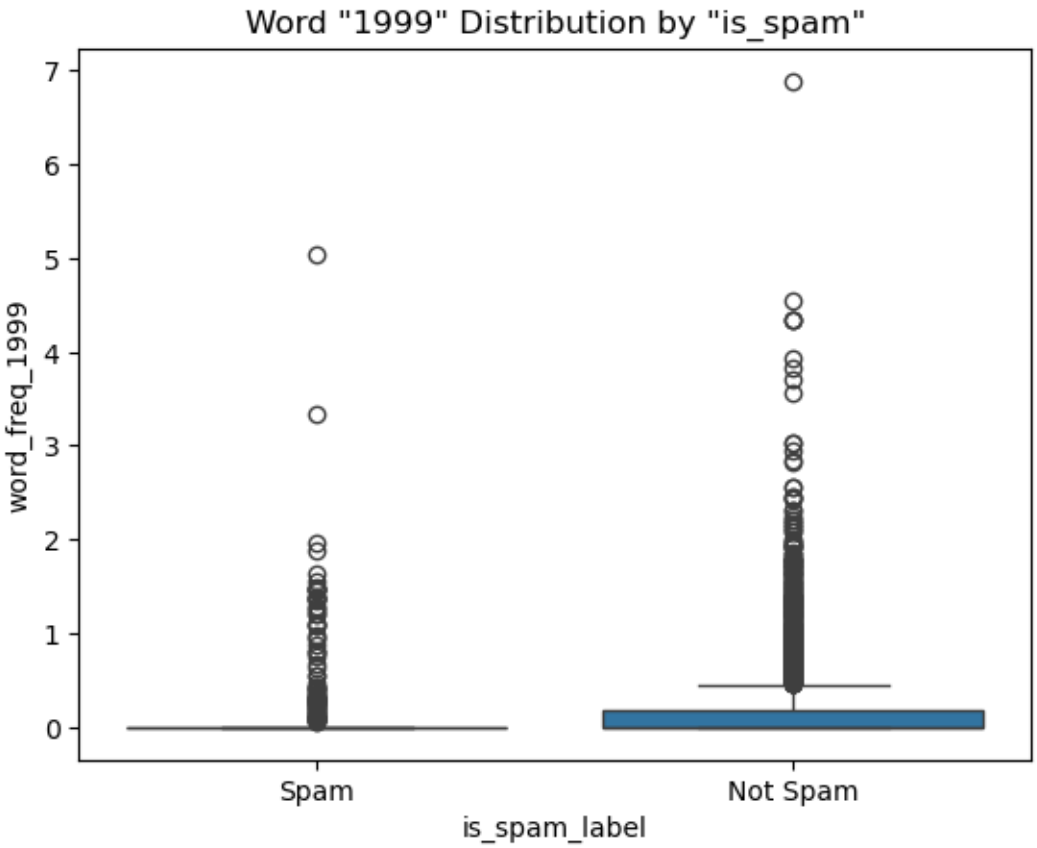
- **Objective:** Examine the word 'all' distribution across different spam or not epam emails.
- **Why this plot:** Boxplots show spread, median, and outliers effectively.
- **Learning:** The word 'all' is another example of a very common word in everyday life, indicating caution in using this variable to determine whether an email is spam or not, due to its high frequency of use in both cases.

[149]:
```
#20
avg_remove = dff.groupby("is_spam")["word_freq_1999"].mean().sort_values()
labels = avg_remove.index.map({0: "Not Spam", 1: "Spam"})
plt.barh(labels, avg_remove.values, color="lightgreen")
plt.title("Average by Word '1999'")
plt.xlabel("Average '1999' Word")
plt.show()
```



- **Objective:** Compare average word ‘1999’ usage in the spam or not spam emails.
- **Why this plot:** Horizontal bars are easier for categorical labels.
- **Learning:** The word ‘1999’ it’s frequently used in non-spam emails.

[150]:
```
dff['is_spam_label']=dff['is_spam'].map({0: "Not Spam", 1: "Spam"})
sns.boxplot(y="word_freq_1999", x="is_spam_label", data=dff)
plt.title('Word "1999" Distribution by "is_spam"')
plt.show()
```



- **Objective:** Examine the word '1999' distribution across different spam or not epam emails.
- **Why this plot:** Boxplots show spread, median, and outliers effectively.
- **Learning:** The word '1999' is normally found in spam emails too, but it is more frequently used in non-spam emails, in some cases excessively as shown by the outliers. Probably because '1999' refers to the year 1999, which makes its use more specific in the email context.

6 EDA analysis

- After the EDA analysis, it was noted that most emails with a high level of words in uppercase letters are a sign of spam, it was also observed that emails with a high presence of symbols, such as exclamation marks and dollar signs, tend to be spam as well. Words like 'business', 'money', and 'receive' also have a high presence in spam emails, showing a strong appeal for attention in these emails; as all these types of variables draw attention when opening an email, this kind of strategy is frequently used in spam.
- Words like 'george', 'hp', and 'hpl' are not common words in people's daily lives and draw little attention, it was observed that these word examples are not commonly used in spam, being a strong indicator that the presence of these words suggests the email may not be spam.
- **Key Observation:** Common words in people's daily lives, such as 'all', 'you', and 'your', are

frequently used in both types of emails, and in some cases even more so in non-spam emails, showing that everyday common words are not a strong indicator for concluding whether the email is spam or not.

- **Conclusions:** All variables present in this analysis are important for classifying the email as spam, but it is necessary to cross-reference the variables with each other to truly determine the type of email. An isolated variable may be a strong indicator, but it is not sufficient to conclude whether it is spam or not, since all variables are present in both types of email.

7 PCA

- **PCA (Principal Component Analysis) serves primarily to reduce the dimensionality of datasets with many correlated variables, transforming them into a smaller set of principal components that capture most of the original data's variance.**
- **First step in the PCA process is import the libraries.**

[151]:
```
from sklearn.decomposition import PCA
sns.set()
%matplotlib inline
```

[152]:
```
dff.head()
```

[152]:

	word_freq_make	word_freq_address	word_freq_all	word_freq_3d	\
0	0.00	0.64	0.64	0.0	
1	0.21	0.28	0.50	0.0	
2	0.06	0.00	0.71	0.0	
3	0.00	0.00	0.00	0.0	
4	0.00	0.00	0.00	0.0	

	word_freq_our	word_freq_over	word_freq_remove	word_freq_internet	\
0	0.32	0.00	0.00	0.00	
1	0.14	0.28	0.21	0.07	
2	1.23	0.19	0.19	0.12	
3	0.63	0.00	0.31	0.63	
4	0.63	0.00	0.31	0.63	

	word_freq_order	word_freq_mail	…	char_freq_(	char_freq_[	\
0	0.00	0.00	…	0.000	0.0	
1	0.00	0.94	…	0.132	0.0	
2	0.64	0.25	…	0.143	0.0	
3	0.31	0.63	…	0.137	0.0	
4	0.31	0.63	…	0.135	0.0	

	char_freq_!	char_freq_$	char_freq_#	capital_run_length_average	\
0	0.778	0.000	0.000	3.756	
1	0.372	0.180	0.048	5.114	
2	0.276	0.184	0.010	9.821	
3	0.137	0.000	0.000	3.537	

```
4        0.135        0.000        0.000                        3.537

   capital_run_length_longest  capital_run_length_total  is_spam  \
0                          61                       278        1
1                         101                      1028        1
2                         485                      2259        1
3                          40                       191        1
4                          40                       191        1

   is_spam_label
0           Spam
1           Spam
2           Spam
3           Spam
4           Spam

[5 rows x 59 columns]
```

[153]:
```
data=dff.drop(columns='is_spam_label')
```

- **The column "is_spam_label" was used in the EDA only to make the visualization of the plots easier, therefore, it is not necessary in the next process.**

[154]:
```
data
```

[154]:
```
      word_freq_make  word_freq_address  word_freq_all  word_freq_3d  \
0               0.00               0.64           0.64           0.0
1               0.21               0.28           0.50           0.0
2               0.06               0.00           0.71           0.0
3               0.00               0.00           0.00           0.0
4               0.00               0.00           0.00           0.0
…                …                  …              …              …
4213            0.31               0.00           0.62           0.0
4214            0.00               0.00           0.00           0.0
4215            0.30               0.00           0.30           0.0
4216            0.96               0.00           0.00           0.0
4217            0.00               0.00           0.65           0.0

      word_freq_our  word_freq_over  word_freq_remove  word_freq_internet  \
0              0.32            0.00              0.00                0.00
1              0.14            0.28              0.21                0.07
2              1.23            0.19              0.19                0.12
3              0.63            0.00              0.31                0.63
4              0.63            0.00              0.31                0.63
…               …               …                 …                   …
4213           0.00            0.31              0.00                0.00
4214           0.00            0.00              0.00                0.00
4215           0.00            0.00              0.00                0.00
```

```
4216           0.32            0.00              0.00                0.00
4217           0.00            0.00              0.00                0.00

      word_freq_order  word_freq_mail  …  char_freq_;  char_freq_(  \
0                0.00            0.00  …        0.000        0.000
1                0.00            0.94  …        0.000        0.132
2                0.64            0.25  …        0.010        0.143
3                0.31            0.63  …        0.000        0.137
4                0.31            0.63  …        0.000        0.135
…                …               …     …           …            …
4213             0.00            0.00  …        0.000        0.232
4214             0.00            0.00  …        0.000        0.000
4215             0.00            0.00  …        0.102        0.718
4216             0.00            0.00  …        0.000        0.057
4217             0.00            0.00  …        0.000        0.000

      char_freq_[  char_freq_!  char_freq_$  char_freq_#  \
0             0.0        0.778        0.000        0.000
1             0.0        0.372        0.180        0.048
2             0.0        0.276        0.184        0.010
3             0.0        0.137        0.000        0.000
4             0.0        0.135        0.000        0.000
…             …          …            …            …
4213          0.0        0.000        0.000        0.000
4214          0.0        0.353        0.000        0.000
4215          0.0        0.000        0.000        0.000
4216          0.0        0.000        0.000        0.000
4217          0.0        0.125        0.000        0.000

      capital_run_length_average  capital_run_length_longest  \
0                          3.756                          61
1                          5.114                         101
2                          9.821                         485
3                          3.537                          40
4                          3.537                          40
…                          …                              …
4213                       1.142                           3
4214                       1.555                           4
4215                       1.404                           6
4216                       1.147                           5
4217                       1.250                           5

      capital_run_length_total  is_spam
0                          278        1
1                         1028        1
2                         2259        1
3                          191        1
```

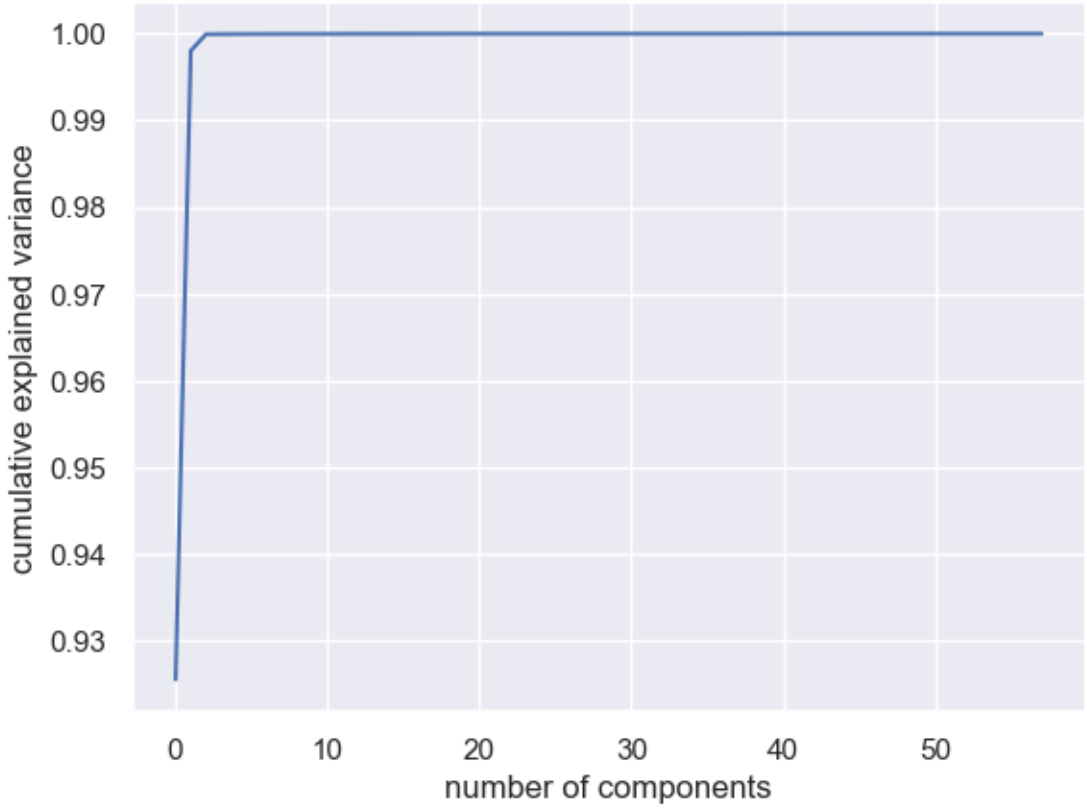
```
4                          191        1
…                          …        …
4213                        88        0
4214                        14        0
4215                       118        0
4216                        78        0
4217                        40        0

[4218 rows x 58 columns]
```

[155]:
```
type(data)
```

[155]:
```
pandas.core.frame.DataFrame
```

[156]:
```
stamp_no_label = data.drop(data["is_spam"])
pca = PCA().fit(stamp_no_label)
plt.plot(np.cumsum(pca.explained_variance_ratio_))
plt.xlabel('number of components')
plt.ylabel('cumulative explained variance')
```

[156]:
```
Text(0, 0.5, 'cumulative explained variance')
```



[157]:
```
pca.explained_variance_ratio_
```

[157]:
```
array([9.25484586e-01, 7.25400156e-02, 1.91382264e-03, 1.04963875e-05,
       7.69062018e-06, 5.97291852e-06, 4.31636952e-06, 3.13689800e-06,
       2.73708904e-06, 2.47138355e-06, 2.11289428e-06, 1.92650051e-06,
       1.73581493e-06, 1.65512112e-06, 1.50822928e-06, 1.34262470e-06,
       1.16831930e-06, 1.04523677e-06, 1.01200152e-06, 9.54967796e-07,
       7.63716467e-07, 7.25337847e-07, 6.53355328e-07, 5.75826609e-07,
       5.35783613e-07, 4.86591391e-07, 4.62375373e-07, 4.32085311e-07,
       4.21722161e-07, 4.11669653e-07, 3.64635193e-07, 3.53347167e-07,
       3.37795152e-07, 3.11455227e-07, 3.00397125e-07, 2.96432369e-07,
       2.71100313e-07, 2.20801707e-07, 2.13069460e-07, 2.09080573e-07,
       1.97887034e-07, 1.87868472e-07, 1.82118382e-07, 1.67347885e-07,
       1.58595929e-07, 1.35691436e-07, 1.28303734e-07, 1.23956819e-07,
       1.15802047e-07, 1.10328352e-07, 9.85158659e-08, 9.43668150e-08,
       8.76600110e-08, 6.61236750e-08, 5.08978874e-08, 2.50547432e-08,
       1.45589749e-08, 1.07327915e-09])
```

- **After analyzing the cumulative explained variance ratio, having the "is_spam" feature as a target, it is possible to see that only two components are needed to reach 99.5% of the variance in this dataframe (92.54 + 7.25 = 99.79).**
- **In this way, I will proceed with the process using the two components necessary to reach the minimum of 99.5% variance.**

[158]:
```
pca = PCA(2)
projected = pca.fit_transform(stamp_no_label)
```

[159]:
```
projected.shape
```

[159]:
```
(4216, 2)
```

[160]:
```
pca.explained_variance_ratio_*100
```

[160]:
```
array([92.54845857,  7.25400156])
```

[161]:
```
stamp_pca = pd.DataFrame(projected, columns=['C1', 'C2'])
```

[162]:
```
stamp_pca.head()
```

[162]:

	C1	C2
0	2012.215809	109.070052
1	-100.384673	4.052396
2	-100.384673	4.052394
3	-239.620221	1.486480
4	-184.158603	-18.779515

[163]:
```
stamp_pca.shape
```

[163]: (4216, 2)

[164]:
```
stamp_pca['is_spam']=data['is_spam']
```

[165]:
```
stamp_pca.head()
```

[165]:

	C1	C2	is_spam
0	2012.215809	109.070052	1
1	-100.384673	4.052396	1
2	-100.384673	4.052394	1
3	-239.620221	1.486480	1
4	-184.158603	-18.779515	1

- **With the PCA process completed, the final step is to split the data for machine learning model testing.**

[166]:
```
stamp_array=stamp_pca.values
```

[167]:
```
from sklearn.model_selection import train_test_split
X = stamp_array[:,0:2]
Y = stamp_array[:,2]
X_train, X_test, Y_train, Y_test = train_test_split(X, Y, test_size=0.20,␣
↪random_state=1)
```

[168]:
```
X_train.shape, X_test.shape, Y_train.shape, Y_test.shape
```

[168]: ((3372, 2), (844, 2), (3372,), (844,))

8 Curse of Dimensionality

The term "curse of dimensionality" refers to a problem that arises in data analysis when the dataset being studied has a large number of variables, which can lead to inaccuracies in machine learning models. Up to a certain point, increasing the number of features improves the model's accuracy and makes the data more useful for machine learning. However, beyond that point, the data becomes sparse, introducing a large amount of irrelevant information that decreases the model's accuracy.

For example, the more dimensions the variables in the analysis have, the greater the number of components present in the dataset. With each additional dimension, the number of components grows exponentially, making it harder to identify important data and reducing the overall performance of the machine learning models. Also, the curse of dimensionality increases the chance of overfitting. Overfitting ocurss when a model starts to memorize the aspect of the training model, and it lost the ability to genelarize.

One way to address this problem is by applying the PCA (Principal Component Analysis) method to reduce data dimensionality and focus on the most relevant information, improving the model's accuracy.

So, when preparing the data for the machine learning model, we first need to understand the dimensionality of the data present in the dataframe and apply the appropriate processing for

dimensionality reduction, if necessary, in order to improve the accuracy of the machine learning model used.

9 References:

Pydata.org. (2025). pandas.Index.map — pandas 2.3.3 documentation. [online] Available at: https://pandas.pydata.org/docs/reference/api/pandas.Index.map.html [Accessed 3 Dec. 2025].

- I have used “index.map” to plot the horizontal bars in EDA part.

Awan, A.A. (2024). A maldição da dimensionalidade no aprendizado de máquina: Desafios, impactos e soluções. [online] Datacamp.com. Available at: https://www.datacamp.com/pt/blog/curse-of-dimensionality-machine-learning.

- Together with the material provided in class, the link above helped me to develop a response about the curse of dimensionality.